



Development of an Uncertainty-Aware Motion Planner Combining Branch MPC With Machine Learning

MASTER'S THESIS

Author: Dennis Schmidt
Examiners: Prof. Dr. Wolfgang Hönig
Prof. Dr. Marc Toussaint
Supervisors: Mohamed-Khalil Bouzidi
Bojan Derajić
Prof. Dr. Wolfgang Hönig

Technische Universität Berlin
Faculty IV
Institute of Computer Engineering and Microelectronics
Intelligent Multi-Robot Coordination Lab

Continental AG
Continental AI Lab Berlin

January 2, 2025

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit eigenständig ohne Hilfe Dritter und ausschließlich unter Verwendung der aufgeführten Quellen und Hilfsmittel angefertigt habe. Alle Stellen die den benutzten Quellen und Hilfsmitteln unverändert oder sinngemäß entnommen sind, habe ich als solche kenntlich gemacht.

Sofern generative KI-Tools verwendet wurden, habe ich Produktnamen, Hersteller, die jeweils verwendete Softwareversion und die jeweiligen Einsatzzwecke (z.B. sprachliche Überprüfung und Verbesserung der Texte, systematische Recherche) benannt. Ich verantworte die Auswahl, die Übernahme und sämtliche Ergebnisse des von mir verwendeten KI-generierten Outputs vollumfänglich selbst.

Die Satzung zur Sicherung guter wissenschaftlicher Praxis an der TU Berlin vom 8. März 2017 habe ich zur Kenntnis genommen.

Ich erkläre weiterhin, dass ich die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt habe.

Berlin, 02.01.2025,

A handwritten signature in dark ink, appearing to read 'Edenst', written over a horizontal line.

Acknowledgments

First, I want to thank Mohamed-Khalil Bouzidi and Bojan Derajić for giving me the opportunity to join their team, build upon their work, and advance the state of the art in autonomous driving. The PhD students in the Continental AI Lab were always helpful and friendly, making it a very pleasant place to work. Special thanks go to my fellow master's thesis student at the AI Lab, Eyvaz Najafli, whose ideas provided me with new perspectives on arising problems.

I want to express my gratitude to Prof. Dr. Wolfgang Hönig for supervising my thesis on combining motion planning and learning-based approaches. Always ready to assist, share a joke, and broaden my horizon with insights on various methods, his support was invaluable to this work. His guidance on scientific and organizational questions, as well as on setting priorities, was greatly encouraging.

Moreover, I would like to thank Julien, Nan, Christoph, Julius, Achim, Max, Jan, Akmaral, Pia, Khaled, Denis, and Eckart for creating an enjoyable working atmosphere with their humor, positivity, and kindness. The environment at the Intelligent Multi-Robot Coordination Lab at TU Berlin was inspiring, motivating, and warm. Special thanks belong to Ilaria and Hongyou from the Learning and Intelligent Systems Lab at TU Berlin.

The work accomplished in this master's thesis would not have been possible without the support of my family. I thank them from the bottom of my heart for always being by my side and never letting me down.

Abstract

Traffic scenarios characterized by multimodal uncertainty pose significant challenges for autonomous vehicle motion planning. Current approaches predominantly rely on either end-to-end deep neural networks or Model Predictive Control. A promising method, Branch Model Predictive Contouring Control (BMPCC), employs a decision-postponing strategy to anticipate and plan for multiple scenarios, waiting until uncertainty resolves. While BMPCC provides safety guarantees, it is still limited in performance and conservativeness. This thesis integrates learning-based decision postponing with BMPCC to enable safer and more efficient driving in uncertain environments. A Monte Carlo Tree Search (MCTS) expert is utilized to generate near-optimal decision postponing sequences, which are subsequently used to train decision-postponing policies in a self-supervised manner. Evaluation results demonstrate that the learned policies improve safety, comfort, and overall performance compared to the rule-based decision-postponing method of the original BMPCC and conventional MPCC approaches, with particularly strong results in scenarios featuring a single traffic participant (TP). The MCTS expert consistently achieves superior performance, exhibiting a 12% higher success rate, 8% fewer emergency brakings, and 4% fewer crashes in complex scenarios involving up to three TPs compared to baseline methods. These findings contribute to the development of uncertainty-aware motion planners that effectively balance conservativeness and performance while ensuring safe and anticipatory driving, advancing the field of autonomous driving.

Zusammenfassung

Verkehrsszenarien, die durch multimodale Unsicherheit gekennzeichnet sind, stellen erhebliche Herausforderungen für die Bewegungsplanung autonomer Fahrzeuge dar. Aktuelle Ansätze basieren vorwiegend entweder auf End-to-End-Ansätzen oder Model Predictive Control. Eine vielversprechende Methode, Branch Model Predictive Contouring Control (BMPCC), nutzt eine Verzögerung beim Entscheidungsprozess, um mehrere Szenarien vorherzusehen und zu planen, bis sich die Unsicherheit im Fahrgeschehen aufgelöst hat. Obwohl BMPCC Sicherheitsgarantien bietet, ist die Methode weiterhin durch Einschränkungen in der Leistung und durch eine relativ hohe Vorsichtigkeit begrenzt. Diese Arbeit kombiniert lernbasierte Entscheidungsverzögerung mit BMPCC, um ein sichereres und effizienteres Fahren in unsicheren Umgebungen zu ermöglichen. Ein Monte Carlo Tree Search (MCTS)-Experte wird verwendet, um nahezu optimale Entscheidungsverzögerungen zu generieren, die anschließend für das automatisierte Lernen eines neuronalen Netzes genutzt werden. Die Evaluation zeigt, dass die gelernten Entscheidungsverzögerungen die Sicherheit, den Komfort und die Gesamtleistung im Vergleich zum regelbasierten Ansatz des ursprünglichen BMPCC und zu herkömmlichen MPCC-Ansätzen verbessern, insbesondere in Szenarien mit einem einzelnen Verkehrsteilnehmer. Der MCTS-Experte zeigt durchweg überlegene Leistung und erreicht eine 12% höhere Erfolgsquote, 8% weniger Notbremsungen und 4% weniger Kollisionen in komplexen Szenarien mit bis zu drei Verkehrsteilnehmern. Die Ergebnisse tragen zur Entwicklung von Bewegungsplanern in Verkehrsszenarien mit multimodaler Unsicherheit bei, wodurch das Feld des autonomen Fahrens weiter vorangetrieben wird.

Contents

Nomenclature	III
1 Introduction	1
2 Related Work	4
3 Background	7
3.1 Simulation Environment	7
3.1.1 Intelligent Driver Model	7
3.1.2 Reference Path Generation	8
3.2 MTR-BMPCC Framework	10
3.2.1 Multimodal Predictor	10
3.2.2 Branch Model Predictive Contouring Control	12
3.2.3 Scenario Selection	16
3.2.4 Decision Postponing	20
3.3 Reinforcement Learning	22
3.3.1 Markov Decision Processes	24
3.3.2 Bellman Equations	25
3.3.3 Policy Evaluation	28
3.3.4 Monte Carlo Prediction Algorithm	29
3.3.5 Temporal-Difference Prediction Algorithm	30
3.3.6 Monte Carlo Tree Search	31
3.4 Imitation Learning	38
3.4.1 Behavior Cloning	39
3.4.2 Dataset Aggregation	41
3.5 Supervised Learning	42
3.5.1 Neural Networks	43
3.5.2 Cross-Entropy Loss	43
3.5.3 Training	44
3.5.4 Class Weighting	46
3.5.5 L2 Regularization	46
3.5.6 Data Augmentation	47
4 Approach	48

4.1	Decision Postponing using Monte Carlo Tree Search	48
4.1.1	Rollout Generator	48
4.1.2	Monte Carlo Tree Search Expert	51
4.2	Learning Decision Postponing Policies	58
4.2.1	Feature Vector	58
4.2.2	Data Collection	60
4.2.3	Behavior Cloning for Decision Postponing	64
5	Evaluation	75
5.1	Multiple Traffic Participants	76
5.1.1	Quantitative Evaluation	76
5.1.2	Qualitative Evaluation	80
5.2	Single Traffic Participant	87
6	Discussion	91
6.1	Results	91
6.2	Limitations	91
6.3	Outlook	92
7	Conclusion	94
A	MCTS Analysis	95
B	Policy Training	105
B.1	Policy Variants	105
B.1.1	MLP256_1	106
B.1.2	MLP256_2	107
B.1.3	MLP256_4	109
B.1.4	MLP32_2	111
B.1.5	MLP32_3	113
B.2	Scaling Curves	115
B.2.1	Training and Test Loss	115
B.2.2	Training and Test Accuracy	116
B.2.3	Training Confusion Matrix	117
B.2.4	Test Confusion Matrix	120

Nomenclature

Acronyms

AV	Autonomous vehicle
TP	Traffic participant
DP	Decision postponing
ADP	Adaptive decision postponing
MPC	Model predictive control
MPCC	Model predictive contouring control
SMPCC	Scenario-based model predictive contouring control
BMPCC	Branch model predictive contouring control
MTR	Motion TRansformer
IDM	Intelligent driver model
RL	Reinforcement learning
IL	Imitation learning
MCTS	Monte Carlo tree search

System states

x	Position along the x-axis [m]
y	Position along the y-axis [m]
ψ	Yaw angle [rad]
v	Velocity [m/s]
a	Acceleration [m/s ²]
δ	Steering angle [rad]
θ	Virtual position along the projected path [m]

Control inputs

j	Jerk [m/s ³]
$\dot{\delta}$	Steering angle rate [rad/s]
$\dot{\theta}$	Virtual speed along the projected path [m/s]

1 Introduction

Autonomous driving is anticipated to significantly reduce car accidents, as human error remains a primary cause of crashes [1]. By minimizing the role of human error, autonomous vehicles promise enhanced safety and the opportunity for drivers and passengers to dedicate their time to other activities. The primary goal is to develop autonomous driving planners that ensure safe and comfortable driving while being efficient and avoiding overly conservative or aggressive behaviors, similar to human drivers.

Currently, there are two main approaches in autonomous driving planning: model-based methods like Model Predictive Control (MPC) and end-to-end approaches that utilize deep neural networks [2]. End-to-end methods can learn through interaction, as in reinforcement learning (RL), or from expert demonstrations, as in imitation learning (IL). Both methodologies have distinct advantages and disadvantages. MPC is capable of finding optimal control solutions and ensuring safety guarantees by embedding constraints into an optimization problem. This approach allows for the integration of domain knowledge via constraints, provides a preview of future behavior, and offers full explainability and control over the algorithm. However, MPC requires manual tuning, which can lead to behaviors that are either too aggressive or too conservative [3]. Conversely, IL and RL, often considered black-box methods, excel in learning complex expert behaviors from extensive datasets or through interaction in simulation environments. These approaches require minimal manual tuning and deliver faster computation times compared to MPC planners. However, they lack explainability and safety guarantees, particularly in edge cases [4, 5]. For example, ALVINN demonstrated the potential of imitation learning by successfully performing a road-following task [6].

Despite the advantages of MPC-based planners, they are more suitable for safety-critical applications due to their inherent safety guarantees. This makes them a more accepted and safer solution for addressing the challenges of autonomous driving in the long term [7]. However, conventional MPC methods typically do not account for multiple scenarios in traffic simultaneously and instead plan for the most probable outcome. Recent advancements in uncertainty-aware motion planning address this limitation by incorporating multi-modal uncertainties inherent in traffic scenarios to enable more anticipatory driving. A promising approach proposed in [8] combines a Motion TRansformer (MTR) with Branch Model Predictive Contouring Control (BMPCC) and introduces an adaptive decision-postponing (ADP) strategy. The adaptive decision-postponing strategy is a key

component of this approach, determining how long multiple scenarios should be considered before committing to a specific reaction. This decision-making process is governed by a branching time parameter that must be carefully selected in each timestep to balance safety, efficiency, and comfort. While this approach demonstrates promising results, it relies on a rule-based decision-postponing strategy that may not produce optimal or near-optimal branching times, particularly in scenarios involving high uncertainty and numerous traffic participants. Furthermore, the framework requires extensive manual tuning to avoid overly aggressive or conservative behavior.

Research in self-supervised learning of MPC parameters has demonstrated success in other domains, such as highly dynamic UAVs, achieving remarkable results [9]. The rise in popularity of reinforcement learning algorithms leveraging Monte Carlo Tree Search (MCTS) was catalyzed by the success of *AlphaGo* [10, 11, 12], which combined MCTS with deep neural networks to defeat the 18-time world champion in Go, Lee Sedol. Building on this concept, [13] demonstrated how RL-based policies could estimate velocity references for MPC in dense and uncertain traffic scenarios, showcasing the potential of integrating learning-based approaches with MPC to exploit the strengths of both methodologies.

This work builds on prior successful research by integrating self-supervised learning of MPC parameters with the MTR-BMPCC framework. Specifically, it introduces an innovative method for learning adaptive decision-postponing policies using near-optimal branching time sequences generated by an MCTS expert in a self-supervised manner. By doing so, the approach improves the safety and performance of the planner in complex traffic scenarios characterized by multi-modal uncertainty, while maintaining comfortable driving behavior and eliminating the need for manual tuning of the decision-postponing strategy.

The contributions of this work include the introduction of an MCTS expert to generate near-optimal adaptive decision-postponing sequences for the BMPCC framework, the application of self-supervised learning to train adaptive decision-postponing policies within the novel MTR-BMPCC framework, and experiments conducted with the expert and learned policy.

The document is structured as follows. Chapter 2 provides an overview of state-of-the-art uncertainty-aware planners in autonomous driving. Chapter 3 introduces the MTR-BMPCC framework in detail and discusses related approaches from reinforcement learning and imitation learning. Chapter 4 explains the simulation engine, which is fundamental for generating autonomous driving scenarios, describes the MCTS expert, and outlines the methodology for learning adaptive decision-postponing policies. Chapter 5 presents quantitative and qualitative evaluations of the learned adaptive decision-postponing policies and the MCTS expert, compared to the original rule-based branching

time estimator and a conventional MPC planner, in scenarios involving up to three traffic participants. Finally, Chapter 6 discusses the strengths and limitations of the proposed approach, explores potential alternatives, and suggests directions for future research.

2 Related Work

Model predictive control has been applied to autonomous driving as a unified optimization framework for motion planning in [7]. The framework allows encoding requirements such as reference and lane paths, obstacle avoidance, and comfortable driving behavior as constraints, offering safety guarantees based on the formulated constraints. Furthermore, the algorithm features full transparency. Achievements include real-time generation of high-quality trajectories for tasks such as lane keeping, lane changes, ramp merging, and intersection crossing. However, this approach has several limitations. It plans for one scenario only, does not account for uncertainty or multimodal uncertainty in traffic participants' future behavior, and is subject to the *frozen robot problem* [14]. The frozen robot problem arises in complex traffic scenarios with multiple participants, where feasible solutions may exist, but the algorithm fails to find them, resulting in a standstill.

Robust MPC [15] addresses multimodal uncertainty by considering multiple scenarios, ensuring collision avoidance under all circumstances. However, this approach can also suffer from the frozen robot problem due to overly conservative behavior.

Stochastic MPC [16] builds on robust MPC by introducing chance constraints, which allow the system to satisfy constraints with a defined probability rather than under all possible scenarios. This approach reduces conservativeness and enables more anticipatory driving, thereby mitigating the frozen robot problem. Nevertheless, directly evaluating chance constraints is computationally infeasible, limiting the direct use of the approach.

A subapproach of stochastic MPC, scenario-based model predictive contouring control (SCMPCC) [17], constructs a scenario tree based on multiple possible scenarios and their probabilities. While it guarantees constraint satisfaction for all sampled scenarios, the computational cost grows rapidly with the number of scenarios, limiting its real-time applicability.

Stochastic and scenario-based MPC approaches can still exert conservative behavior when facing high multimodal uncertainty because they plan to react to multiple scenarios for the complete planning horizon which often is not necessary due to resolving uncertainty. To solve this, [8] introduces Branch Model Predictive Contouring Control (BM-PCC). BMPCC includes an adaptive decision-postponing strategy that plans for multiple modes only up to a future point in time, referred to as the branching time. This method incorporates an interaction-aware prediction module based on a Motion TTransformer

(MTR), which integrates multimodal uncertainty from traffic participants. Additionally, a novel scenario selection method renders the framework real-time capable for 2-3 traffic participants at a planning frequency of approximately 10 Hz. BMPCC improves performance and reduces conservativeness while maintaining safety and avoiding the frozen robot problem.

Despite these advancements, BMPCC has limitations. The adaptive decision-postponing strategy calculates branching times using a heuristic based on the Bhattacharyya Distance and a hand-tuned uncertainty threshold to evaluate the similarity of risk-relevant traffic participant's predictions. This rule-based method can lead to oscillations in branching times, resulting in overly conservative or overly reactive behavior depending on a higher or lower branching time. Moreover, it focuses on the uncertainty in predictions without considering its impact on the ego vehicle, which may degrade planner performance in clear traffic scenes.

Extensive research has been conducted to integrate learning with MPC, aiming to leverage the strengths of both approaches. [9] proposes a general framework for merging learning and MPC, demonstrated in UAV applications. A neural network is trained using an expert in a self-supervised manner to learn the timing required to navigate through a moving pendulum at high speeds. The training dataset is generated by aggregating trajectories produced by an MPC, which determines the optimal fly-through times. Once trained, the neural network estimates these fly-through times online, using the states of both the pendulum and itself, and passes the parameter to the MPC. This approach outperforms conventional MPC and RL methods where the desired fly-through time is not explicitly known. However, the process of generating data using MPC is computationally intensive. Additionally, the experimental results rely on ground truth motion capture data and do not account for uncertainty.

Another work merging learning and MPC is presented in [13]. It investigates learning interaction-aware continuous velocity references using reinforcement learning. This method conditions on the state of both the autonomous vehicle and traffic participants, optimizing for driving time and collision avoidance. By feeding the estimated velocity reference into an MPC, this approach achieves high success rates and zero collisions in dense traffic with uncooperative drivers, showing that planner performance can be improved via learned policies while ensuring safety by employing MPC. However, while the trained policy estimates driver cooperativeness, it does not consider the multimodal uncertainty of traffic participants, which is crucial for anticipatory driving.

This work addresses the problem of merging learning and MPC to balance conservativeness and aggressiveness while ensuring safety, efficiency, and comfort. Existing approaches either do not account for multimodal uncertainty and multiple scenarios, leading to the frozen robot problem, or plan for all possible scenarios, resulting in exceedingly

conservative behavior. This work is building on [8] which addressed these limitations, and extends it by introducing learning-based adaptive decision-postponing policies and an MCTS expert that overcome limitations of hand-tuning and suboptimal branching times inherent to the prior rule-based adaptive decision postponing approach. This allows to improve conservativeness and planner performance while maintaining safety and comfort. The proposed approach outperforms both the baseline method in [8] and conventional MPCC, as demonstrated by the results presented in Chapter 5.

3 Background

3.1 Simulation Environment

3.1.1 Intelligent Driver Model

The Intelligent Driver Model (IDM) [18] is used within the MTR-BMPCC framework as a base to create trajectories for the traffic participants (TPs) surrounding the ego vehicle. It is a widespread and popular microscopic traffic flow model and simulates the longitudinal dynamics of vehicles, including acceleration and braking deceleration, providing realistic behaviors for modeling traffic scenarios. This model is particularly useful for generating trajectories for multiple traffic participants, as it accounts for the interactions between a vehicle and its leading vehicle.

The acceleration $\dot{v}$ of the vehicle based on its current speed v , as well as the current gap to the leading vehicle s is described in Eq. 3.1, whereas Eq. 3.2 determines the desired distance to the leading vehicle s^* . The parameters v_0 , T , s_0 , a , b , δ define driving styles and vehicle types, and are explained in Table 3.1.

$$\dot{v} = a \left[1 - \left(\frac{v}{v_0} \right)^\delta - \left(\frac{s^*(v, \Delta v)}{s} \right)^2 \right] \quad (3.1)$$

$$s^*(v, \Delta v) = s_0 + \max \left[0, \left(vT + \frac{v\Delta v}{2\sqrt{ab}} \right) \right] \quad (3.2)$$

Using the IDM equations, the state variables of the vehicle, such as speed v , front bumper position x , and the gap to the leading vehicle s can be updated with the time interval Δt through numerical integration. The leading vehicle is characterized by the front bumper position x_l and length L_l . The following integration formulas [19] can be used:

$$v(t + \Delta t) = v(t) + \dot{v}\Delta t, \quad (3.3)$$

$$x(t + \Delta t) = x(t) + v(t)\Delta t + \frac{1}{2}\dot{v}(\Delta t)^2, \quad (3.4)$$

$$s(t + \Delta t) = x_l(t + \Delta t) - x(t + \Delta t) - L_l. \quad (3.5)$$

The gap s may be set to a large value if no leading vehicle is present.

Table 3.1: IDM parameters with typical values for cars and trucks [19].

Parameter [Unit]	Symbol	Car	Truck
Desired speed [km/h]	v_0	120	80
Desired time headway [s]	T	1.5	1.7
Minimum gap [m]	s_0	2.0	2.0
Maximum acceleration [m/s^2]	a	0.3	0.3
Comfortable deceleration [m/s^2]	b	3.0	2.0
Acceleration exponent [–]	δ	4	4

3.1.2 Reference Path Generation



Figure 3.1: An exemplary map from the WOMD, showing road geometry, including lane markings and road lines (light and dark gray), as well as road edges (black). The x- and y-axes indicate the dimensions of the map in meters. Multiple intersections are present within one map.

The MTR-BMPCC framework creates reference lanes for the ego vehicle and traffic participants (TPs) using road maps provided by the Waymo Open Motion Dataset (WOMD) [20]. These maps contain vectorized lane geometries defined by polylines representing

lane markings, road boundaries, and fixed object trajectories for 103,354 scenes. An example map is shown in Fig. 3.1.

Reference lanes are constructed by connecting available lane segments provided by the WOMD as seen in Fig. 3.3. These lanes are represented by reference lane points $\mathbf{p}_i = (x_i, y_i)$, which serve as the basis for generating a continuous reference path P_{ref} . This path is modeled using Bézier curves to ensure smoothness and continuity. The reference path is parameterized by an arc length parameter $\theta \in [0, \theta_{\text{max}}]$, mapping it to a tuple that includes the reference position, road boundaries, and orientation:

$$P_{\text{ref}}(\theta) = (x_{\text{ref}}(\theta), y_{\text{ref}}(\theta), d_{\text{lb}}(\theta), d_{\text{rb}}(\theta), \psi_{\text{ref}}(\theta)), \quad (3.6)$$

where d_{lb} and d_{rb} are the distances to the left and right road boundaries, and ψ_{ref} is the orientation angle of the path. For the TP reference path, the arc length θ represents the progress along the virtual path generated by the IDM model, effectively linking the longitudinal dynamics of the TP to its position on the reference path and providing a ground truth trajectory.

To parameterize $P_{\text{ref}}(\theta)$, a Bézier curve of degree $n = 5$ is employed. This curve is defined by control points $\mathbf{c}_j \in \mathbb{R}^2$ ($j = 0, \dots, n$) and a blending function:

$$\mathbf{P}(\theta) = \sum_{j=0}^n B_j^n(\theta) \mathbf{c}_j, \quad \theta \in [0, 1], \quad (3.7)$$

where the Bernstein polynomials $B_j^n(\theta)$ are given by:

$$B_j^n(\theta) = \binom{n}{j} \theta^j (1 - \theta)^{n-j}. \quad (3.8)$$

The Bézier curve function $\mathbf{P}(\theta)$ directly underpins the parameterized reference path $P_{\text{ref}}(\theta)$. By fitting the Bézier curve to the discrete reference lane points $\mathbf{p}_i$ through an optimization process that minimizes fitting error, the control points $\mathbf{c}_j$ are derived. The resulting curve ensures smooth transitions between the discrete points and enables computation of essential path properties such as curvature κ and orientation ψ :

$$\kappa(\theta) = \frac{\dot{x}\ddot{y} - \dot{y}\ddot{x}}{(\dot{x}^2 + \dot{y}^2)^{3/2}}, \quad (3.9)$$

$$\psi(\theta) = \arctan 2(\dot{y}, \dot{x}), \quad (3.10)$$

where $\dot{x}$ and $\ddot{x}$ represent the first and second derivatives of $x_{\text{ref}}(\theta)$ with respect to θ , and similarly for $y_{\text{ref}}(\theta)$.

3.2 MTR-BMPCC Framework

This section covers the uncertainty-aware MTR-BMPCC motion planning framework, adopting the notation introduced in [8]. An overview of the modules comprising the baseline approach is given in Fig. 3.2.

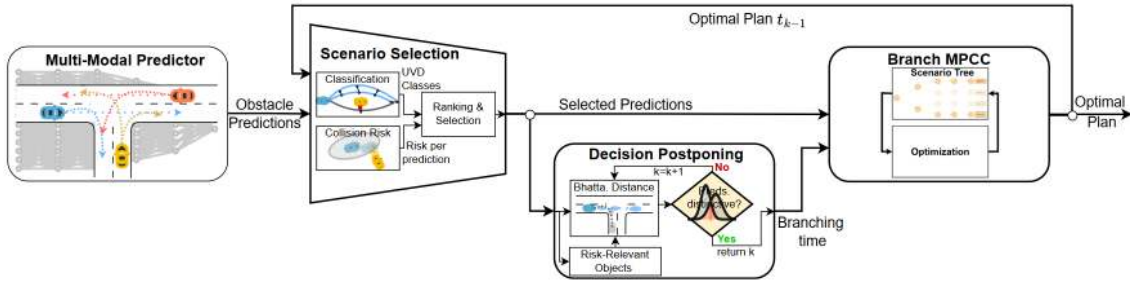


Figure 3.2: Overview of the MTR-BMPCC framework. [8]

3.2.1 Multimodal Predictor

To effectively incorporate multimodal uncertainty from surrounding traffic participants (TPs) in complex traffic scenarios, the MTR-BMPCC framework employs a Motion Transformer (MTR) [21] as a core component. The MTR has demonstrated state-of-the-art performance on large-scale benchmarks for motion forecasting, such as the WOMD.

The MTR framework addresses motion prediction as a multimodal optimization problem, where the goal is to model the future trajectories of traffic participants (TPs) under uncertain and dynamic conditions. The framework combines global intention localization, which identifies the most probable movement intents of an agent, with local movement refinement to ensure trajectory accuracy. By leveraging large-scale datasets such as the WOMD, the model is trained on diverse traffic scenarios with detailed annotations of map information, TP trajectories, and multimodal behaviors. The training process minimizes prediction errors across multiple future trajectories within the WOMD.

The MTR framework consists of four main components: a Transformer-based Context Encoder, a Dense Future Prediction Module, a Dynamic Map Collection Module, and a Motion Decoder Network. These components work together to process spatiotemporal data from the environment, capture dependencies in TP behavior, and generate accurate multimodal predictions. The Transformer-based Context Encoder extracts temporal and

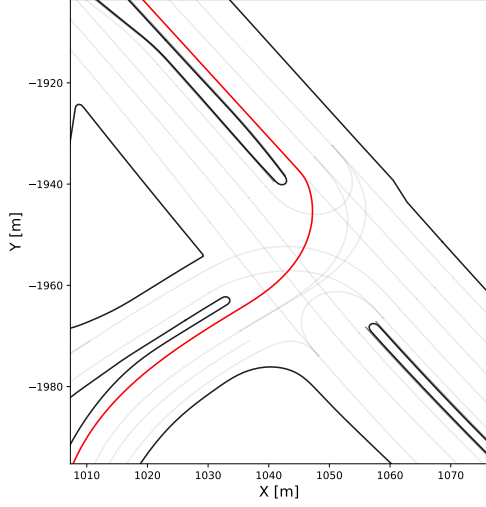


Figure 3.3: Example of a reference path generated based on the lane markings provided in the WOMD. The x- and y-axes are given in meters, and the reference path is represented by a tuple of x and y coordinates.

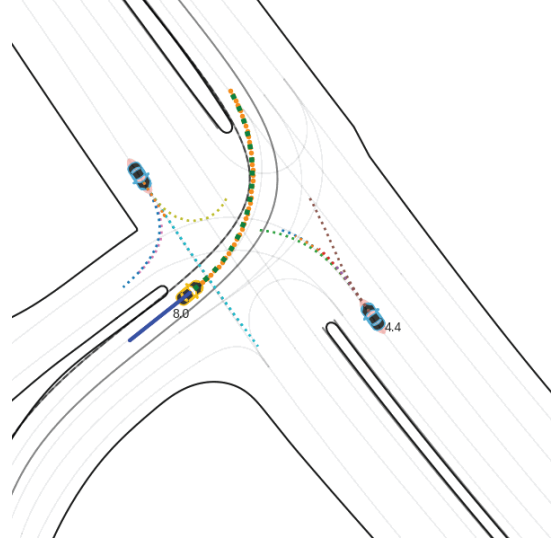


Figure 3.4: Visualization of the multi-modal predictions generated by the Motion Transformer. The yellow car represents the ego vehicle, and its planned trajectories are also shown. The predicted future trajectory of a TP o , represented in blue, and modes $m \in \mathcal{M}$, represented by different colors, illustrate distinct possible future directions for the TPs.

spatial relationships from TP history and map data, while the Dense Future Prediction Module focuses on generating probabilistic future trajectories. The Dynamic Map Collection Module dynamically integrates map features, and the Motion Decoder Network refines the final trajectories for prediction.

The input to the MTR includes a vectorized map M_{in} from the WOMD containing road attributes such as lanes, road boundaries, and polylines to represent road geometries. Furthermore, the history of each TP, including prior position, heading angle, and velocity, is considered. The MTR uses the reference path $P_{ref}(\theta)$ introduced in Eq. 3.6, extracting the position $x_{ref}(\theta)$, $y_{ref}(\theta)$, and heading angle $\psi_{ref}(\theta)$ for each TP. To ensure temporal context, the historical data for each TP is stored in a buffer, which maintains a sequence of past positions, headings, and velocities.

To capture the multimodal nature of future trajectories, the MTR employs a Gaussian Mixture Model (GMM). For each TP o and future timestep k , the GMM predicts a probability density function p_k^o , which is expressed as a weighted sum of Gaussian components:

$$p_k^o = \sum_{m \in \mathcal{M}} \pi_m^o \mathcal{N}_{k,m}^o(\boldsymbol{\mu}_{k,m}^o, \boldsymbol{\Sigma}_{k,m}^o), \quad (3.11)$$

where:

- π_m^o is the probability of motion mode m ,
- $\mathcal{N}_{k,m}^o(\boldsymbol{\mu}_{k,m}^o, \boldsymbol{\Sigma}_{k,m}^o)$ is the Gaussian component representing mode m , parameterized by mean $\boldsymbol{\mu}_{k,m}^o$ and covariance $\boldsymbol{\Sigma}_{k,m}^o$,
- $\mathcal{M}$ is the set of all possible motion modes, i.e., predicted trajectories for a TP.

The GMM aggregates predictions for all potential motion modes, with each mode $m \in \mathcal{M}$ represented by a single Gaussian distribution $\mathcal{N}_{k,m}^o(\boldsymbol{\mu}_{k,m}^o, \boldsymbol{\Sigma}_{k,m}^o)$. The mixture components π_m^o ensure that the sum of probabilities for all modes equals one.

The predicted state for each TP o in motion mode m at timestep k is represented as a vector $\mathbf{o}_{k,m}$, which includes the x-axis position $x_{k,m}^o$, the y-axis position $y_{k,m}^o$, the heading angle $\psi_{k,m}^o$, and the velocity $v_{k,m}^o$, expressed as:

$$\mathbf{o}_{k,m} = \begin{bmatrix} x_{k,m}^o \\ y_{k,m}^o \\ \psi_{k,m}^o \\ v_{k,m}^o \end{bmatrix}. \quad (3.12)$$

A visualization of different motion modes for multiple TPs generated by the MTR is shown in Figure 3.4.

3.2.2 Branch Model Predictive Contouring Control

The Branch Model Predictive Contouring Controller (BMPCC) is a trajectory planning approach that integrates vehicle dynamics with a cost function to solve an optimization problem at each timestep. BMPCC employs a scenario tree and adaptive branching time to address uncertainties, enabling the consideration of multiple potential traffic scenarios. It builds upon previous work on Model Predictive Contouring Control (MPCC) [24] and Scenario-based Model Predictive Control (SMPCC) [17]. A visualization of the multimodal uncertainty in a traffic scene and the corresponding plans produced by the BMPCC can be seen in Fig. 3.4.

Vehicle Dynamics

The BMPCC models the ego vehicle's behavior using the kinematic bicycle model as its foundation. This model defines the vehicle dynamics in a state-space representation. An

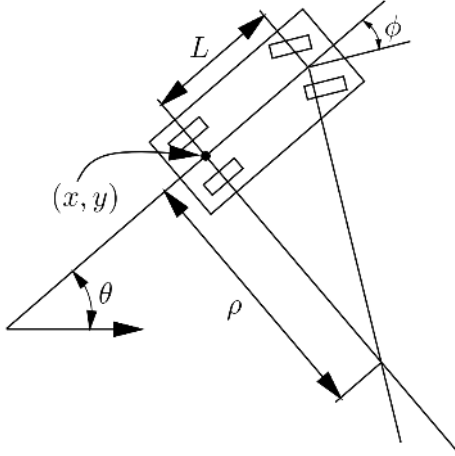


Figure 3.5: Schematic representation of a vehicle based on the kinematic bicycle model and the notation from [22]. In this illustration, the vehicle's position is denoted by (x, y) , and its heading angle by θ . The wheelbase is L , and the steering angle is ϕ . The turning radius is represented by ρ .

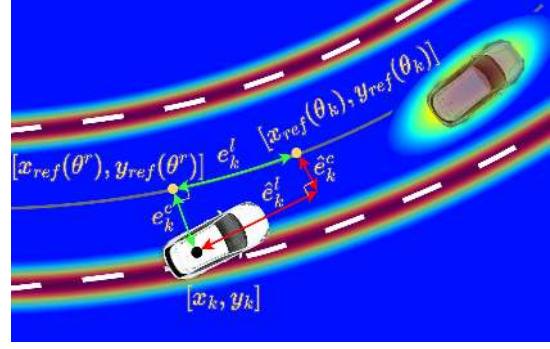


Figure 3.6: Heatmap illustrating the potential fields used in the cost function of the BMPCC [23]. The vehicle's position is denoted as $[x_k, y_k]$, and the next reference path waypoint is defined by $[x_{\text{ref}}(\theta_k), y_{\text{ref}}(\theta_k)]$ along the desired trajectory. The approximated lag and contouring errors are represented as $\hat{e}_k^l$ and $\hat{e}_k^c$, respectively, while e_k^l and e_k^c denote the actual lag and contouring errors. The heatmap indicates high costs near lane markings, road boundaries, and other TPs in dark red and low costs in blue and green.

illustration of the vehicle model is provided in Fig. 3.5. The continuous-time dynamics are expressed as:

$$\dot{\mathbf{z}}(t) = f(\mathbf{z}(t), \mathbf{u}(t)), \quad (3.13)$$

where $\mathbf{z}(t) \in \mathbb{R}^{n_z}$ represents the vehicle state with dimension n_z , and $\mathbf{u}(t) \in \mathbb{R}^{n_u}$ denotes the control inputs with dimension n_u . The dynamics are discretized using the sampling time Δt and can be written as:

$$\mathbf{z}_{k+1} = f(\mathbf{z}_k, \mathbf{u}_k), \quad (3.14)$$

where k indicates the discrete time step. The vehicle dynamics $\dot{\mathbf{z}}_k \in \mathbb{R}^{n_z}$, state vector $\mathbf{z}_k \in \mathbb{R}^{n_z}$, and control vector $\mathbf{u}_k \in \mathbb{R}^{n_u}$ are defined as follows:

$$\dot{\mathbf{z}}_k = \begin{bmatrix} v_k \cos(\psi_k) \\ v_k \sin(\psi_k) \\ \frac{1}{l} v_k \tan(\delta_k) \\ a_k \\ j_k \\ \dot{\delta}_k \\ \dot{\theta}_k \end{bmatrix}, \quad \mathbf{z}_k = \begin{bmatrix} x_k \\ y_k \\ \psi_k \\ v_k \\ a_k \\ \delta_k \\ \theta_k \end{bmatrix}, \quad \mathbf{u}_k = \begin{bmatrix} j_k \\ \dot{\delta}_k \\ \dot{\theta}_k \end{bmatrix}. \quad (3.15)$$

Here:

- x_k, y_k : Vehicle's position coordinates.
- ψ_k : Vehicle's heading angle.
- v_k : Longitudinal velocity.
- a_k : Acceleration.
- δ_k : Steering angle.
- j_k : Vehicle jerk.
- $\dot{\delta}_k$: Steering angle rate.
- $\dot{\theta}_k$: Virtual speed projected onto the path.
- l : Vehicle wheelbase.
- θ_k : Virtual position on the projected path.

Cost Function

A key component of BMPCC is its cost function, designed to achieve the following objectives:

- Maximize the vehicle's velocity and distance traveled, captured by the cost term J_k^v .
- Minimize contouring error $\hat{e}_k^c$ and lag error $\hat{e}_k^l$, represented by J_k^e .
- Penalize excessive control effort, represented by J_k^u .
- Incorporate potential field costs J_k^p based on the Potential Field Method in [25], accounting for road boundaries, lane markings, and other traffic participants under

uncertainty.

A heatmap of the potential fields, along with contouring and lag errors, is shown in Fig. 3.6. The errors are defined as:

$$\hat{\mathbf{e}}_k = \begin{bmatrix} \hat{e}_k^c \\ \hat{e}_k^l \end{bmatrix} = \begin{bmatrix} \sin(\psi_{\text{ref}}(\theta_k)) & -\cos(\psi_{\text{ref}}(\theta_k)) \\ -\cos(\psi_{\text{ref}}(\theta_k)) & -\sin(\psi_{\text{ref}}(\theta_k)) \end{bmatrix} \begin{bmatrix} x_k - x_{\text{ref}}(\theta_k) \\ y_k - y_{\text{ref}}(\theta_k) \end{bmatrix}. \quad (3.16)$$

The total running cost at each timestep k is expressed as the sum of all cost terms:

$$J_k = J_k^e + J_k^u + J_k^v + J_k^p \quad (3.17)$$

$$= \hat{\mathbf{e}}_k^\top \mathbf{Q} \hat{\mathbf{e}}_k + \mathbf{u}_k^\top \mathbf{R} \mathbf{u}_k - q_v v_k^p + J_k^p, \quad (3.18)$$

where $\mathbf{Q} \in \mathbb{R}^{2 \times 2}$ and $\mathbf{R} \in \mathbb{R}^{2 \times 2}$ are weight matrices for contouring and lag errors, and q_v is a weight for the velocity cost.

The potential field cost J_k^p is formulated as:

$$J_k^p = q_{ob} \sum_{o=0}^{O-1} \exp \left(- \left(\frac{\Delta x_k^o}{l^o} \right)^2 - \left(\frac{\Delta y_k^o}{w^o} \right)^2 \right) + q_{lm} \sum_{l=0}^{L-1} \exp \left(- \left(\frac{d_{lm}^l - \hat{e}_k^c(\theta_k)}{\sigma} \right)^2 \right), \quad (3.19)$$

where:

- $\Delta x_k^i, \Delta y_k^i$: Distances to nearby obstacles.
- l^i, w^i : Length and width of detected obstacles under unimodal uncertainty.
- d_{lm}^l : Signed distance from the reference path to the adjacent lane marker l .
- σ : Scaling factor influencing potential field decay.
- q_{ob}, q_{lm} : Weights for obstacle and lane marker costs.

Scenario Tree Optimization

The BMPCC constructs a scenario tree to manage uncertainties in the traffic environment. Each branch in the tree represents a potential traffic scenario, with N nodes spanning the prediction horizon. For simplicity, consider two branches with a single branching point, as illustrated in Fig. 3.7. The total number of scenarios included in the scenario tree is denoted by $|\mathcal{S}|$. For each scenario $s \in \mathcal{S}$ and timestep k , the ego vehicle maintains state variables $\mathbf{z}_{k,s} \in \mathbb{R}^{n_z}$ and control inputs $\mathbf{u}_{k,s} \in \mathbb{R}^{n_u}$. The set I aggregates tuples combining

a certain timestep with a specific scenario (k, s) . The scenario selection process used to extract $\mathcal{S}$ is introduced in Sec. 3.2.3.

The complete optimization problem is defined as:

$$\min_{\mathbf{z}_{k,s}, \mathbf{u}_{k,s}, \forall (k,s) \in I} \sum_{s \in \mathcal{S}} \omega_s \sum_{k=0}^{N-1} J_k(\mathbf{z}_{k,s}, \mathbf{u}_{k,s}) \quad (3.20)$$

$$\text{s.t. } \mathbf{z}_{k+1,s} = f(\mathbf{z}_{k,s}, \mathbf{u}_{k,s}), \quad \forall (k, s) \in I, \quad (3.21)$$

$$\mathbf{z}_{0,s} = \mathbf{z}(0), \quad \forall s \in \mathcal{S}, \quad (3.22)$$

$$\mathbf{z}_{k,s} \in \mathcal{Z}(\mathbf{o}_{k,s}), \quad \mathbf{u}_{k,s} \in \mathcal{U}, \quad \forall (k, s) \in I, \quad (3.23)$$

$$\mathbf{u}_{k,i} = \mathbf{u}_{k,j}, \quad \forall (i, j) \in \mathcal{S}, \forall k \in [0, b], \quad (3.24)$$

where the optimization minimizes the total weighted cost across scenarios, with scenario weights ω_s determined during the scenario selection process. State constraints $\mathcal{Z}$ reflect interactions with traffic participants $\mathbf{o}_{k,s}$, and actuator constraints $\mathcal{U}$ limit control actions as:

$$j_k \in [j_{\min}, j_{\max}], \quad \delta_k \in [\delta_{\min}, \delta_{\max}] \quad \dot{\delta}_k \in [\dot{\delta}_{\min}, \dot{\delta}_{\max}]. \quad (3.25)$$

Similar to [26] the lateral and longitudinal acceleration is limited. The non-anticipatory constraint in Eq. 3.24 ensures identical control inputs and states across all scenarios up to timestep $k = b$ and branching time t_b . This allows the vehicle to follow a single consensus plan until t_b , after which the constraint is relaxed. At t_b , the ego vehicle commits to the most probable scenario, essentially postponing its decision up to this point in time where the uncertainty in the traffic scene is expected to resolve. At each timestep, the first control input $\mathbf{u}_0 = \mathbf{u}_{0,i} = \mathbf{u}_{0,j}$ is applied to ensure optimal control. The parameters used for the BMPCC are listed in Tab. 3.2

3.2.3 Scenario Selection

The scenario selection process is a vital part of the MTR-BMPCC framework, enabling efficient handling of multimodal predictions from traffic participants (TPs). By selecting a subset of scenarios $\mathcal{S}$ from the total set of available scenarios $\mathcal{M}$, where

$$\mathcal{S} \subseteq \mathcal{M}, \quad |\mathcal{S}| \leq |\mathcal{M}|, \quad (3.26)$$

this process ensures the framework remains lightweight and real-time capable while providing valuable information for the construction of a scenario tree. This selection process

Table 3.2: BMPCC parameters with values, units, and descriptions. The elements of the diagonal matrices $\mathbf{Q}$ and $\mathbf{R}$ are listed separately.

Parameter [Unit]	Value	Description
Δt [s]	0.1	Sampling time
N [-]	40	Prediction horizon
$v_{\max}$ [m/s]	8	Maximum speed
$a_{\max}^x$ [m/s ²]	3.0	Maximum longitudinal acceleration
$a_{\min}^x$ [m/s ²]	-6.0	Minimum longitudinal acceleration
$a_{\max}^y$ [m/s ²]	4.5	Maximum lateral acceleration
$\delta_{\max}$ [rad]	$\pi/6$	Maximum steering angle
$\dot{\delta}_{\max}$ [rad/s]	0.5	Maximum steering rate
$j_{\max}$ [m/s ³]	3.0	Maximum jerk
l [m]	3.0	Vehicle wheelbase
q_v [-]	2	Weight for speed cost
q_{ob} [-]	80	Weight for obstacle cost
q_{lm} [-]	3	Weight for lane marker cost
$\mathbf{Q}_{11}$ [-]	0.5	Weight for contouring error
$\mathbf{Q}_{22}$ [-]	200	Weight for Lag error
$\mathbf{R}_{11}$ [-]	0.5	Weight for jerk control effort
$\mathbf{R}_{22}$ [-]	135.0	Weight for steering rate control effort
$\mathbf{R}_{33}$ [-]	0.0	Weight for path-following speed control effort
S [-]	2	Number of scenarios ($S = 1$: standard MPCC)

incorporates two key stages: scenario clustering and scenario ranking. The scenario selection process utilizes the optimal plan from the last timestep, τ_{k-1}^* , as an approximation for the yet-to-be-determined optimal plan of the current timestep, τ_k^* .

Mode Clustering

The goal of scenario clustering is to group similar predicted trajectories of TPs into clusters, as trajectories within the same cluster tend to have a similar effect on the ego vehicle's planner. By identifying and clustering these similar trajectories, redundant information is eliminated, and the computational burden is reduced. This concept is illustrated in Fig. 3.8, where four predicted scenarios are clustered into two distinct groups based on their influence on the ego vehicle's behavior, either causing a leading to a braking or accelerating maneuver.

The output of the clustering procedure is a partition of all available scenarios, where all scenarios are grouped into scenario clusters. This is expressed as:

$$\bigcup_{s \in \mathcal{S}} C_s = \mathcal{M}, \quad (3.27)$$

where C_s represents a cluster of scenarios for each $s \in \mathcal{S}$.

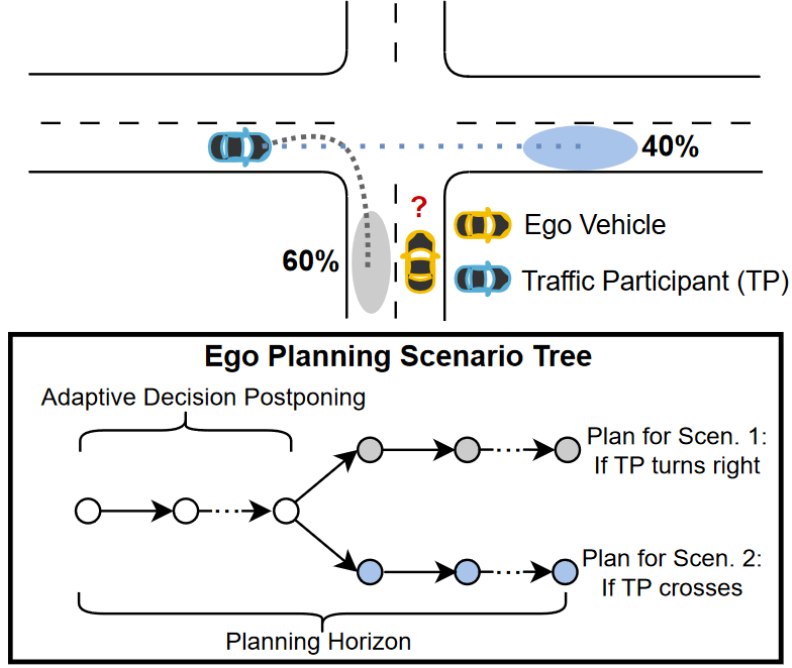


Figure 3.7: Illustration of a traffic scene and the corresponding scenario tree generated by the BMPCC framework [8]. The upper part depicts a potential traffic situation where the ego vehicle predicts two possible behaviors of a TP: turning right (40% probability) or crossing the ego's path (60% probability). The lower part visualizes the scenario tree, where decision postponing is employed. The tree structure allows the ego vehicle to maintain a consensus plan and defer final decisions until more information becomes available at branching time t_b . Each node represents the tuple $(\mathbf{z}_{k,s}, \mathbf{u}_{k,s})$.

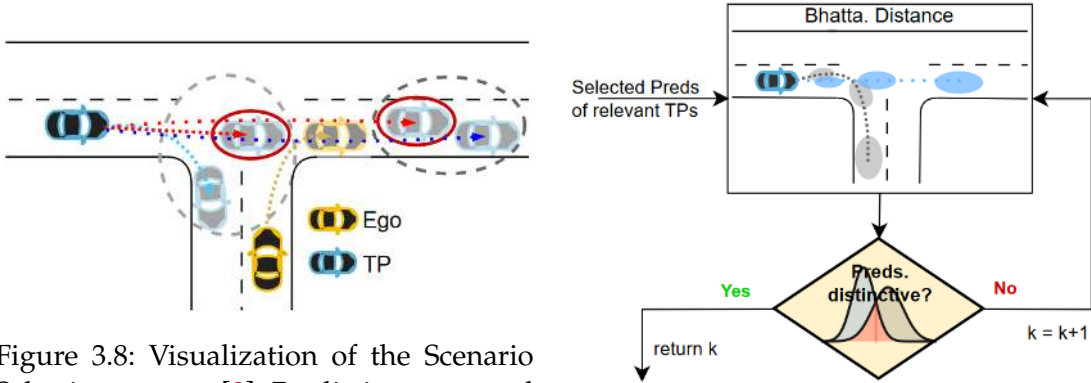


Figure 3.8: Visualization of the Scenario Selection process [8]. Predictions grouped within gray ellipses represent clusters of similar scenarios. Red ellipses highlight the representative scenarios chosen from each cluster. The ego vehicle (yellow) interacts with potential trajectories of the traffic participant (blue).

Figure 3.9: Illustration of the online branching time estimation process using the Bhattacharyya Distance [8]. The algorithm evaluates the predictions of relevant TPs to quantify the similarity of the predicted distributions for a TP. It then returns a branching timestep k .

To achieve this clustering, the MTR-BMPCC framework employs the Uniform Visibility Deformation (UVD) method, a technique commonly used to compare trajectories and identify topologically distinct clusters, as described in [27, 28]. The UVD procedure determines whether two predicted trajectories belong to the same cluster by analyzing their geometry and interaction with the ego vehicle's trajectory.

The steps of the procedure are as follows:

1. Discretize the predicted trajectories $\tau_1(\alpha)$ and $\tau_2(\alpha)$ into equidistant points across the parameterization range $\alpha \in [0, 1]$.
2. For each corresponding point on the two trajectories (i.e., points at the same timestep), connect the points with a line segment.
3. Check whether any of these line segments intersect the ego vehicle's trajectory from the previous timestep, τ_{k-1}^* .
4. If a line segment intersects the ego vehicle's trajectory, the two predicted trajectories are assigned to different clusters.

For scenarios involving multiple TPs, the procedure checks all trajectory pairs for each TP. If any trajectory pair is not topologically equivalent (i.e., intersects with the ego trajectory), all predictions of that TP are assigned to different clusters. This ensures a clear separation of scenarios based on their potential impact on the ego vehicle. A visualization of the clustering process is illustrated in Fig. 3.2.

Mode Ranking

The process of scenario ranking and selection follows the clustering stage (see Fig. 3.2). In this step, the modes of each cluster are ranked to identify the most relevant ones for constructing the scenario tree. The goal is to ensure that the final selection of scenarios, $\mathcal{S}$, provides sufficient information for BMPCC to generate an optimal trajectory under multimodal uncertainty.

The most relevant prediction of a cluster is selected by evaluating the collision risk, as proposed in [29]. This metric calculates the probability of a collision event between two oriented rectangles analytically.

The Collision Event Probability (CEP) density $\dot{C}_m^o(t_k)$ represents the probability density of a collision between a TP $o_{k,m}$ and the ego vehicle z_k at a given timestep t_k . The cumulative probability of a collision over a time interval $[t_i, t_j]$ is obtained by integrating the CEP density. Using a first-order Taylor approximation the CEP density is estimated based on the TP's pose, velocity, and Gaussian uncertainty at t_k , as well as the ego vehicle's pose

and velocity derived from the ego plan τ_{k-1}^* .

To compute the cumulative CEP over the prediction horizon with a sample time Δt , the following summation is performed:

$$C_m^o(t_0, t_{N-1}) = \sum_{k=0}^N \dot{C}_m^o(t_k) \Delta t. \quad (3.28)$$

This collision risk is calculated for each TP prediction and is used to define a metric for evaluating trajectory safety. A prediction's risk is combined with its probability using the following decision function:

$$D_m^o = C_m^o(t_0, t_N) + \lambda \pi_m^o \quad (3.29)$$

Here, λ is a tunable parameter that balances conservativeness and performance. A higher focus on the collision risk C_m^o may result in overly cautious behavior, potentially impeding performance, whereas focusing solely on prediction's probability π_m^o could lead to riskier decisions.

For each cluster C_s , the prediction mode with the highest decision function value is selected as the representative scenario s to be included in the scenario tree:

$$s = \arg \max_{m \in C_s} \sum_{o=0}^{O-1} D_m^o \quad (3.30)$$

The total number of scenarios selected for the scenario tree is defined by $|\mathcal{S}|$. After selecting the most relevant scenarios, the weight ω_s for each scenario s in Eq. 3.20 can be computed. This weight is determined by summing the probabilities of all individual predictions within each cluster as follows:

$$\omega_s = \sum_{m \in C_s} \pi_m^o. \quad (3.31)$$

3.2.4 Decision Postponing

The branching timestep b , mentioned in Eq. 3.24, is calculated by the MTR-BMPCC framework adaptively at each timestep k using a rule-based approach. The branching time provides an estimate of when the uncertainty in the traffic scene is expected to resolve. It represents the point in time when all relevant traffic participants (TPs) are predicted

to have modes that are more or less similar. Accurate estimation of the branching time is crucial, as discrepancies between the estimated branching time and the real outcome can have significant implications on the behavior of the ego vehicle. A high branching time may lead to overly cautious behavior, while a low branching time (as seen in conventional Model Predictive Contouring Control (MPCC), which plans only for the most probable outcome) may result in reactive and non-anticipatory driving.

The Bhattacharyya Distance (BD) is employed to evaluate the similarity between two probability distributions $p(x)$ and $q(x)$. It is defined as:

$$B = -\ln \left(\int \sqrt{p(x)q(x)} dx \right), \quad (3.32)$$

where the term $\int \sqrt{p(x)q(x)} dx$ is referred to as the Bhattacharyya coefficient, which measures the overlap between $p(x)$ and $q(x)$. A smaller Bhattacharyya Distance indicates greater similarity between the two distributions.

For Gaussian distributions, BD is used to quantify the overlap between two multivariate Gaussian distributions $\mathcal{N}_{k,i}^o$ and $\mathcal{N}_{k,j}^o$ at a specific timestep k within the prediction horizon, where $i, j \in S$ represent the components of the Gaussian Mixture Model (GMM) defined in Eq. 3.11. Using this notation, the Bhattacharyya Distance for two multivariate Gaussian distributions provided by the MTR is given by:

$$B(o, k, i, j) = \frac{1}{8} (\mu_{k,i}^o - \mu_{k,j}^o)^T (\Sigma_k^o)^{-1} (\mu_{k,i}^o - \mu_{k,j}^o) + \frac{1}{2} \ln \left(\frac{\det \Sigma_k^o}{\sqrt{\det \Sigma_{k,i}^o \det \Sigma_{k,j}^o}} \right), \quad (3.33)$$

where the covariance matrix Σ_k^o is the average of the two covariance matrices:

$$\Sigma_k^o = \frac{\Sigma_{k,i}^o + \Sigma_{k,j}^o}{2}. \quad (3.34)$$

The algorithm iterates through the timesteps k until the prediction horizon and identifies the branching timestep b as the first timestep where the computed distance $B(o, k, i, j)$ exceeds the predefined threshold B_{th} as seen by

$$B(o, k, i, j) \geq B_{th}. \quad (3.35)$$

For scenarios with multiple TPs, collision risk is first employed to filter out irrelevant TPs for decision-making. When more than two TPs remain, the branching time t_b^o is

calculated for each TP. The final branching time t_b is selected as the maximum of the individual branching times:

$$t_b = \max_o t_b^o. \quad (3.36)$$

If $|S| > 2$, the distinguishability between all combinations of Gaussian components is evaluated first, and then the maximum branching time is chosen.

3.3 Reinforcement Learning

This section explains the fundamentals of Reinforcement Learning (RL). It is mainly based on [30, 31, 32].

Reinforcement Learning is one of the three main areas of machine learning, alongside supervised and unsupervised learning. Unlike supervised learning, where labeled data guides the learning process, RL is a computational approach where an agent learns to make decisions through trial-and-error interactions with an environment. The goal is to discover a policy that maximizes cumulative rewards over time. RL can also be considered an approach where no pre-existing data is gathered. Instead, the policy improves over time by interacting with the environment and learning from these interactions. Moreover, RL carefully balances *exploration* (trying new actions to discover their effects) and *exploitation* (choosing actions that maximize known rewards) to find an optimal or good policy in a reasonable amount of time.

The interaction between an agent and its environment is illustrated in Fig. 3.10. At each timestep t , the agent observes the current state S_t , selects an action A_t according to a policy π , and receives a reward R_{t+1} from the environment. The environment transitions to a new state S_{t+1} , and this cycle repeats. These interactions are typically accumulated in an episode and can be expressed as a tuple:

$$(S_0, A_0, R_1, S_1, A_1, \dots, S_{T-1}, A_{T-1}, R_T). \quad (3.37)$$

Through this process, the agent aims to maximize the total cumulative reward over its interactions with the environment.

The key components of an RL system are:

- **Policy:** A policy is a mapping from states to actions, determining the agent's behavior. Policies can be deterministic ($a = \pi(s)$, where a specific action is chosen

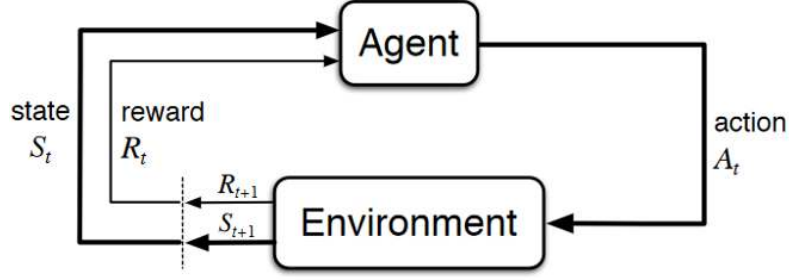


Figure 3.10: Interaction between an agent and its environment. The agent observes the current state S_t , selects an action A_t , and receives a reward R_{t+1} from the environment. The environment transitions to the next state S_{t+1} , completing the interaction loop. Adapted from [30].

for each state) or stochastic ($\pi(a|s) = \mathbb{P}[A_t = a|S_t = s]$, where probabilities are assigned to actions given the state). Policies can also be computationally expensive, such as when implemented through search processes, where extensive evaluations of possible actions and outcomes are conducted.

- **Reward Signal:** At each timestep, the environment provides a scalar reward R_t to the agent, which evaluates the immediate quality of its actions. The agent's objective is to maximize the cumulative reward over time, expressed as the *return*, which is defined as:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}. \quad (3.38)$$

Here, $\gamma \in [0, 1]$ is the *discount factor*, which prioritizes immediate rewards over distant ones. If $\gamma = 1$, all rewards are treated equally, and the agent considers the long-term impact of its actions. If $\gamma = 0$, the agent only considers immediate rewards, leading to greedy behavior.

- **Value Function:** The value function estimates the long-term desirability of states or state-action pairs by predicting the expected return. The state value function is defined as:

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s], \quad (3.39)$$

which evaluates the expected return from a state s under policy π . Similarly, the action value function is given by:

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a], \quad (3.40)$$

which extends this concept to specific actions.

- **Model of the Environment:** A model predicts how the environment transitions between states and provides expected rewards. Models are useful in *model-based*

RL, where agents can simulate interactions to plan actions. In contrast, *model-free RL* relies entirely on direct interactions without explicitly modeling the environment.

3.3.1 Markov Decision Processes

Markov Decision Processes (MDPs) are fundamental to RL as they provide a mathematical framework to model decision-making problems where outcomes are partly random and partly under the control of an agent. MDPs capture the interaction between an agent and its environment, helping to formalize the process of learning optimal behavior through rewards.

An MDP is defined as a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where:

- $\mathcal{S}$: A finite set of states.
- $\mathcal{A}$: A finite set of actions.
- $\mathcal{P}$: State transition probability matrix:

$$\mathcal{P}_{ss'}^a = \mathbb{P}[S_{t+1} = s' | S_t = s, A_t = a]. \quad (3.41)$$

- $\mathcal{R}$: A reward function:

$$\mathcal{R}_s^a = \mathbb{E}[R_{t+1} | S_t = s, A_t = a]. \quad (3.42)$$

- $\gamma \in [0, 1]$: The discount factor determining the importance of future rewards.

MDPs rely on the Markov property, which states that the future depends only on the current state and action, not on the full history:

$$\mathbb{P}[S_{t+1} | S_t] = \mathbb{P}[S_{t+1} | S_1, \dots, S_t]. \quad (3.43)$$

The dynamics of an MDP and the agent's interaction with it are visualized in Fig. 3.11, using the example of a student managing time between scrolling through Facebook, studying, and other activities. For instance, consider the state with a value function $v_*(s) = 10$, where the student has studied for two times in a row. At this point, the student could:

- Choose to study again, earning a reward of +10, which maintains the high state value.
- Choose to go to the pub receiving an immediate reward of +1, but then transitioning to other states with probabilities 0.4, 0.4, and 0.2, which lead to lower state

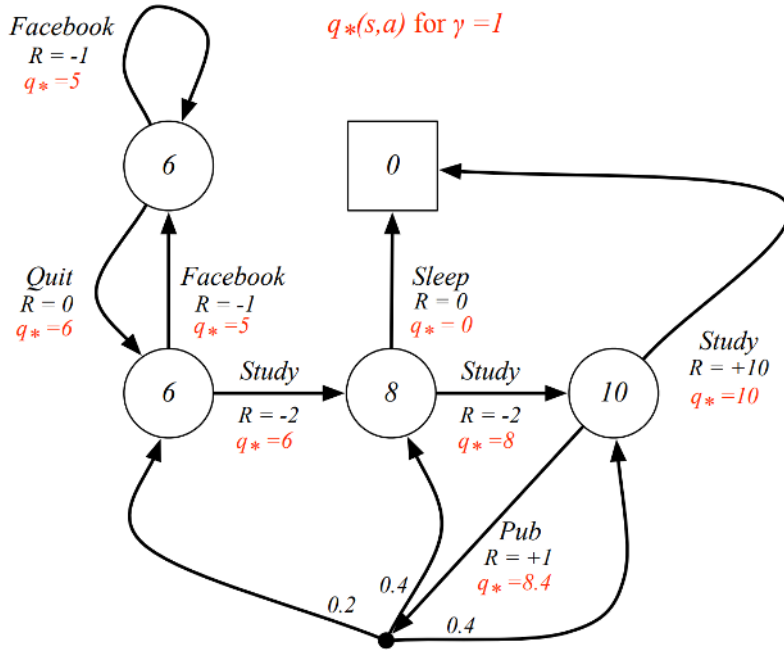


Figure 3.11: An example MDP showing a student's decision-making process [31]. Each state has associated rewards R , optimal action values $q^*(s, a)$, and optimal state values $v^*(s)$ computed with the discount factor $\gamma = 1$. The figure illustrates that spending time on Facebook is suboptimal, while studying leads to higher cumulative rewards.

values.

This example demonstrates that the optimal strategy is to keep studying, as indicated by both the optimal state value function $v_*(s)$ and the optimal action value function $q_*(s, a)$. Solving the MDP analytically is possible but often computationally intractable. RL algorithms are employed to approximate the optimal state or action value functions, which, once acquired, can be used online to guide the agent in selecting the best actions at each state.

In many real-world scenarios, the agent may not have full knowledge of the state S_t . This leads to Partially Observable Markov Decision Processes (POMDPs), where the agent maintains a belief distribution over possible states based on available observations. POMDPs extend MDPs and are more computationally challenging due to this uncertainty, but they better model realistic environments with incomplete information.

3.3.2 Bellman Equations

The Bellman equations provide a recursive decomposition of value functions and serve as a foundational approach to solve Markov Decision Processes (MDPs) analytically. They describe how the value of a state (or state-action pair) depends on the rewards and the

values of subsequent states. These equations have a traditional role in RL, as they enable the computation of optimal policies, provided that a perfect MDP model with known transition dynamics and reward functions is given.

The Bellman equation for the state value function $v_\pi(s)$ can be derived as follows. The process begins by splitting the value function into two components: the immediate reward and the discounted value of future states:

$$v_\pi(s) = \mathbb{E}_\pi [G_t \mid S_t = s], \quad (3.44)$$

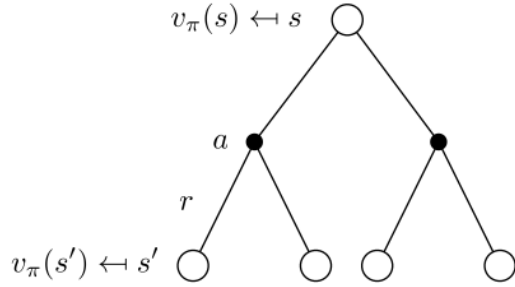
$$= \mathbb{E}_\pi [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s], \quad (3.45)$$

$$= \mathbb{E}_\pi [R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]. \quad (3.46)$$

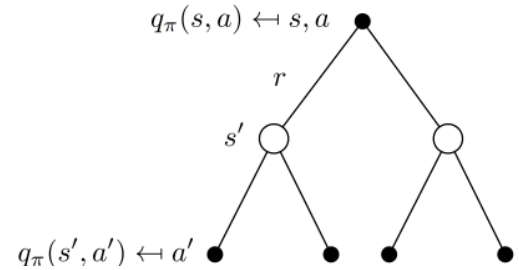
This recursive formulation establishes a relationship between the value of a state, the immediate reward received, and the discounted value of subsequent states. Similarly, the Bellman expectation equation for the action value function $q_\pi(s, a)$ expresses the value of a state-action pair as:

$$q_\pi(s, a) = \mathbb{E}_\pi [R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]. \quad (3.47)$$

Bellman Expectation Equations



(a) Recursive structure for state value function $v_\pi(s)$. It shows how the value of a state depends on immediate rewards and the values of subsequent states.



(b) Recursive structure for action value function $q_\pi(s, a)$. It highlights how the value of an action depends on the rewards and the action value of the next state.

Figure 3.12: Bellman expectation equations for state and action value functions. These equations connect values recursively and form the basis for policy evaluation in RL. [30]

The Bellman expectation equations provide a recursive way to express the relationship between the value of a state or state-action pair and the expected return under a given policy π . They demonstrate how values depend on the immediate reward and the discounted values of subsequent states.

The relationship between the state and action value functions under a policy π is:

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a | s) q_\pi(s, a), \quad (3.48)$$

$$q_\pi(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s'). \quad (3.49)$$

Combining these two equations gives the Bellman expectation equation for the state value function:

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a | s) \left[\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s') \right]. \quad (3.50)$$

For the action value function, the Bellman expectation equation becomes:

$$q_\pi(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a' | s') q_\pi(s', a'). \quad (3.51)$$

Figures 3.12(a) and 3.12(b) illustrate these equations. The first shows the recursive structure for state value functions, while the second shows how action value functions recursively depend on both rewards and state transitions.

Bellman Optimality Equations

The Bellman optimality equations extend the expectation equations to define optimal value functions, where an optimal policy π_* maximizes the expected return. These equations are:

$$v_*(s) = \max_a \left[\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s') \right], \quad (3.52)$$

$$q_*(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \max_{a'} q_*(s', a'). \quad (3.53)$$

The optimal policy π_* is defined as:

$$\pi_*(a | s) = \begin{cases} 1 & \text{if } a = \arg \max_{a \in \mathcal{A}} q_*(s, a), \\ 0 & \text{otherwise.} \end{cases} \quad (3.54)$$

Figures 3.13(a) and 3.13(b) depict the recursive relationships for the optimal state and action value functions.

The Bellman optimality equations differ from the expectation equations as they explicitly consider the optimal action selection. The equations recursively compute the maximum value or action value at each state. The MDP is considered to be solved if the optimal

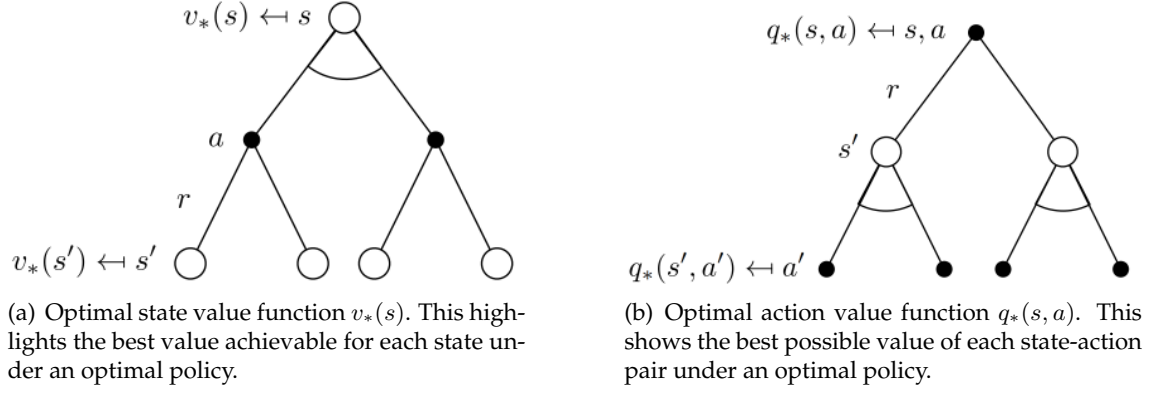


Figure 3.13: Bellman optimality equations for state and action value functions. These equations define the maximum achievable value functions under the optimal policy. [30]

value functions have been found. The optimal value functions represent the best possible performance that may be achieved within the MDP.

3.3.3 Policy Evaluation

Policy Evaluation (PE) is a classical RL algorithm rooted in Dynamic Programming (DP), where solving MDPs is done by assuming a perfect model of the environment including known transition dynamics $\mathcal{P}$ and rewards $\mathcal{R}$. DP algorithms serve as a theoretical foundation for RL and provide a “gold standard” for approximating value functions and optimal policies. However, their real-world use is limited due to their computational expense and reliance on full knowledge of the MDP. Practical RL algorithms often aim to approximate the solutions derived using DP.

The objective of PE is to compute the state value function $v_\pi(s)$ for a given policy π iteratively using the Bellman expectation equation:

$$v_{k+1}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left[\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_k(s') \right]. \quad (3.55)$$

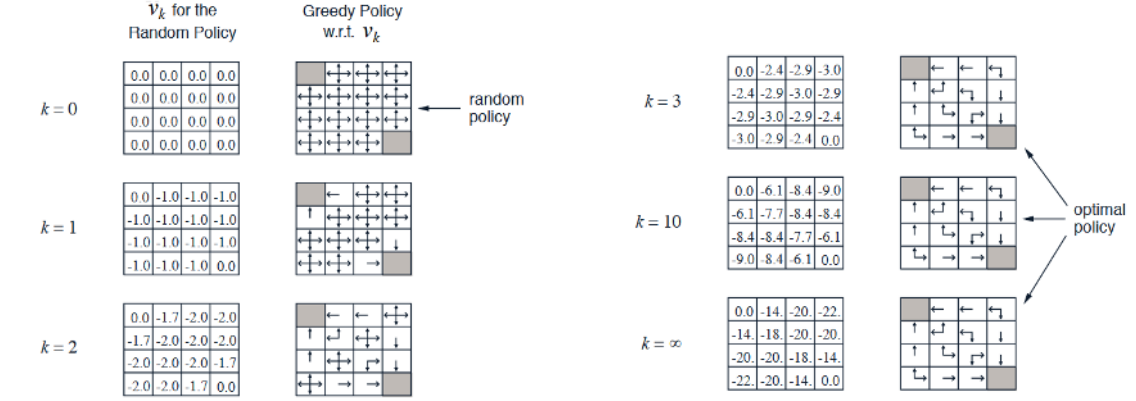
Starting from an initial guess $v_0(s) = 0$ at timestep $k = 0$, this process continues until convergence, producing the optimal value function $v_*(s)$. The discount factor $\gamma < 1$ ensures convergence by reducing the impact of future rewards.

The example in Fig. 3.14 illustrates policy evaluation in a gridworld environment:

- The agent receives a reward of $R = -1$ for every non-terminal state transition, encouraging shorter paths.

- The terminal states, visualized in dark gray, are terminal states where the episode ends.

The left column in each subfigure shows the estimated state values $v_k(s)$ after k iterations, starting from a random, uniform policy. The right column shows the corresponding greedy policy with respect to $v_k(s)$. Initially ($k = 0$), the value function yields zero. As the Bellman equation (Eq. 3.55) is applied iteratively, the initially random policy converges to the optimal policy π_* .



(a) Policy evaluation at $k = 0, 1, 2$. The left column shows the estimated state values $v_k(s)$, while the right column shows the greedy policy w.r.t. $v_k(s)$. At $k = 0$, the policy is random and uniform.

(b) Policy evaluation at $k = 3, 10, \infty$. By $k = 3$, the value function begins converging to the optimal value function $v_*(s)$. At $k = \infty$, the policy evaluation fully converges.

Figure 3.14: Policy evaluation in a gridworld example. The Bellman equation is iteratively applied to compute the state values $v_k(s)$, starting from $v_0(s) = 0$. The greedy policy with respect to $v_k(s)$ improves over time, ultimately calculating the optimal value function $v_*(s)$. [30]

3.3.4 Monte Carlo Prediction Algorithm

Monte Carlo (MC) methods differ from Dynamic Programming (DP) as they do not require a perfect model of the environment. Instead, MC methods rely solely on experience, represented as sequences of states, actions, and rewards $(S_0, A_0, R_1, S_1, A_1, R_2, \dots)$. This makes MC methods particularly useful in scenarios where the environment's dynamics are unknown or difficult to model. Unlike DP, which iteratively updates value functions using a complete model, MC methods estimate value functions by sampling episodes generated by a given policy π .

One variant of MC methods is the First-Visit Monte Carlo Prediction Algorithm, which estimates the state value function $v_\pi(s)$ by averaging the returns from the first visit to each state s across multiple episodes. For a state s , the return G_t is computed starting from the first time t that state s is visited in the episode. This is calculated using Eq. 3.38.

The algorithm iteratively updates the value function by taking the average of all observed returns for each state. The pseudocode is provided in Algorithm 1.

Algorithm 1 First-visit MC Prediction, for estimating $V \approx v_\pi$ [30]

Input: A policy π to be evaluated

Output: State-value function $V(s)$

```

1: Initialize  $V(s) \in \mathbb{R}$  arbitrarily, for all  $s \in \mathcal{S}$ 
2: Initialize  $Returns(s) \leftarrow \emptyset$  (an empty list), for all  $s \in \mathcal{S}$ 
3: while true do
4:   Generate an episode following  $\pi$ :  $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$ 
5:    $G \leftarrow 0$ 
6:   for  $t = T - 1, T - 2, \dots, 0$  do
7:      $G \leftarrow \gamma G + R_{t+1}$ 
8:     if  $S_t$  does not appear in  $S_0, S_1, \dots, S_{t-1}$  then
9:       Append  $G$  to  $Returns(S_t)$ 
10:       $V(S_t) \leftarrow \text{average}(Returns(S_t))$ 
11:    end if
12:  end for
13: end while

```

Monte Carlo methods offer several advantages. They are unbiased because they directly use actual returns from episodes, and the *Law of Large Numbers* ensures that the value estimates converge to the true expected values as the number of episodes goes to infinity. Additionally, MC methods are not sensitive to the initial value estimates of $V(s)$, which makes them flexible in practical applications.

However, MC methods have notable limitations. They are less efficient compared to methods like Temporal-Difference (TD) learning because they require complete episodes to compute returns. This reliance on full episodes introduces high variance, as updates depend on the entire sequence of rewards, which can vary significantly. Consequently, MC methods may converge slowly, especially in environments with long episodes or high variability in returns.

3.3.5 Temporal-Difference Prediction Algorithm

Temporal-Difference (TD) learning combines key ideas from both Dynamic Programming (DP) and Monte Carlo (MC) methods. Like MC, TD learning does not require prior knowledge of the MDP dynamics, making it applicable in model-free settings. At the same time, it updates value estimates incrementally using estimates of future returns, similar to DP methods. This combination of features makes TD learning an efficient and flexible approach in RL.

The simplest TD method, TD(0), updates the state value function $v(s)$ after every transition within an episode. The update is based on the TD error:

$$\delta_t = R_{t+1} + \gamma v(S_{t+1}) - v(S_t), \quad (3.56)$$

which represents the difference between the observed reward and the current estimate of the value. Using this error, the value function is updated incrementally as:

$$v(S_t) \leftarrow v(S_t) + \alpha \delta_t, \quad (3.57)$$

where $\alpha \in (0, 1]$ is a step-size parameter that controls the magnitude of the update. The pseudocode is presented in Algorithm 2.

Algorithm 2 Tabular TD(0) for estimating v_π

Input: A policy π to be evaluated, step size parameter $\alpha \in (0, 1]$

Output: State-value function $V(s)$

```

1: Initialize  $V(s)$  arbitrarily, for all  $s \in \mathcal{S}^+$ , except  $V(\text{terminal}) = 0$ 
2: while true do
3:   Initialize  $S$ 
4:   while  $S$  is not terminal do
5:      $A \leftarrow$  action given by  $\pi$  for  $S$ 
6:     Take action  $A$ , observe  $R, S'$ 
7:      $V(S) \leftarrow V(S) + \alpha [R + \gamma V(S') - V(S)]$ 
8:      $S \leftarrow S'$ 
9:   end while
10: end while

```

TD learning has several notable advantages. First, it is more data-efficient than MC methods, as it updates the value function incrementally at every step of an episode instead of waiting until the end. This makes TD faster in practice, particularly in environments with long episodes. Additionally, TD methods exhibit low variance because each update depends on an immediate reward and the next state value, rather than the variability of complete episodes.

However, TD learning is biased, as updates rely on current estimates of the value function. This bias decreases over time as more updates refine the estimates. Furthermore, it is sensitive to the choice of the initial value function $v(s)$ and the step-size parameter α . Poor choices of α can lead to oscillations or divergence, particularly if the step size does not decay appropriately.

3.3.6 Monte Carlo Tree Search

This subsection is based on the comprehensive survey by Browne et al. [11], which explains the foundations and evolution of Monte Carlo Tree Search (MCTS), citing a wide

range of papers in the field.

MCTS combines the precision of tree search algorithms with the generality of random sampling to solve complex decision-making problems. It originated from Monte Carlo methods in statistical physics, which were developed to approximate solutions for intractable integrals. Research on game theory and its application in artificial intelligence played a significant role in the development of MCTS, particularly in balancing exploration and exploitation through techniques like the Upper Confidence Bound (UCB). UCB calculates an upper bound for the regret of not playing a certain move, helping to find a balance between exploring new actions that might yield better long-term rewards and exploiting actions already known to provide good results.

MCTS is built on two fundamental ideas. The first is that true optimal actions can be approximated by running random simulations, also known as rollouts. These simulations, which start from a root state and proceed to a terminal state (forming an episode), provide critical information about expected rewards. The second idea is that the outcomes of these simulations are used to incrementally build a search tree, where nodes represent states and edges correspond to actions. Each node in this tree is associated with a value that represents the expected reward. The tree is constructed iteratively, and simulations continue until the computational budget is exhausted. Once the tree is built, it can be searched to extract an optimal or near-optimal action sequence for an episode.

A key advantage of MCTS is that it can operate in a model-free setting, as the reward is only processed after a simulation is completed. This terminal reward, often referred to as the terminal cost, is particularly useful for evaluating long-term outcomes. With every new simulation, additional information is incorporated into the tree, continuously improving its representation of the decision space and guiding policy development.

MCTS has been applied in various games, including computer Go, Scrabble, and Hex. Achievements in research as well as game AI are outlined in Table 3.3.

Table 3.3: Milestones in MCTS research. Adapted from [11].

Year	Milestone in MCTS Research
1990	Abramson shows how Monte Carlo simulations compute state values.
1993	Brügmann applies Monte Carlo methods to computer Go.
1998	MAVEN defeats the world Scrabble champion.
2006	Kocsis and Szepesvári introduce UCT.
2006	Gelly et al. achieve success with UCT in computer Go using MoGo.
2009	Fuego defeats a top human professional in 9x9 Go.
2009	MoHex becomes the world champion Hex player.

MCTS operates iteratively, following the process of **Selection**, **Expansion**, **Simulation**, and **Backpropagation**, as illustrated in Figure 3.15. Each iteration extends and refines the tree, improving the policy. This process ends when a predefined number of simulations

is reached.

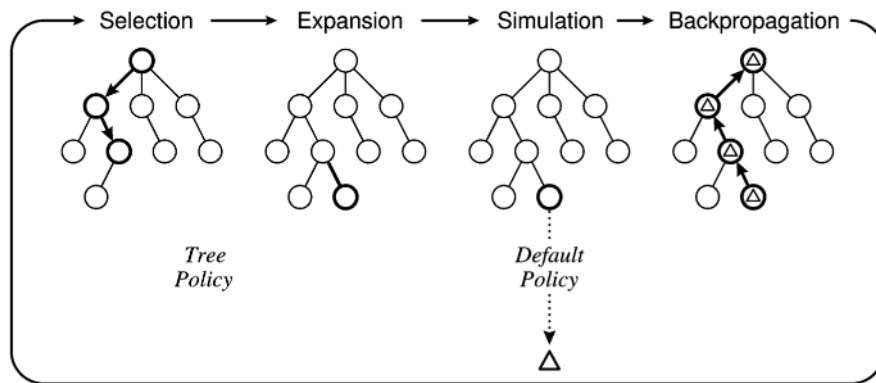


Figure 3.15: The MCTS process consists of four stages: Selection, Expansion, Simulation, and Backpropagation. Selection identifies the most promising node to expand; Expansion adds a new child node; Simulation estimates the reward using a default policy; and Backpropagation updates the tree with the simulation results. Adapted from [11].

The MCTS algorithm starts from a root node representing the initial state of the environment and proceeds iteratively through four main stages in each iteration:

1. **Selection:** The tree policy is responsible for selecting which nodes to explore next. The process starts at the root node, with the tree policy deciding which child node to traverse based on a balance of *exploitation* (choosing high-reward nodes) and *exploration* (trying new, less-visited nodes). A basic tree policy selects the best child node at each step, continuing until a leaf node is found. The selection method defines how the tree is built, making it a core component of the algorithm. Alternative selection strategies include:
 - **Max Child:** Selects the child node with the maximum reward, resulting in a greedy and potentially suboptimal tree-building process.
 - **Robust Child:** Selects the most-visited child node, which balances exploration over multiple simulations.
 - **Max-Robust Child:** Selects the child node with the highest reward and visit count. If no child meets these criteria, an additional simulation budget is allocated to explore further and identify such a node, often resulting in improved solutions.
 - **Secure Child:** Chooses the child node that maximizes a lower confidence bound, which may be represented by a heuristic or learned function.
2. **Expansion:** When a leaf node is reached, the tree policy expands the tree by adding a new child node. This typically involves selecting an untried action from the ac-

tion space, either randomly or through a more sophisticated method. The primary goal of this step is to explore new actions that may lead to better rewards in future simulations.

3. **Simulation:** After expansion, the default policy takes over to simulate a trajectory (rollout) from the newly added node to a terminal state. The simplest default policy uses random actions, but more advanced policies can serve as stronger examples for guiding the search. The simulation generates a terminal reward based on the interaction with the environment, which is then used to evaluate the potential of the newly added node and its predecessors.
4. **Backpropagation:** The simulation reward is propagated back through the visited nodes, starting from the newly added node up to the root. At each node, the visit count is incremented, and the reward is averaged with previous rewards. This ensures that the value estimates at each node reflect the cumulative knowledge gained from all simulations that have passed that node.

The general MCTS approach is formalized in Algorithm 3. Its iterative process produces an asymmetric search tree that focuses computational resources on promising regions of the state space. Figure 3.16 shows an example of such a tree.

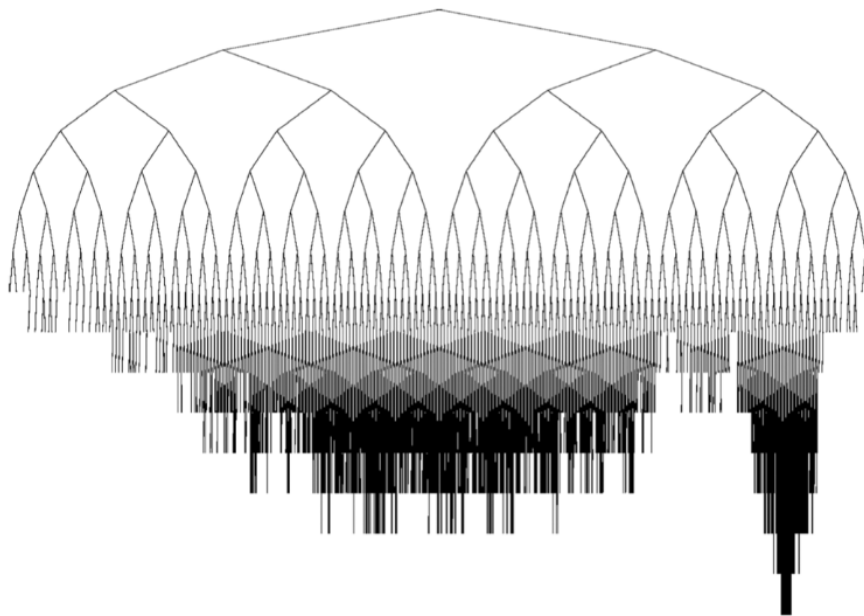


Figure 3.16: An asymmetric search tree built by MCTS. The tree grows more in regions of interest, providing insights into the problem domain. Adapted from [11].

Algorithm 3 General MCTS Approach [11]**Input:** Initial state s_0 **Output:** Best action a

```

1: function MCTSSEARCH( $s_0$ )
2:   Create root node  $v_0$  with state  $s_0$ 
3:   while within computational budget do
4:      $v_l \leftarrow \text{TREEPOLICY}(v_0)$ 
5:      $\Delta \leftarrow \text{DEFAULTPOLICY}(S(v_l))$ 
6:     BACKUP( $v_l, \Delta$ )
7:   end while
8:   return A(BESTCHILD( $v_0$ ))
9: end function

```

Upper Confidence Bounds for Trees

To balance exploration and exploitation in an even better way, Kocsis and Szepesvári introduced the Upper Confidence Bounds for Trees (UCT) algorithm [33]. The UCT algorithm combines Upper Confidence Bound (UCB)-based exploration with tree search to minimize regret and converge to the game-theoretic optimum over time. It was designed as a Monte Carlo search algorithm with a low probability of error if stopped prematurely and guarantees convergence to the optimal policy given sufficient simulations.

The UCT formula is given by:

$$\text{UCT} = \bar{X}_j + 2C_p \sqrt{\frac{2 \ln(n)}{n_j}} \quad (3.58)$$

where:

- $\bar{X}_j$: Average reward of child j .
- C_p : Exploration constant.
- n : Number of times the parent node has been visited.
- n_j : Number of times child j has been visited.

The second term in the formula ensures that unvisited children ($n_j = 0$) are assigned the largest possible value (infinity), guaranteeing that every child is visited at least once before any child is expanded further. This mechanism allows UCT to effectively balance exploration and exploitation during search. If two or more children have identical UCT values, one is selected randomly.

As the term under the square root diminishes for frequently visited nodes, the algorithm

prioritizes exploring less-visited nodes, facilitating a thorough search of the decision space. The exploration constant, C_p , plays a crucial role: if set too high, exploration dominates, while if set too low, the algorithm becomes overly greedy, focusing only on the highest-reward nodes. Proper tuning of C_p is essential for achieving optimal performance. Kocsis and Szepesvári suggest setting C_p to $\frac{1}{\sqrt{2}}$ when $\bar{X}_j \in [0, 1]$. However, if rewards fall outside this range, a different constant may be more appropriate.

The pseudocode for the UCT method is provided in Algorithm 4.

MCTS Enhancements

Although vanilla MCTS is a powerful and versatile algorithm, its performance can be significantly enhanced by incorporating domain-specific knowledge and various modifications, making this a major focus of current MCTS research. For instance, all MCTS-based programs for the game of Go have incorporated game-specific information, which reduces the number of required simulations and decreases variance in results. However, these improvements come with tradeoffs, as domain knowledge may introduce biases that limit the exploration of novel strategies.

Several MCTS enhancements and variations are outlined below:

- **Real-Time MCTS:** Uses approximations to increase the efficiency of the forward model, allowing faster simulation of rewards.
- **Tree Policy Enhancements:** Incorporates heuristic functions or domain knowledge into the selection procedure to focus on specific nodes that are likely to yield better results.
- **Policy and Tree Integration:** Combines tree construction with policy improvement. For example, the policy can guide tree building during simulation, and the tree policy can refine the policy during backpropagation. This hybrid approach allows MCTS algorithm to improve its search tree and policy simultaneously.
- **Neural Network Integration:** AlphaGo [12], a program developed by DeepMind, demonstrated the potential of combining neural networks with MCTS. In this approach, a policy network predicts the probabilities of possible moves given the current board state, while a value network evaluates board positions by estimating the expected outcome. These predictions are integrated into the MCTS process to enhance both the tree policy and the default policy, significantly improving player performance.
- **Other Variations:**

Algorithm 4 The UCT Algorithm [11]

Input: Initial state s_0 , exploration constant C_p **Output:** Best action a

```

1: function UCTSEARCH( $s_0$ )
2:   Create root node  $v_0$  with state  $s_0$ 
3:   while within computational budget do
4:      $v_l \leftarrow \text{TREEPOLICY}(v_0)$ 
5:      $\Delta \leftarrow \text{DEFAULTPOLICY}(S(v_l))$ 
6:      $\text{BACKUP}(v_l, \Delta)$ 
7:   end while
8:   return  $A(\text{BESTCHILD}(v_0))$ 
9: end function
10: function TREEPOLICY( $v$ )
11:   while  $v$  is non-terminal do
12:     if  $v$  not fully expanded then
13:       return  $\text{EXPAND}(v)$ 
14:     else
15:        $v \leftarrow \text{BESTCHILD}(v, C_p)$ 
16:     end if
17:   end while
18:   return  $v$ 
19: end function
20: function EXPAND( $v$ )
21:   Choose an untried action  $a$  from  $A(S(v))$ 
22:   Add a new child  $v'$  to  $v$  with  $s(v') = f(S(v), a)$  and  $a(v') = a$ 
23:   return  $v'$ 
24: end function
25: function BESTCHILD( $v, C_p$ )
26:   return  $\arg \max_{v' \in \text{children of } v} \frac{Q(v')}{N(v')} + C_p \sqrt{\frac{2 \ln N(v)}{N(v')}}$ 
27: end function
28: function DEFAULTPOLICY( $s$ )
29:   while  $s$  is non-terminal do
30:     Choose  $a \in A(s)$  uniformly at random
31:      $s \leftarrow f(s, a)$ 
32:   end while
33:   return reward for state  $s$ 
34: end function
35: function BACKUP( $v, \Delta$ )
36:   while  $v$  is not null do
37:      $N(v) \leftarrow N(v) + 1$ 
38:      $Q(v) \leftarrow Q(v) + \Delta$ 
39:      $\Delta \leftarrow -\Delta$ 
40:      $v \leftarrow \text{parent of } v$ 
41:   end while
42: end function

```

- Stochastic tree policies that incorporate randomness to diversify the selection process.
- Parallelized MCTS implementations to distribute simulations on multiple processors for faster convergence.

3.4 Imitation Learning

This section is based on [34], [35], [36], and [37].

Imitation learning studies algorithms that attempt to mimic an expert or teacher by learning a policy $\pi_\theta(s)$, as mentioned in Sec. 3.3. This approach is particularly valuable in robotics, where creating explicit rules for complex motions or sequential decisions is often infeasible, time-consuming, or expensive. The key idea is that imitation learning incorporates feedback, as it not only relies on an expert dataset but also queries the expert to provide additional information until the policy converges to the near-optimal policy $\pi^*(s)$.

Imitation learning differs from supervised learning, which does not consider the effects of its outputs on the environment. Supervised learning uses a static dataset, and when testing the learned policy, this can lead to cascading errors and deviations from the expert’s trajectory due to *distribution shift*. Distribution shift occurs because the learned mapping of states to actions does not fully match the expert’s mapping, as the dataset cannot cover all possible expert behaviors. This mismatch can result in catastrophic failures in novel states, where the learned policy’s behavior is undefined or erroneous.

The core distinction of imitation learning is its ability to incorporate expert feedback in new states, especially where interactions with the environment are sparsely represented in the dataset. This process allows the agent to find desirable actions and integrate them into its behavior. To implement imitation learning, a demonstrator to provide expert trajectories (from either simulated or real-world data), a policy class to represent the mapping from states to actions, a loss function to guide training, and a learning algorithm to optimize the policy are needed.

In the following, two algorithms from the area of imitation learning are introduced: Behavior Cloning (BC) and Dataset Aggregation (DAgger).

3.4.1 Behavior Cloning

Behavior cloning, first introduced in [6], formulates imitation learning as a supervised learning problem. The objective is to learn a policy π_θ that maps states to actions by mimicking the behavior of an expert. Given expert demonstration data

$$D = \left\{ (s_{1:T_i}^i, a_{1:T_i}^i) \right\}_{i=1}^N, \quad (3.59)$$

where $s_{1:T_i}^i$ represents the i th state trajectory, $a_{1:T_i}^i$ represents the corresponding control trajectory, and N is the number of demonstrations, the goal is to find a policy π_θ that minimizes the supervised loss:

$$\min_{\theta} \sum_{i,t} \mathcal{L}(a_t^i, \pi_\theta(s_t^i)), \quad (3.60)$$

where $\mathcal{L}$ is the loss function, t denotes the timestep, and $\pi_\theta : s \mapsto a$ is a deterministic policy such as a neural network.

The aim is to minimize the deviation from the expert's actions along the demonstrated trajectories, ensuring that the policy closely imitates the expert's behavior. Furthermore, the provided expert actions are assumed to perfectly represent the expert's behavior.

By leveraging the supervised learning paradigm, behavior cloning provides a simple and efficient method for imitation learning. Algorithm 5 summarizes the behavior cloning process. The policy π_θ is initialized with random parameters and iteratively updated using an optimizer to minimize the loss $\mathcal{L}$, computed over the expert's dataset D .

Algorithm 5 Behavior Cloning

Input: Expert data $D = \left\{ (s_{1:T_i}^i, a_{1:T_i}^i) \right\}_{i=1}^N$, Policy π_θ , Loss function $\mathcal{L}(a, \pi_\theta(s))$

Output: Trained policy π_θ

```

1: function BEHAVIORCLONING( $D, \pi_\theta$ )
2:   Initialize policy parameters  $\theta$  randomly
3:   while not converged do
4:     Sample a batch of state-action pairs  $\{(s_t^i, a_t^i), \dots\} \sim D$ 
5:     Compute predicted actions  $\hat{a}_t^i = \pi_\theta(s_t^i)$  for each  $s_t^i$ 
6:     Compute the loss:  $\mathcal{L} = \sum_{i,t} \mathcal{L}(a_t^i, \hat{a}_t^i)$ 
7:     Update  $\theta$  using an optimizer, e.g. gradient descent:  $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}$ 
8:   end while
9:   return  $\pi_\theta$ 
10: end function

```

Despite its effectiveness, behavior cloning suffers from a critical issue known as distribution shift. During training, the policy learns from the expert's demonstrated trajectories, but during deployment, it must generate its own states and actions. When the policy deviates from the expert's path, it encounters new states that were not present in the

training data, leading to compounding errors. These cascading errors can significantly degrade performance, potentially resulting in catastrophic failures in real-world applications.

Furthermore, in behavior cloning, low training error does not necessarily translate to good test performance. This is because training and test data in imitation learning are not drawn from the same distribution, violating the supervised learning assumption of independent and identically distributed data ($\neq$ i.i.d.). In supervised learning, the training and test data typically follow the same distribution:

$$x, y \sim p(x, y). \quad (3.61)$$

However, in imitation learning, where a sequential decision process has to be learned, the training distribution differs fundamentally from the test distribution. During testing, the policy's actions influence the states it encounters, resulting in a test distribution $p_\pi(s)$ that depends on the learned policy π .

To illustrate this, a lane-following algorithm can be considered as an example. If the policy makes an error and the vehicle steers toward the edge of the road, it enters a state that was not well-represented in the training data. Without guidance from the expert, the policy may fail to recover and drive off the road (see Fig. 3.18). This fundamental problem arises because the learned mapping from states to actions does not align with the test distribution $p_\pi(s)$, as the test distribution is influenced by the policy itself.

Behavior cloning minimizes the objective expressed in Eq. 3.60 which can also be written as:

$$\min_{\theta} \mathbb{E}_{\pi^*} [\mathcal{L}(a, \pi_\theta(s))]. \quad (3.62)$$

Still, to perfectly imitate the expert policy, the actual test distribution would require minimizing:

$$\min_{\theta} \mathbb{E}_{\pi_\theta} [\mathcal{L}(\pi^*(s), \pi_\theta(s))]. \quad (3.63)$$

This means that using behavior cloning, the policy attempts to imitate the expert, but its performance relies heavily on how well the dataset reflects the full range of expert behaviors. This limitation creates room for errors, particularly in states that are under-represented or absent in the training data.

A notable application of behavior cloning is Pomerleau's ALVINN project [6], which used a neural network to map camera images to steering angles for autonomous lane-following. The dataset was generated by simulating camera images (states) along with their corresponding steering angles (actions) using a road generator. To address distri-

bution shift, training images were simulated with realistic noise levels and off-center scenarios, where steering labels were adjusted to guide the vehicle back to the center of the road. While this approach enabled the autonomous vehicle to drive 400 meters at slow speeds on a sunny fall day, it was limited to simple lane-following tasks.

To mitigate distribution shift in behavior cloning, it is essential to:

- Ensure the dataset represents the potential states encountered during deployment of the policy sufficiently good.
- Introduce perturbations and augmentations to simulate off-policy behaviors.
- Collect data interactively by allowing the policy to explore new states.

3.4.2 Dataset Aggregation

Dataset Aggregation (DAgger) [37] improves upon behavioral cloning by addressing the problem of distribution mismatch. The key idea is to iteratively query the expert for corrections during rollouts of the learned policy, allowing the policy to visit new states by itself and asking the expert what actions it would take in those situations. This process aggregates new data into the training set, enabling the policy to approximate the actual distribution of the expert $p_\pi(s)$, as presented in Eq. 3.63. By incrementally refining the dataset and the policy, DAgger ensures reliable convergence to a near-optimal policy.

DAgger offers good scalability and guarantees convergence, making it a robust solution for overcoming the limitations of behavior cloning. However, it requires frequent expert queries during training, which may be inefficient depending on the application. Figure 3.17 provides an example of DAgger in action, illustrating its successful application in the 3D racing game *Super Tux Kart*.

Algorithm 6 outlines the dataset aggregation process. The policy is iteratively refined by collecting new data points through rollouts, querying the expert for corrective actions, and aggregating the results into an expanding training dataset.

Algorithm 6 DAgger [37]

Input: Expert policy π^* , Initial policy $\hat{\pi}_1$, Number of iterations N , Number of steps per trajectory T

Output: Best policy $\hat{\pi}_i$ (validated on validation set)

- 1: Initialize dataset $\mathcal{D} \leftarrow \emptyset$
- 2: Initialize $\hat{\pi}_1$ to any policy in Π
- 3: **for** $i = 1$ to N **do**
- 4: Let $\pi_i = \beta_i \pi^* + (1 - \beta_i) \hat{\pi}_i$
- 5: Sample T -step trajectories using π_i
- 6: Get dataset $\mathcal{D}_i = \{(s, \pi^*(s))\}$ of visited states by π_i and actions from expert
- 7: Aggregate datasets: $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}_i$
- 8: Train classifier $\hat{\pi}_{i+1}$ on $\mathcal{D}$
- 9: **end for**
- 10: **return** Best $\hat{\pi}_i$ on validation



Figure 3.17: An example from [37], illustrating DAgger in a challenging environment, such as a track with sharp turns and the risk of falling off the road (Rainbow Road example).

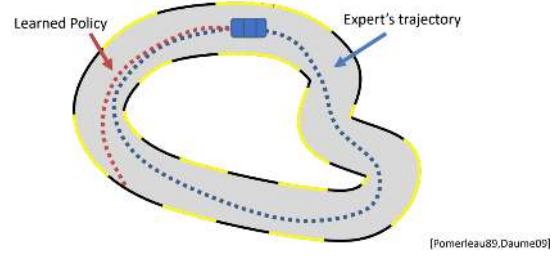


Figure 3.18: The image shows the expert's trajectory (blue dashed) and the learned policy's trajectory (red dashed). In behavior cloning, compounding errors can cause the learned policy to deviate significantly from the expert's optimal path [38].

3.5 Supervised Learning

This section is based on [39].

Supervised learning is a machine learning paradigm where a model learns to map inputs to outputs by minimizing a loss function on a labeled dataset. The dataset consists of input-output pairs (x, y) , where x represents the input features, and y represents the corresponding ground-truth labels. The goal of supervised learning is to train a model $f_\theta(x)$, parameterized by θ , such that the predicted outputs match the ground truth as closely as possible.

Neural networks are a common choice for supervised learning tasks due to their ability to approximate complex, non-linear functions. These networks are composed of layers

of interconnected neurons and employ activation functions, such as ReLU (see Fig. 3.20), to introduce non-linearity. The training process involves iteratively updating the model parameters using optimization algorithms to minimize the chosen loss function.

3.5.1 Neural Networks

A neural network consists of multiple layers of neurons. Each neuron computes a weighted sum of its inputs, applies a non-linear activation function, and passes the result to the next layer. One of the most commonly used activation functions is the Rectified Linear Unit (ReLU), defined as:

$$f(x) = \max(0, x). \quad (3.64)$$

The architecture of a neural network typically includes three types of layers:

- **Input layer:** This layer has the same size as the input vector and serves as the entry point of the data into the network.
- **Hidden layers:** These layers process the data by performing transformations through weighted sums and activation functions. The number and size of hidden layers determine the network's capacity to model complex relationships in the data.
- **Output layer:** The size of this layer can vary depending on the task, e.g., one output for regression or multiple outputs for classification tasks.

The output of the network is computed using a forward pass through the layers. For a multi-layer perceptron (MLP), the network can be expressed mathematically as:

$$y = f_{\theta}(x) = W^{(L)} \cdot f^{(L-1)}(W^{(L-1)} \cdot \dots \cdot f^{(1)}(W^{(1)} \cdot x + b^{(1)}) + b^{(L-1)}) + b^{(L)}, \quad (3.65)$$

where $W^{(l)}$ and $b^{(l)}$ are the weights and biases of layer l , and $f^{(l)}$ represents the activation function.

3.5.2 Cross-Entropy Loss

For classification tasks, cross-entropy loss is widely used to measure the difference between the true labels and the predicted probabilities. The cross-entropy loss function is defined as:

$$\mathcal{L}_{\text{CE}}(y, \hat{y}) = - \sum_{i=1}^N y_i \log(\hat{y}_i), \quad (3.66)$$

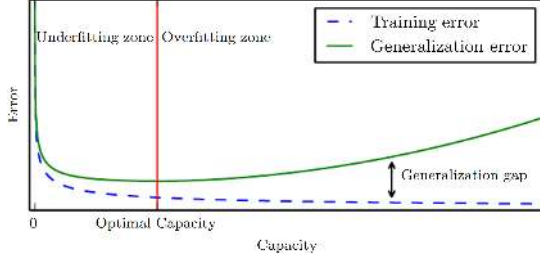


Figure 3.19: Illustration of the underfitting and overfitting zones. Initially, both training and generalization errors are high. As the model capacity increases, the generalization error decreases until reaching the optimal capacity. Beyond this point, the training error continues to decrease, but the generalization error increases due to overfitting. Adapted from [39].

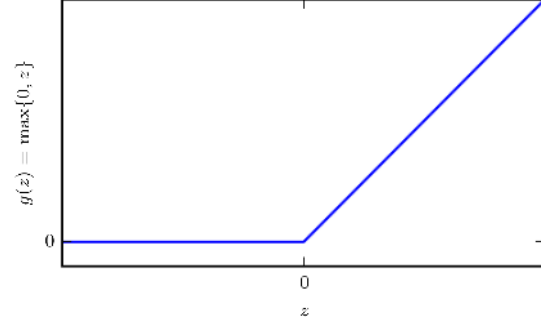


Figure 3.20: The ReLU activation function [39]. It is a non-linear function that is linear for positive inputs and zero for negative inputs. This simplicity makes it computationally efficient and well-suited for gradient-based optimization. ReLU is the default activation function used in feed-forward models like the multi-layer perceptron (MLP).

where y_i represents the true label, $\hat{y}_i$ is the predicted probability for class i , and N is the number of classes.

In practice, the true labels y are often represented as a one-hot encoded vector. A one-hot encoded vector is a binary vector of length N (the number of classes) where the entry corresponding to the true class is 1, and all other entries are 0. For example, if there are three classes and the true label corresponds to the second class, the one-hot encoded vector for y would be $[0, 1, 0]$. The cross-entropy loss effectively penalizes the predicted probability $\hat{y}_i$ for the true class i if it deviates from 1, encouraging the model to output probabilities closer to the true labels.

3.5.3 Training

The training process involves minimizing a loss function by iteratively updating the parameters θ using an optimization algorithm. To train a model f_θ , labeled data (x, y) is used. The training process consists of the following steps:

1. **Forward pass:** Compute the model's predictions $f_\theta(x)$.
2. **Loss computation:** Evaluate the loss function $\mathcal{L}(\theta)$ based on the difference between the predictions and ground truth y .
3. **Backpropagation:** Compute the gradients of the loss function with respect to the model parameters $\nabla_\theta \mathcal{L}(\theta)$ using the chain rule.

4. **Parameter updates:** Update the parameters using an optimizer.

One popular optimization algorithm is *Adam* [40], which combines the benefits of momentum and adaptive learning rates. It is especially effective when training models with noisy gradients or on problems with sparse data. Adam requires the gradient of the loss function with respect to the parameters $\nabla_{\theta}\mathcal{L}(\theta)$ at each step. The parameter update rule for Adam is given by:

$$\theta_{t+1} = \theta_t - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}, \quad (3.67)$$

where $\hat{m}_t$ and $\hat{v}_t$ are the bias-corrected estimates of the first and second moments of the gradients, and α is the learning rate.

A higher learning rate can make convergence faster but may introduce more noise, causing the optimization to overshoot or oscillate around the minimum. Conversely, a lower learning rate provides smoother updates, reducing noise but potentially slowing down convergence and risking getting stuck in local minima. The default hyperparameters recommended in the original paper are:

- $\alpha = 0.001$ (learning rate),
- $\beta_1 = 0.9$ (exponential decay rate for the first moment estimates),
- $\beta_2 = 0.999$ (exponential decay rate for the second moment estimates),
- $\epsilon = 10^{-8}$ (small constant to avoid division by zero).

The Adam algorithm is presented in Algorithm 7.

Algorithm 7 Adam [40]

Input: Learning rate α , exponential decay rates β_1, β_2 , constant ϵ , initial parameters θ_0

Output: Updated parameters θ_t

```

1: Initialize  $m_0 = 0$  (1st moment vector),  $v_0 = 0$  (2nd moment vector),  $t = 0$ 
2: while  $\theta_t$  not converged do
3:    $t \leftarrow t + 1$ 
4:    $g_t = \nabla_{\theta}\mathcal{L}(\theta_{t-1})$  (Compute gradient)
5:    $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1)g_t$  (Update biased first moment estimate)
6:    $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2)g_t^2$  (Update biased second moment estimate)
7:    $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$  (Compute bias-corrected first moment estimate)
8:    $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$  (Compute bias-corrected second moment estimate)
9:    $\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$  (Update parameters)
10: end while
11: return  $\theta_t$ 

```

3.5.4 Class Weighting

In datasets with imbalanced class distributions, class weighting is used to mitigate the bias towards overrepresented classes by penalizing errors on underrepresented classes more heavily. This ensures that the model pays more attention to minority classes, improving its performance on imbalanced datasets.

The weighted cross-entropy loss is expressed as:

$$\mathcal{L}_{\text{weighted}}(y, \hat{y}) = - \sum_{i=1}^N w_i \cdot y_i \cdot \log(\hat{y}_i), \quad (3.68)$$

where the class weight w_i is typically defined as:

$$w_i = \frac{1}{n_i}, \quad (3.69)$$

with n_i being the number of samples in class i in the training dataset.

3.5.5 L2 Regularization

Overfitting occurs when a model performs well on the training set but fails to generalize to unseen data. This is often caused by a lack of sufficient training data or a lack of diversity in the data. Overfitting is characterized by low training error but high generalization error, as shown in Figure 3.19. While the training accuracy is high, the model's performance on the test set deteriorates.

To mitigate the effects of overfitting, L2 regularization is often added to the loss function. The regularization term penalizes large weights in the model, which helps to reduce the model's ability to memorize the training data. The total loss function with L2 regularization is expressed as:

$$\mathcal{L}_{\text{L2R}} = \mathcal{L}_{\text{CW}} + \frac{\lambda}{2} \sum_j w_j^2, \quad (3.70)$$

where:

- $\mathcal{L}_{\text{CW}}$ is the class-weighted cross-entropy loss,
- λ is the regularization coefficient, and
- j indexes over all the weights w_j in the model.

The parameter λ controls the strength of the regularization. For instance:

- If λ is large (e.g., 0.01), regularization is strong. This can help lower the test loss

by improving generalization but may increase the training loss as the model is restricted from fitting the training data too closely.

- If λ is small (e.g., 0.0001), the effect of regularization is minimal, allowing the model to fit the training data more closely, but with a higher risk of overfitting.

L2 regularization encourages the model to use smaller weights, reducing the risk of overfitting and improving its ability to generalize to unseen data.

3.5.6 Data Augmentation

Data augmentation is a technique used to artificially expand the training dataset by applying transformations to the original data. This approach increases the diversity of the training data, improves the model's robustness, and helps reduce overfitting.

One common augmentation method is adding white noise to the inputs. This can be mathematically defined as:

$$z_i \sim \mathcal{N}(0, \sigma^2), \quad x_{\text{noisy},i} = x_i + z_i, \quad \text{for } i = 1, 2, \dots, n, \quad (3.71)$$

where:

- x_i represents the original input sample,
- z_i is random noise sampled from a normal distribution $\mathcal{N}(0, \sigma^2)$ with mean 0 and variance σ^2 ,
- $x_{\text{noisy},i}$ is the resulting noisy input sample, and
- n is the total number of input samples.

In addition to adding noise, other common data augmentation techniques include rotation and mirroring. Rotation involves modifying data by rotating it by different angles, which introduces variations in the training dataset and exposes the model to a broader range of scenarios. Mirroring, on the other hand, involves flipping data horizontally or vertically.

While data augmentation is an effective method for reducing the effect of model overfitting, it needs to be applied with care to achieve optimal results. When applied appropriately, it decreases the generalization error while maintaining a reasonable training error. Excessive augmentation can cause the generalization error to increase again, as the model may learn incorrect patterns.

4 Approach

4.1 Decision Postponing using Monte Carlo Tree Search

This section explains the approach used to learn near-optimal decision postponing policies from a Monte Carlo Tree Search (MCTS) expert for less conservative, efficient, safe and comfortable driving. It is divided into two parts. Section 4.1 describes the MCTS expert to generate expert decision postponing sequences, while Section 4.2 details the process of learning these sequences to make them real-time capable within the BMPCC framework for online usage. The overall approach is illustrated in Fig. 4.1.

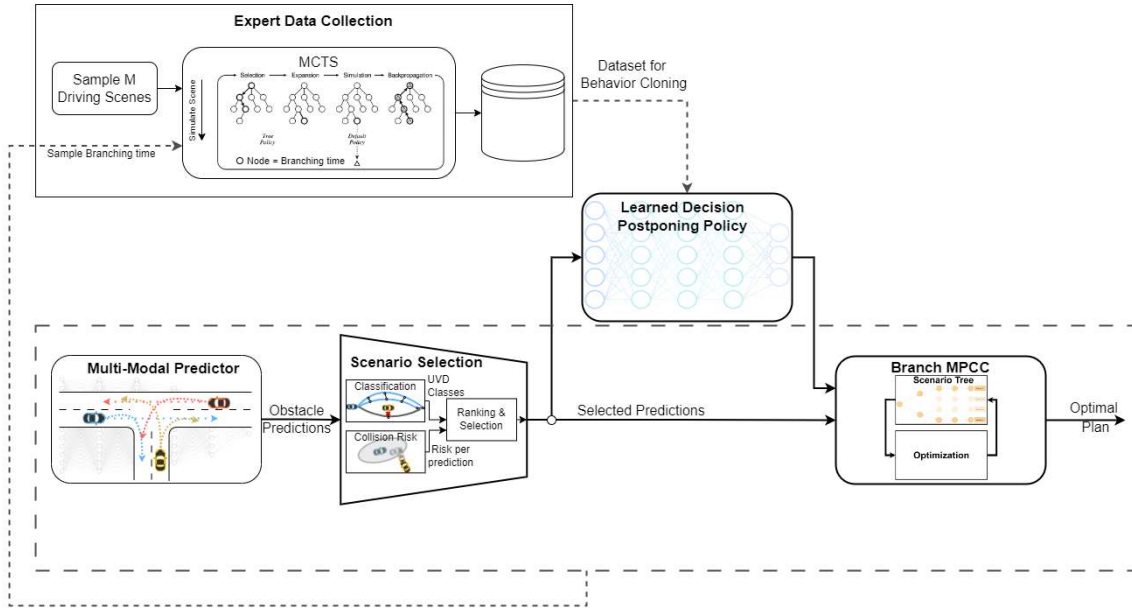


Figure 4.1: Overview of the approach. Monte Carlo Tree Search is used with a rollout sampling mechanism to generate expert decision postponing sequences for different traffic scenarios. Observations collected at each timestep, including the state of the AV and TPs, are stored for training a policy. The learned policies is later used online within the MTR-BMPCC framework.

4.1.1 Rollout Generator

The goal of the rollout generator is to create realistic driving scenarios with high multi-modal uncertainty for the BMPCC-based Monte Carlo Tree Search expert. These scenar-

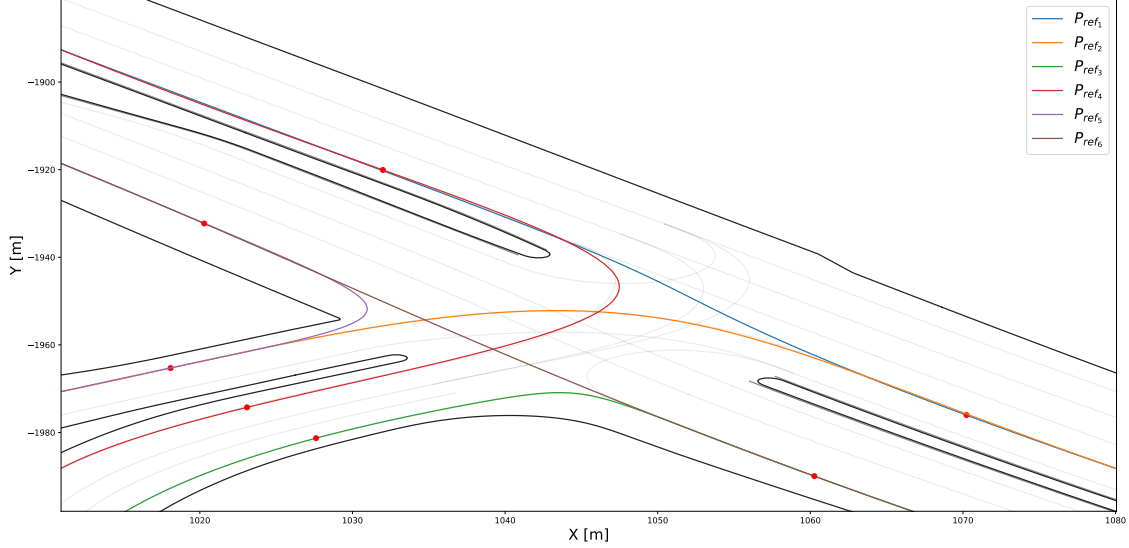


Figure 4.2: Reference lanes used for the rollout generator. Six lanes form the basis for traffic scenarios on this intersection, with dots representing start and end points to ensure similar lane lengths for all reference combinations.

ios simulate situations with multimodal driving behavior, for example, where a traffic participant (TP) may drive straight in one rollout and turn right in another, while the autonomous vehicle (AV) follows the same reference.

The rollout generator is designed to be scalable for more than one TP. It uses one map and one intersection from the Waymo Open Motion Dataset (WOMD) [20]. This intersection features multiple lanes and possibilities to create uncertain traffic situations with multiple TPs. The reference lanes used by the rollout generator are visualized in Fig. 4.2, numbered clockwise for clarity. Dots represent the start and end points of each lane to ensure similar distances. For example, if two TPs follow reference lane 1 and the ego vehicle follows reference lane 4, the timing between them could result in the ego vehicle performing both a street-crossing and a merging maneuver. This makes the intersection a versatile basis for the rollout generator.

To scale the rollout generator, reference combinations are used. A reference combination consists of a pair of two reference lanes: one for the AV and one for the TP. For multiple TPs, multiple reference combinations are sampled, with the constraint that the AV maintains the same reference lane. This setup allows the rollout generator to sample diverse scenarios easily. Table 4.1 lists the reference combinations, sorted by AV reference lanes. In total, 16 out of 24 possible combinations are used based on the six selected reference lanes.

The reference path object contains the x and y positions, orientation ψ , splines, and control points for the respective Bézier curve. These are pre-stored in a map dictionary and can be conveniently loaded using the respective reference combination. Fig. 4.3 illustrates

Table 4.1: Reference lane combinations used by the rollout generator and sorted by AV reference lane for better readability.

ID	AV	TP	ID	AV	TP	ID	AV	TP	ID	AV	TP
10	1	3	12	2	5	14	4	1	4	5	1
11	1	4	13	2	6	8	4	2	0	5	2
6	2	3	7	3	5	2	4	5	15	6	3
9	2	4	1	3	6	5	4	6	3	6	4

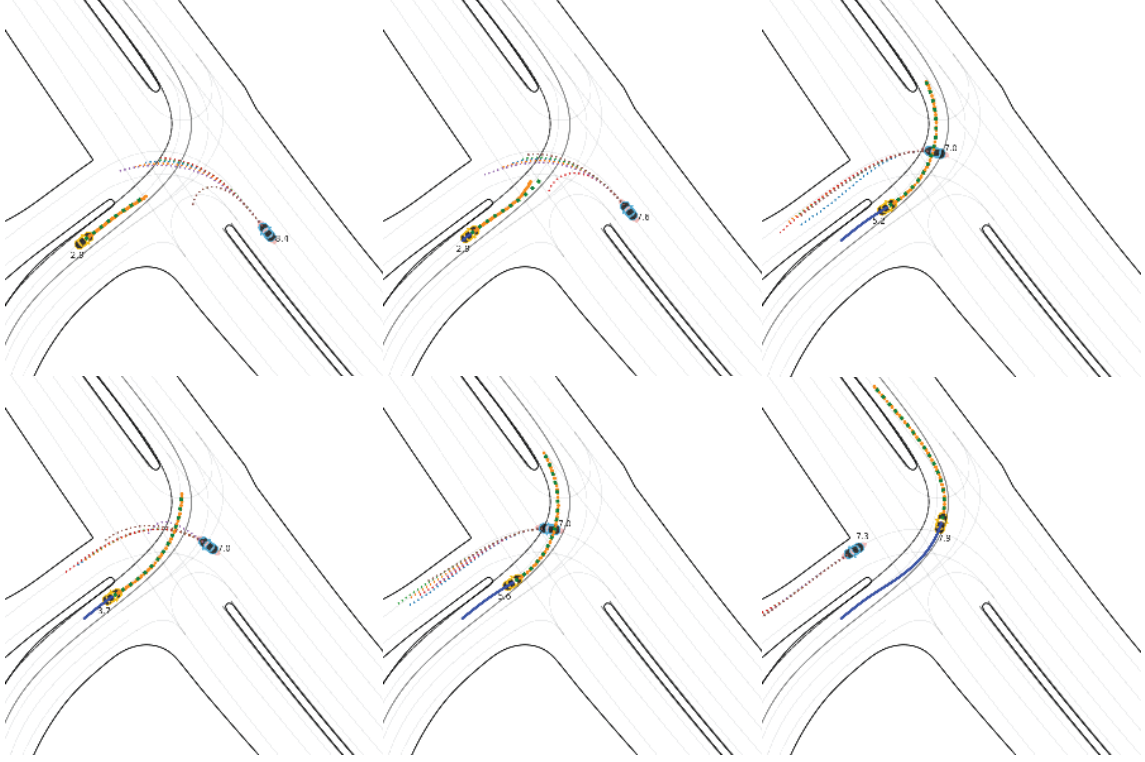


Figure 4.3: An example of reference combination 8: the ego vehicle (yellow) follows reference lane 4, while a TP (blue) follows reference lane 2.

an example of reference combination 8, where the ego vehicle follows lane 4, and a TP follows lane 2.

The pseudocode for generating a single rollout is provided in Algorithm 8. The functions involved in the rollout generation process are described below:

- **SAMPLEROLLOUT()**: This function initializes the rollout by sampling the required parameters, such as IDM parameters, initial positions, and initial speeds, for the AV and TPs. The sampled ranges are provided in Table 4.2.
- **EXTRACTPATHS()**: This function loads the path object for the AV and TPs based on the reference combinations. The path object contains the x and y positions, orientation ψ , and pre-defined Bézier curve parameters for the reference path.

- **GENERATEOBSTACLEARRAY()**: This function creates the obstacle array for the TPs. Each obstacle object stores the ground truth data and tracks the current pose and relevant parameters of the TP. TP trajectories, including speeds and positions, are simulated using the sampled IDM parameters. The virtual path from the IDM simulation is then mapped to the Bézier curve of the reference path, as described in Section 3.1.2.
- **INITIALIZEBMPCC()**: Finally, the BMPCC object is initialized with the reference path, initial position, and speed for the AV.

Algorithm 8 Rollout Generator

Input: Map Dictionary M , Number of Traffic Participants N_{TP}

Output: Rollout ID r , Rollout Dictionary R , Obstacle Array O , BMPCC Object B

```

1: function GENERATEROLLOUT( $M, N_{TP}$ )
2:    $r, R \leftarrow \text{SAMPLEROLLOUT}(N_{TP})$ 
3:    $P \leftarrow \text{EXTRACTPATHS}(M, R)$  ▷ Eq. 3.6
4:    $O \leftarrow \text{GENERATEOBSTACLEARRAY}(R, P)$  ▷ Sec. 3.1.1
5:    $B \leftarrow \text{INITIALIZEBMPCC}(R, P)$ 
6:   return  $r, R, O, B$ 
7: end function
8: function SAMPLEROLLOUT( $N_{TP}$ )
9:    $r \leftarrow \text{GENERATEROLLOUTID}()$ 
10:   $R \leftarrow \{\}$ 
11:   $m \leftarrow \text{SAMPLEMAP}()$ 
12:   $C \leftarrow \text{SAMPLEREFERENCECOMBINATION}(N_{TP})$ 
13:   $v_0, T, a, b, s_0, \delta, l \leftarrow \text{SAMPLEIDMPARAMETERS}(N_{TP})$ 
14:   $V_{TP}, v_{AV} \leftarrow \text{SAMPLEINITIALSPEED}(N_{TP})$ 
15:   $\Theta_{TP}, \theta_{AV} \leftarrow \text{SAMPLEINITIALPOSITION}(N_{TP})$ 
16:   $R \leftarrow R \cup \{m, C, v_0, T, a, b, s_0, \delta, l, V_{TP}, \Theta_{TP}, v_{AV}, \theta_{AV}\}$ 
17:  return  $r, R$ 
18: end function

```

4.1.2 Monte Carlo Tree Search Expert

In this section, the Monte Carlo Tree Search (MCTS) expert for near-optimal decision postponing (DP) is introduced. It builds upon the vanilla MCTS algorithm described in Section 3.3.6, with several key enhancements designed to optimize the decision postponing for the MTR-BMPCC Framework explained in Sec. 3.2.

Multiple MCTS variants are evaluated in a qualitative analysis to identify the best-performing approach. The selected variant, `MAX_VALUE_GAUSSIAN_RES5_WITH_EXPANSION_LIMIT`, serves as the successor of this analysis and is chosen based on its performance. This analysis benchmarks all developed MCTS variants under the same simulation budget of 165 steps and is detailed

Table 4.2: Rollout parameter ranges, including IDM parameters, initial speeds, positions for TPs and AV.

Parameter [Unit]	Range	Description
n_{map} [-]	[1, 1]	Number of maps
v_0 [m/s]	[5.0, 10.0]	Desired speed range for IDM
T [s]	[3.0, 5.0]	Desired time headway range for IDM
s_0 [m]	[1.0, 5.0]	Minimum distance to lead vehicle for IDM
a [m/s ²]	[0.5, 2.0]	Maximum acceleration range for IDM
b [m/s ²]	[4.0, 8.0]	Comfortable deceleration range for IDM
δ [-]	4.0	Acceleration exponent for IDM
l [m]	5.0	Wheelbase of a TP vehicle
d [m]	[30.0, 50.0]	Initial distance to lead vehicle range
θ_{TP} [%]	[0, 20]	Initial position for TP
θ_{AV} [%]	[0, 20]	Initial position for AV
v_{TP} [m/s]	[3.0, 10.0]	Initial speed range for TP
v_{AV} [m/s]	[0.0, 10.0]	Initial speed range for AV

in Appendix A. The limit of 165 simulations is necessary due to time constraints during the later data collection phases.

The selected MCTS variant, used as an expert, offers several key advantages:

- The best overall cost performance,
- Fast MPC solving times, and
- The smoothest decision postponing sequences and ego vehicle states.

An exemplary tree constructed by this MCTS variant is shown in Fig. 4.4. Initially, the tree explores broadly, but later uses its simulation budget more strategically, constructing a robust, long tree by exploring new actions only every 0.5 seconds.

Before constructing the tree, the MCTS expert queries the rollout generator to obtain a rollout dictionary containing all relevant scenario information. This process is described in the pseudocode in Algorithm 8.

The MCTS expert follows the pseudocode outlined in Algorithm 9, starting from a root node v_0 and iteratively building a tree using the tree policy and default policy. At each timestep, the algorithm selects a branching time from the action space $A = \{1, 2, 3, \dots, 30\}$, constructing a DP sequence that maximizes the reward for the rollout. Once the simulation is complete, the tree is searched to identify the best DP sequence for that specific rollout.

The MCTS expert is structured around the following main components:

1. **Selection:** The tree policy handles the selection process by identifying the child node with the highest maximum value. As nodes are selected, their corresponding DP values are stored to form a DP tree sequence.
2. **Expansion:** The root node expands its children uniformly at random, with an expansion limit of 15. Nodes following the root are expanded using a Gaussian method, and have an expansion limit of 5. The DP of a child node is sampled from a normal distribution centered around the parent's DP reduced by one, with a standard deviation of 14. The number of expansions is limited starting from tree length of 23. Then, a node with an expansion limit of 5 is added every 5 timesteps, corresponding to a time interval of 0.5 s as defined by the sampling time in Table 3.2. In between, the tree is filled with nodes following a linearly decreasing DP sequence without computing a reward, using the simulation budget sparingly to generate a longer tree. The mechanism to limit node expansion is also referred to as *grid building*, representing a discretization of the decision space.
3. **Simulation:** Each new node is simulated using the accumulated DP sequence up to the expanded node, called the tree DP sequence, to compute a reward Δ from the start using the MTR-BMPCC and the sampled rollout. The total reward is the negative sum of the running costs at each timestep of the simulation. The running cost of an AV using the BMPCC is described in Eq. 3.17. The simulated tree DP sequence and reward are stored in a priority queue for later comparisons, providing backup candidates if needed.
4. **Backpropagation:** During backpropagation, the maximum reward at each node is updated if the new reward exceeds the current maximum. Average rewards and visit counts are also tracked.
5. **Search:** The search process identifies the near-optimal DP sequence for a specific rollout by traversing the tree from the root node v_0 using max-child selection. The resulting sequence is simulated, and its reward is compared with candidates in the priority queue. The best sequence is returned as the expert DP sequence.

The full pseudocode for the MCTS expert and its tree policy is provided in Algorithm 9 and Algorithm 10.

The design choices and motivations for the proposed method are outlined below, along with insights into the features, reasons, and effects of the chosen variants.

The MCTS expert can construct decision postponing sequences up to 65 timesteps, if the full simulation budget is exhausted. This length ensures good coverage of rollouts in the intersection scenarios described in Sec. 4.1.1, which typically exhibit high uncertainty initially and average 90 timesteps in total. Experiments have shown that a high sequence

Algorithm 9 MCTS Expert

Input: Simulation budget N , rollout ID r , rollout Dictionary R , obstacle Array O , BMPCC object B

Output: Expert DP Sequence dp_{expert}

```

1: function DPMCTS( $N, r, R, O, B$ )
2:   Create root node  $v_0$ 
3:   while within simulation budget  $N$  do
4:      $v_l, dp_{\text{tree}} \leftarrow \text{TREEPOLICY}(v_0)$ 
5:      $\Delta \leftarrow \text{DEFAULTPOLICY}(dp_{\text{tree}}, r, R, O, B)$ 
6:      $\text{BACKUP}(v_l, \Delta)$ 
7:   end while
8:   return  $\text{SEARCH}(v_0)$ 
9: end function
10: function DEFAULTPOLICY( $dp_{\text{tree}}, r, R, O, B$ )
11:    $\Delta \leftarrow \text{SIMULATE}(dp_{\text{tree}}, r, R, O, B)$ 
12:    $\text{ADDBESTCANDIDATE}(\Delta, dp_{\text{tree}})$ 
13:   return  $\Delta$ 
14: end function
15: function BACKUP( $v, \Delta$ )
16:   while  $v \neq \text{null}$  do
17:      $N(v) \leftarrow N(v) + 1$ 
18:      $Q(v) \leftarrow Q(v) + \Delta$ 
19:     if  $Q_{\text{MAX}}(v) < \Delta$  then
20:        $Q_{\text{MAX}}(v) \leftarrow \Delta$ 
21:     end if
22:      $v \leftarrow \text{parent of } v$ 
23:   end while
24: end function
25: function SEARCH( $v_0$ )
26:   Create an empty list  $dp_{\text{expert}}$ 
27:    $v \leftarrow v_0$ 
28:   while  $v$  has children do
29:      $v \leftarrow \text{BESTCHILD}(v)$ 
30:     Append  $v$  to  $dp_{\text{expert}}$ 
31:   end while
32:    $\Delta \leftarrow \text{SIMULATE}(dp_{\text{expert}})$ 
33:    $\Delta, dp_{\text{expert}} \leftarrow \text{COMPARECANDIDATES}(dp_{\text{expert}})$ 
34:   return  $dp_{\text{expert}}$ 
35: end function

```

Algorithm 10 Tree Policy

Input: Node v **Output:** Selected node v , tree policy DP sequence dp_{tree}

```

1: function TREEPOLICY( $v$ )
2:   Create an empty list  $dp_{\text{tree}}$ 
3:   Initialize  $terminate \leftarrow \mathbf{False}$ 
4:   while not  $terminate$  do
5:     if  $v$  is not fully expanded then
6:       if build_grid is True then
7:          $v, dp_{\text{tree}} \leftarrow \text{BUILDGRID}(v, dp_{\text{tree}})$ 
8:       else
9:          $v \leftarrow \text{EXPAND}(v, \text{method})$ 
10:      end if
11:    else
12:       $v \leftarrow \text{BESTCHILD}(v)$ 
13:       $terminate \leftarrow \mathbf{True}$ 
14:    end if
15:    Append  $A(v)$  to  $dp_{\text{tree}}$ 
16:  end while
17:  return  $v, dp_{\text{tree}}$ 
18: end function
19: function EXPAND( $v, \text{method}$ )
20:  if  $\text{method} == \text{random}$  then
21:    Choose an untried  $dp$  from  $A$ 
22:  else if  $\text{method} == \text{gaussian}$  then
23:    Sample an untried  $dp$  that is normally distributed around  $\text{ACTION}(v) - 1$ 
24:  end if
25:  Add a new child  $v'$  to  $v$  based on  $dp$ 
26:  return  $v'$ 
27: end function
28: function BESTCHILD( $v$ )
29:  return  $\arg \max_{v' \in \text{children of } v} Q_{\text{MAX}}(v')$ 
30: end function

```

length with continuous DP belief updates is more critical than achieving finer resolution in specific time intervals, making this variant very robust against longer periods of uncertainty.

While MCTS variants with max-child selection lack strong convergence guarantees and risk landing in local minima, this limitation also affects UCT MCTS under capped budgets, such as 165 simulations per rollout as considered here. Despite this, variants using th max-child selection often perform well in generating high-quality DP sequences, balancing computational constraints and decision-making in uncertain traffic scenarios. The candidate backup strategy is employed to mitigate the effects of potential suboptimality.

The root expansion size is set to 15 instead of 5 to ensure better decision quality at the beginning of the traffic scenario when decision postponing is highly likely to be required within the next 30 timesteps. In such cases, the initial DP is critical for performance, as sampling fewer options could result in suboptimal outcomes or even potential failures for the complete rollout. Fully expanding the root with 30 children, however, is excessive and computationally expensive. A root expansion of 15 achieves an optimal balance, offering sufficient information for a robust start in the uncertain traffic situation most of the times.

Gaussian expansion is designed to produce smoother DP sequences, making the optimization proces for the BMPCC more effective due to similar initial guesses. Sampling the next child for a parent at $DP = 30$ with a standard deviation of 14 ensures good coverage of the action space, balancing potential high DP jumps with a Gaussian bias for smoother transitions. Also this variant minimizes unnecessary oscillations in DP values and the ego vehicle's states, improving stability and consistency. Gaussian expansions can recover from incorrectly sampled DPs within a short time, typically 1 to 3 timesteps, and introduce inertia by frequently maintaining the DP value at the maximum or minimum (30 or 1). This approach improves MPC solving times while enhancing overall smoothness and robustness in dynamic traffic scenarios.

Extra updates do not always increase the reward, as the timestep at which a node is expanded does not necessarily align with the timestep where additional exploration significantly impacts the cost. However, using higher-resolution updates (every 0.5 seconds instead of every 1 second) mitigates this issue by enabling more frequent exploration during periods of changing uncertainty, resulting in better-suited DP sequences while maintaining sufficient length within the predefined simulation budget.

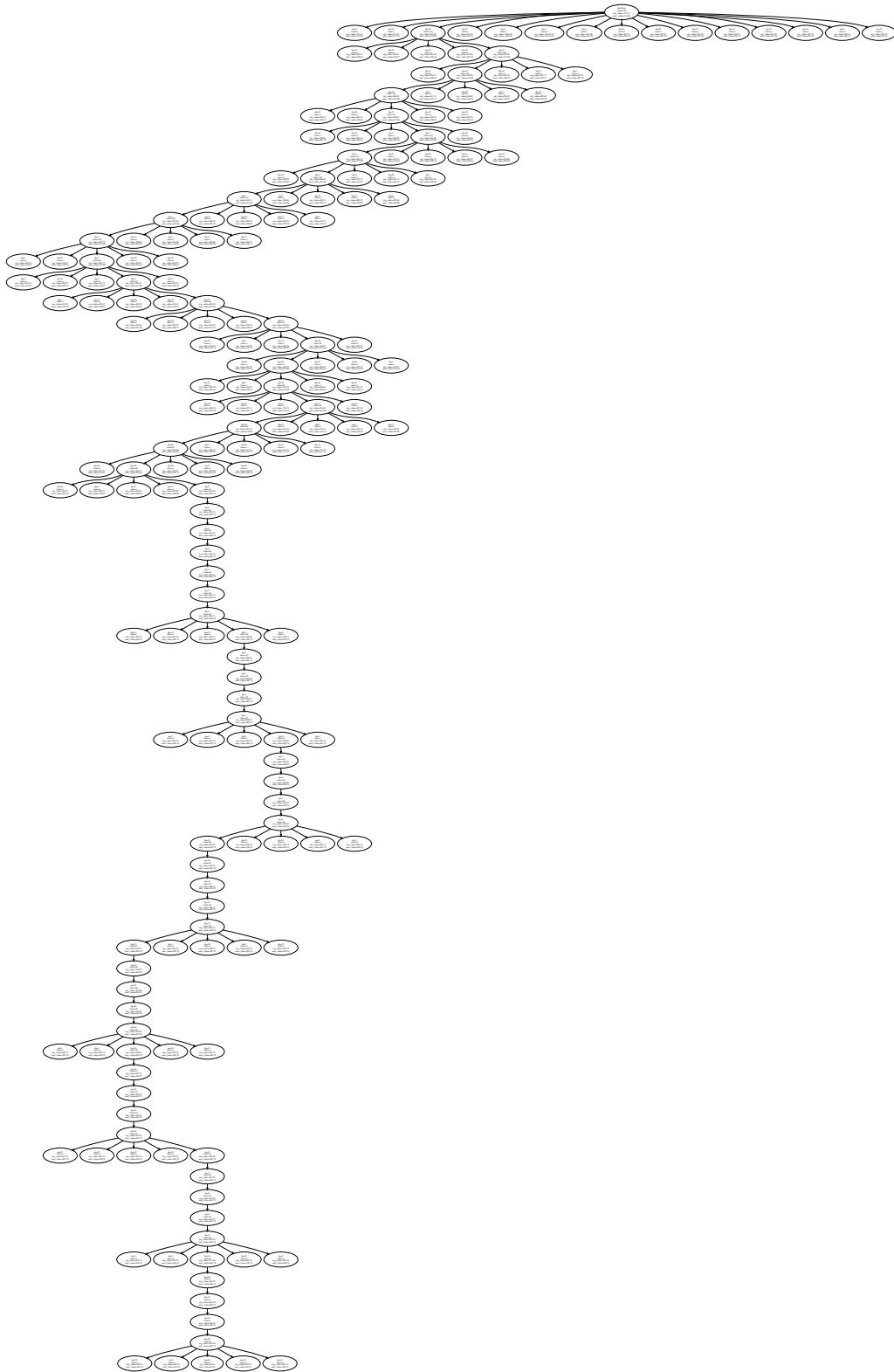


Figure 4.4: Illustration of the full tree constructed by the MCTS expert variant `MAX_VALUE_GAUSSIAN_RES5_WITH_EXPANSION_LIMIT`, exhausting its simulation budget of 165 steps. The tree initially explores broadly, but later uses its budget sparingly, exploring new actions every 0.5 seconds to construct a robust decision postponing tree.

4.2 Learning Decision Postponing Policies

4.2.1 Feature Vector

To train a neural network, a feature vector is constructed to include relevant information about the traffic scene, the traffic participant (TP), the autonomous vehicle (AV), and predictions from the MTR. The feature vector can store information for a single TP, resulting in $\mathbf{f} \in \mathbb{R}^{34}$. For multiple TPs, the feature vector scales as:

$$\mathbf{f} \in \mathbb{R}^{4+30 \cdot N_{\text{TP}}} \quad (4.1)$$

In this section, a single TP is considered for simplicity. An example of the feature vector is shown in Fig. 4.5.

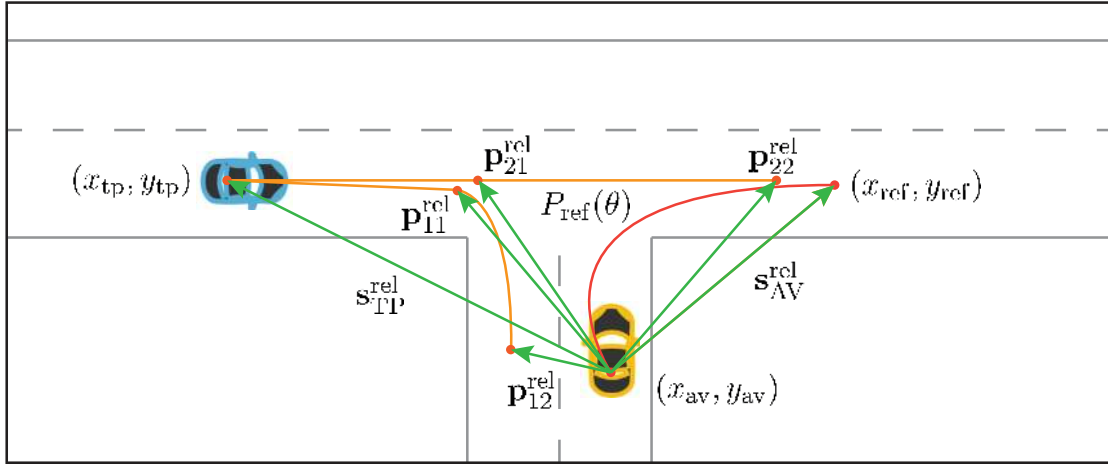


Figure 4.5: The AV is represented as the yellow car, and the TP is represented as the blue car. Illustration of the feature vector components showing the relative AV vector $\mathbf{s}_{AV}^{rel}$ and TP vector $\mathbf{s}_{TP}^{rel}$, as well as the relative components of the prediction vectors $\mathbf{p}_{11}^{rel}, \mathbf{p}_{12}^{rel}, \mathbf{p}_{21}^{rel}, \mathbf{p}_{22}^{rel}$. The reference path of the AV is denoted by $P_{ref}(\theta)$. The AV's position and end reference point are given by (x_{av}, y_{av}) and (x_{ref}, y_{ref}) , whereas the TP position is given by (x_{tp}, y_{tp}) . The feature vector is later used for data collection and policy training.

The feature vector includes the AV state vector $\mathbf{s}_{AV}$, which contains information about the AV's reference path relative to its position, as well as its velocity v_{av} and orientation ψ_{av} . The variables x_{ref} and y_{ref} represent the coordinates of the last point in the reference path $P_{ref}(\theta)$, as described in Sec. 3.1.2:

$$\mathbf{s}_{AV}^{\text{rel}} = \begin{bmatrix} x_{\text{ref}} - x_{\text{av}} \\ y_{\text{ref}} - y_{\text{av}} \end{bmatrix}, \quad \mathbf{s}_{AV}^{\text{abs}} = \begin{bmatrix} \psi_{\text{av}} \\ v_{\text{av}} \end{bmatrix}, \quad \mathbf{s}_{AV} = \begin{bmatrix} \mathbf{s}_{AV}^{\text{rel}} \\ \mathbf{s}_{AV}^{\text{abs}} \end{bmatrix} \in \mathbb{R}^4 \quad (4.2)$$

Similarly, the TP state vector, $\mathbf{s}_{TP}$, includes the relative position of the TP with respect to the AV and its absolute state:

$$\mathbf{s}_{TP}^{\text{rel}} = \begin{bmatrix} x_{\text{tp}} - x_{\text{av}} \\ y_{\text{tp}} - y_{\text{av}} \end{bmatrix}, \quad \mathbf{s}_{TP}^{\text{abs}} = \begin{bmatrix} \psi_{\text{tp}} \\ v_{\text{tp}} \end{bmatrix}, \quad \mathbf{s}_{TP} = \begin{bmatrix} \mathbf{s}_{TP}^{\text{rel}} \\ \mathbf{s}_{TP}^{\text{abs}} \end{bmatrix} \in \mathbb{R}^4 \quad (4.3)$$

Predictions for two scenarios, denoted as $m = 1$ and $m = 2$, are included in the feature vector. These scenarios are generated by the clustering and selection procedure, as explained in Sec. 3.2.3. Each scenario predicts the TP state over 31 timesteps, with key predictions at $k = 15$ (middle) and $k = 30$ (end). The current state of the TP can be extracted from timestep $k = 0$. The middle prediction is particularly important to determine whether the two predictions overlap, thereby ensuring traffic scene context while keeping the feature vector size manageable.

The prediction vectors for the first scenario ($m = 1$) are defined as:

$$\mathbf{p}_{11}^{\text{rel}} = \begin{bmatrix} x_{15,1} - x_{\text{av}} \\ y_{15,1} - y_{\text{av}} \end{bmatrix}, \quad \mathbf{p}_{11}^{\text{abs}} = \begin{bmatrix} \psi_{15,1} \\ v_{15,1} \end{bmatrix}, \quad \mathbf{p}_{11} = \begin{bmatrix} \mathbf{p}_{11}^{\text{rel}} \\ \mathbf{p}_{11}^{\text{abs}} \\ \sigma_{x_{15,1}} \\ \sigma_{y_{15,1}} \end{bmatrix} \in \mathbb{R}^6 \quad (4.4)$$

$$\mathbf{p}_{12}^{\text{rel}} = \begin{bmatrix} x_{30,1} - x_{\text{av}} \\ y_{30,1} - y_{\text{av}} \end{bmatrix}, \quad \mathbf{p}_{12}^{\text{abs}} = \begin{bmatrix} \psi_{30,1} \\ v_{30,1} \end{bmatrix}, \quad \mathbf{p}_{12} = \begin{bmatrix} \mathbf{p}_{12}^{\text{rel}} \\ \mathbf{p}_{12}^{\text{abs}} \\ \sigma_{x_{30,1}} \\ \sigma_{y_{30,1}} \end{bmatrix} \in \mathbb{R}^6 \quad (4.5)$$

where σ_x represents the predicted standard deviation for the x -position, and σ_y represents the predicted standard deviation for the y -position of the TP.

The same structure applies to the second scenario ($m = 2$):

$$\mathbf{p}_{21}^{\text{rel}} = \begin{bmatrix} x_{15,2} - x_{\text{av}} \\ y_{15,2} - y_{\text{av}} \end{bmatrix}, \quad \mathbf{p}_{21}^{\text{abs}} = \begin{bmatrix} \psi_{15,2} \\ v_{15,2} \end{bmatrix}, \quad \mathbf{p}_{21} = \begin{bmatrix} \mathbf{p}_{21}^{\text{rel}} \\ \mathbf{p}_{21}^{\text{abs}} \\ \sigma_{x_{15,2}} \\ \sigma_{y_{15,2}} \end{bmatrix} \in \mathbb{R}^6 \quad (4.6)$$

$$\mathbf{p}_{22}^{\text{rel}} = \begin{bmatrix} x_{30,2} - x_{\text{av}} \\ y_{30,2} - y_{\text{av}} \end{bmatrix}, \quad \mathbf{p}_{22}^{\text{abs}} = \begin{bmatrix} \psi_{30,2} \\ v_{30,2} \end{bmatrix}, \quad \mathbf{p}_{22} = \begin{bmatrix} \mathbf{p}_{22}^{\text{rel}} \\ \mathbf{p}_{22}^{\text{abs}} \\ \sigma_{x_{30,2}} \\ \sigma_{y_{30,2}} \end{bmatrix} \in \mathbb{R}^6 \quad (4.7)$$

Both prediction vectors for the two scenarios are summarized using the cumulative probabilities π_1 and π_2 , which represent the likelihood of each scenario cluster C_s . These probabilities are derived from the scenario selection process, as explained in Sec. 3.2.3. Each scenario's cumulative probability helps ensure that the feature vector accurately reflects the traffic scene's uncertainty and context.

The summarized prediction vectors for the two scenarios are defined as:

$$\mathbf{p}_1 = \begin{bmatrix} \mathbf{p}_{11} \\ \mathbf{p}_{12} \\ \pi_1 \end{bmatrix} \in \mathbb{R}^{13}, \quad \mathbf{p}_2 = \begin{bmatrix} \mathbf{p}_{21} \\ \mathbf{p}_{22} \\ \pi_2 \end{bmatrix} \in \mathbb{R}^{13} \quad (4.8)$$

Combining the AV state vector, TP state vector, and prediction vectors, the complete feature vector is obtained by:

$$\mathbf{f} = \begin{bmatrix} \mathbf{s}_{\text{AV}} \\ \mathbf{s}_{\text{TP}} \\ \mathbf{p}_1 \\ \mathbf{p}_2 \end{bmatrix} \in \mathbb{R}^{34} \quad (4.9)$$

4.2.2 Data Collection

Since the MCTS expert is not real-time capable, an dataset is aggregated, serving as the foundation for learning a decision postponing policy. The neural network is designed to

later predict DP values online, and in real-time.

The data collection process was carried out according to the pseudocode detailed in Algorithm 11. The primary objective was to generate batches of expert DP sequences and their corresponding feature vectors (see Sec. 4.2.1), which were then saved to memory for later processing and use in training.

The inputs to the data collection procedure include the total number of rollouts to be simulated N_{rollouts} , the batch size N_{batch} , a map dictionary M defining relevant paths on the intersection, and the number of traffic participants N_{TP} . The procedure can be summarized in the following way:

1. **Initialization:** An empty batch is initialized to temporarily store the collected data.
2. **Rollout Generation and Expert Query:** For each rollout:
 - The GENERATEROLLOUT function generates a traffic scenario based on the provided map and number of traffic participants using the IDM.
 - The MCTS function is used to compute expert DP sequences and feature vectors for the generated scenario using the Obstacle array and BMPCC object. These data points are logged for additional analysis later, and appended to the current batch.
3. **Batch and Final Saving:** After every N_{batch} rollouts, the batch is saved to memory, and the batch is reset to avoid total loss of simulated data due to a potential crash. Any remaining data in the batch after completing all rollouts is also saved to memory.

Running the data collection process yielded 29 batches with a batch size of 20 rollouts each, resulting in 580 simulated rollouts with expert demonstrations in total. The dataset is titled “*dataset_1_TP_intersection*”, as it focuses on scenarios involving a single TP in intersection-based traffic scenes. The collected data will later be utilized to train a Multi-Layer Perceptron (MLP) capable of reacting to multiple TPs using multiple inference queries.

The final dataset consists of 52,903 pairs of labels and feature vectors. The labels represent the expert DP values at each timestep, while the feature vectors include information about the state of the AV, the TP, and the selected scenarios predicted by the MTR. The average rollout length is 91.21 timesteps as seen in Fig. 4.6. When excluding outliers (rollouts with the maximum length of 151 timesteps), the average rollout length is reduced to 85.82 timesteps. Rollouts reaching the maximum length typically involved either complex traffic scenarios with high uncertainty, where the AV had to brake and

Algorithm 11 Data Collection Procedure

Input: Number of Rollouts N_{rollouts} , Batch Size N_{batch} , Map Dictionary M , Number of Traffic Participants N_{TP}

Output: Batches of Expert DP Sequences and Feature Vectors saved to memory

```

1: function DATACOLLECTION( $N_{\text{rollouts}}, N_{\text{batch}}, M, N_{TP}$ )
2:   Initialize empty Batch  $batch \leftarrow \{\}$ 
3:   for  $i \leftarrow 1$  to  $N_{\text{rollouts}}$  do
4:      $r, R, O, B \leftarrow \text{GENERATEROLLOUT}(M, N_{TP})$  ▷ Algorithm 8
5:      $dp_{\text{expert}}, f_{\text{list}}, log \leftarrow \text{DPMCTS}(N, r, R, O, B, \text{logging}=\text{True})$  ▷ Algorithm 9
6:      $batch \leftarrow batch \cup \{dp_{\text{expert}}, f_{\text{list}}, log\}$ 
7:     if  $i \bmod N_{\text{batch}} = 0$  then
8:       SAVE( $batch$ ) ▷ Save batch to memory
9:        $batch \leftarrow \{\}$  ▷ Reset batch after saving
10:    end if
11:  end for
12:  if  $batch \neq \{\}$  then
13:    SAVE( $batch$ ) ▷ Save remaining data in batch
14:  end if
15: end function

```

reduce its velocity significantly, or incidents such as crashes or aborted maneuvers. Scenarios where the maneuver was aborted resulted in emergency braking due to incorrect decision postponing predictions by the MCTS expert. The dataset comprises the reference combinations elaborated in Sec. 4.1.1, with the specific distribution of reference combinations shown in Fig. 4.7. Rollouts with the same reference combination are present in the dataset, ranging from approximately 28 to 46 occurrences, resulting in a total of 580 rollouts where the expert was queried.

The data collection process spanned approximately 28 days, achieving a speed of approximately 0.02 labels/feature vectors per second. It was executed on a system equipped with an Intel Xeon Gold 6246R CPU and an Nvidia RTX A6000 GPU. While the Monte Carlo Tree Search (MCTS) process, including the nonlinear optimization procedure of the BMPCC using IPOPT [41], ran on the CPU, the MTR performed predictions on the GPU. The nonlinear optimization with IPOPT was accelerated by the solver for systems of linear equations MA57 from the Harwell Subroutines Library (HSL) [42]. However, the data collection process faced several interruptions due to increasing RAM usage in unknown areas of the code base, which made periodic restarts necessary.

The collected dataset includes a significant class imbalance, as illustrated in Fig. 4.8. The majority of DP values, exceeding 31,000 instances, correspond to DP=1, while the remaining values are approximately evenly distributed.

Furthermore, the dataset contains outliers in the standard deviation of the predicted x and y positions of a TP, as shown in the histogram of Fig. 4.9(c), which represents the

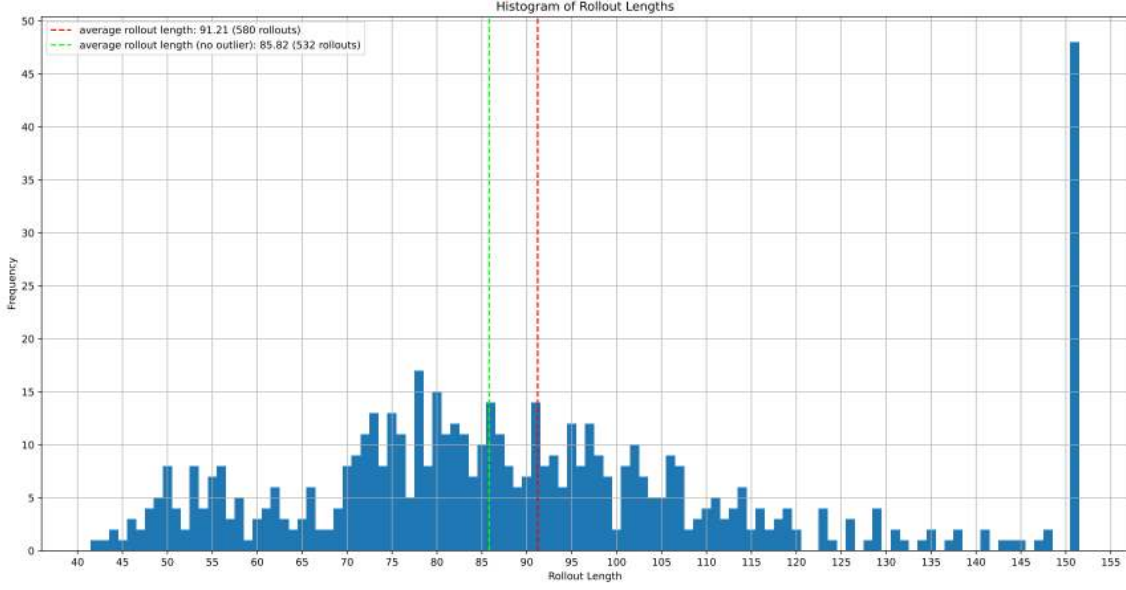


Figure 4.6: Histogram showing the length of the rollouts simulated (in timesteps). The length of the collected rollouts lies in between approx 40 and 151 timesteps. The average rollout length is 91.21 timesteps whereas the average rollout length without outlier rollouts (rollouts that feature the maximum length of 151 timesteps) is 85.82. The outlier rollouts achieved the maximum length of a rollout due to various reasons: Some included complex traffic scenarios with high uncertainty whereas in some a crash occurred.

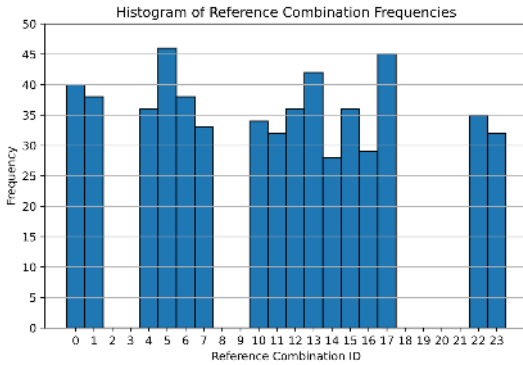


Figure 4.7: Histogram showing the distribution of reference combinations in the simulated 580 rollouts of the data collection. Out of the 24 possible reference combinations, 16 have been utilized, as described in Sec. 4.1.1. All selected reference combinations are approximately equally represented in the dataset.

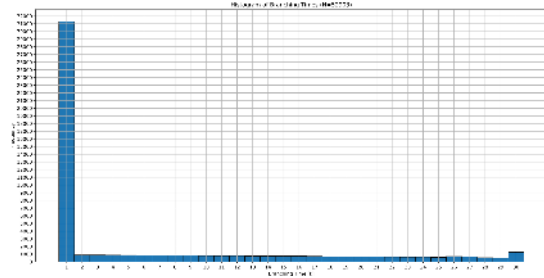


Figure 4.8: Histogram showing the distribution of DP values collected during the data collection process with 580 simulated rollouts. The DP values range from 1 to 30, with a significant class imbalance. The dataset includes over 31,000 instances of DP=1, accounting for the majority of the 52,903 labels in total. Other DP values are approximately equally represented in the dataset, highlighting the strong imbalance in label distribution.

standard deviation of the predicted x position of a TP ($\sigma_{x_{15,1}}$). The primary range of the predicted standard deviation for the x position of a TP lies between 0 and 10 meters. However, outliers extend this range up to 80 meters, indicating unreasonably high

predictions. In contrast, other TP predictions remain reasonable, as demonstrated by the relative x position in the relative vector of the AV (s_{AV}^{rel}) which ranges approximately from negative 50 to positive 40 meters, and the absolute heading angle (ψ_{av}) which lies between negative 4 and positive 6 radians. These attributes are visualized using histograms in Fig. 4.9.

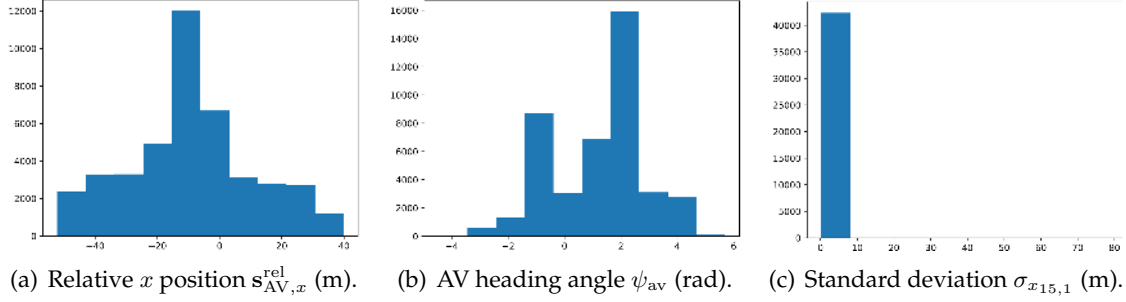


Figure 4.9: The subplots show the relative x distance from the AV to the end of the reference path, ranging between negative 50 and positive 40 meters, the absolute heading angle of the AV, ranging from negative 4 to positive 6 radians, and the standard deviation for the predicted x position of the TP, spanning from 0 to 80 meters. The majority of values for the standard deviation lie below 10 meters, though outliers extend significantly beyond this range.

4.2.3 Behavior Cloning for Decision Postponing

To learn a decision postponing policy, behavior cloning is employed as described in Sec. 3.4.1. For this purpose, the collected data in the form of batches is restructured into a large dataset containing pairs of feature vectors and labels. The feature vectors represent the states of the AV, TP, and selected scenarios predicted by the MTR. The labels are provided by the Monte Carlo Tree Search (MCTS) expert. These pairs are formatted similarly to Eq. 3.59.

Multiple variants of the policy are trained, and the most advanced variant is explained here. Appendix B provides details on other variants, parts of which are also used in the evaluation. The two neural networks considered have opposite capacities: one is relatively small, and the other relatively large, to cover a broader range of capacity. No intermediate-sized network demonstrated desirable results for the task of learning decision postponing, leading to the selection of the following two variants.

Two different Multi-Layer Perceptrons (MLPs) are trained as described in Sec. 3.5.1. The first network, referred to as MLP32, processes an input vector of size 34, corresponding to the feature vector for one TP as described in Sec. 4.2.1. It consists of two hidden layers with 32 neurons each and 30 output neurons. Each output neuron represents a DP value in the range $DP \in \{1, 2, \dots, 29, 30\}$. The specific variant of this network is

named MLP32_1000_LR00005_L2REG00001. Similarly, a larger network, referred to as MLP256, is designed with similar architecture but scaled-up capacity. The specific variant of this network is named MLP256_1000_LR00005_L2REG00001_DA10. Both neural networks are designed for classification tasks, assigning probabilities to the corresponding DP class based on the input feature vector. The sizes of 32 and 256 are referred to as the policy types. Both networks use ReLU as the activation function.

Algorithm 12 Learning a Decision Postponing Policy

Input: Expert data $D = \{(s_{1:T_i}^i, a_{1:T_i}^i)\}_{i=1}^N$, Learning rate α , L2 Regularization Factor $l2reg$, Policy type T , Epochs E

Output: Trained policy π_θ

```

1: function DECISIONPOSTPONINGCLONING( $D, \pi_\theta, T, \alpha, l2reg, E$ )
2:   Setup:
3:    $device \leftarrow \text{GETDEVICE}()$ 
4:    $\pi_\theta \leftarrow \text{INITIALIZEMODEL}(device, T)$ 
5:    $D_{\text{train}}, D_{\text{test}} \leftarrow \text{TRAINTESTSPLIT}(D, \text{test\_size} = 0.2)$ 
6:   Preprocessing and Data Augmentation:
7:    $\text{CLIP}(D_{\text{train}})$  ▷ Clip standard deviation  $\sigma_x, \sigma_y$  (see Sec. 4.2.1)
8:    $\text{SCALE}(D_{\text{train}})$  ▷ Standardization
9:    $\text{DATAAUGMENTATION}(D_{\text{train}}, \text{factor})$  ▷ Add data using rotation
10:  Loss and Optimizer:
11:   $w \leftarrow \text{GETCLASSWEIGHTS}(D_{\text{train}})$  ▷ Address class imbalance
12:   $\mathcal{L}_{\text{CE}} \leftarrow \text{CROSSENTROPYLOSS}(w)$ 
13:   $optimizer \leftarrow \text{ADAM}(\text{PARAMETERS}(\pi_\theta), \alpha, l2reg)$ 
14:  Training Loop:
15:  for  $epoch \leftarrow 1$  to  $E$  do
16:    for  $i \leftarrow 1$  to  $\lceil |D_{\text{train}}|/N_B \rceil$  do
17:       $\{(s_t, a_t)\} \sim D_{\text{train}}$  ▷ Sample a batch of state-action pairs
18:       $\{\hat{a}_t\} \leftarrow \pi_\theta(\{s_t\})$  ▷ Compute a batch of predicted actions
19:       $\mathcal{L}_{\text{CE}} \leftarrow \text{CROSSENTROPYLOSS}(\{a_t\}, \{\hat{a}_t\})$ 
20:       $\theta \leftarrow \text{Adam}(\mathcal{L}_{\text{CE}}, \theta)$  ▷ Update policy parameters
21:    end for
22:  end for
23:  Evaluate on Test Data:
24:  Evaluate  $D_{\text{test}}$  to validate performance
25:  return  $\pi_\theta$ 
26: end function

```

Both neural networks are trained for 1000 epochs using the procedure detailed in Algorithm 12. The complete dataset, consisting of 52,903 feature vectors and corresponding expert labels (described in Sec. 4.2.2), is split into 80% training data and 20% test data. To handle outliers, the standard deviations of the predicted x and y positions, σ_x and σ_y , are clipped to the range $[0, 10]$ meters to facilitate training. To improve the training process, features of all training feature vectors are scaled using standardization. For this, the mean μ_j and standard deviation σ_j of each feature j are computed across the dataset, and the following transformation is applied:

$$\mu_j = \frac{1}{n} \sum_{i=1}^n f_{j,i}, \quad \sigma_j = \sqrt{\frac{1}{n} \sum_{i=1}^n (f_{j,i} - \mu_j)^2}, \quad f'_{j,i} = \frac{f_{j,i} - \mu_j}{\sigma_j}. \quad (4.10)$$

Here, $n = 52,903$ corresponds to the total number of feature vectors in the dataset, and i represents the index of a specific feature vector. For each feature j (where $j \in \{1, 2, \dots, 34\}$), μ_j and σ_j are computed across all feature vectors in the dataset, and $f_{j,i}$ denotes the value of feature j for the i -th feature vector.

To compute the classification error, the cross-entropy loss function as described in Sec. 3.5.2 is used. Class weighting (Sec. 3.5.4) is applied to address the high imbalance in the dataset, where most labels correspond to a decision postponing of $DP = 1$. This ensures the model can also predict other DP classes effectively, avoiding biased and limited prediction capabilities.

The training uses the Adam optimizer with standard parameters (see Sec 3.5.3), except for a learning rate of 0.0005 to achieve smoother batch updates and better convergence at the expense of longer training time. L2 regularization (Sec. 3.5.5) is applied to mitigate overfitting, penalizing high weights. A value of $l2reg = 0.0001$ is used for this purpose. Even the smaller network, MLP32, shows signs of overfitting, and L2 regularization improves overall performance in this case.

To enhance the learning process and simulate additional data, data augmentation techniques (described in Sec. 3.5.6) are incorporated. The data augmentation process is applied to the MLP256 with a factor of 1.0 due to its larger capacity. This involves rotating the relative position vectors in the feature vector using a rotational transformation matrix R for the 2D xy-plane, depending on an angle ϕ :

$$R(\phi) = \begin{bmatrix} \cos(\phi) & -\sin(\phi) \\ \sin(\phi) & \cos(\phi) \end{bmatrix}. \quad (4.11)$$

The relative position vectors $\mathbf{s}_{AV}^{rel}, \mathbf{s}_{TP}^{rel}, \mathbf{p}_{11}^{rel}, \mathbf{p}_{12}^{rel}, \mathbf{p}_{21}^{rel}, \mathbf{p}_{22}^{rel}$ (see Sec. 4.2.1) are rotated using R with angles $\phi \in \{90^\circ, 180^\circ, 270^\circ, -90^\circ, -180^\circ, -270^\circ\}$.

The speed of the AV and TP, as well as the predicted TP speed v , remains unchanged. Heading angles are normalized within the range $[0, 2\pi)$ and are adjusted during augmentation as:

$$\psi' = (\psi + \phi) \mod (2\pi), \quad (4.12)$$

where ψ is the original heading angle, ϕ is the rotational change applied during augmentation, and ψ' is the updated heading angle.

The number of augmented data samples n_{DA} is determined by:

$$n_{\text{DA}} = \left\lceil n \cdot \frac{\text{factor}}{6} \right\rceil, \quad (4.13)$$

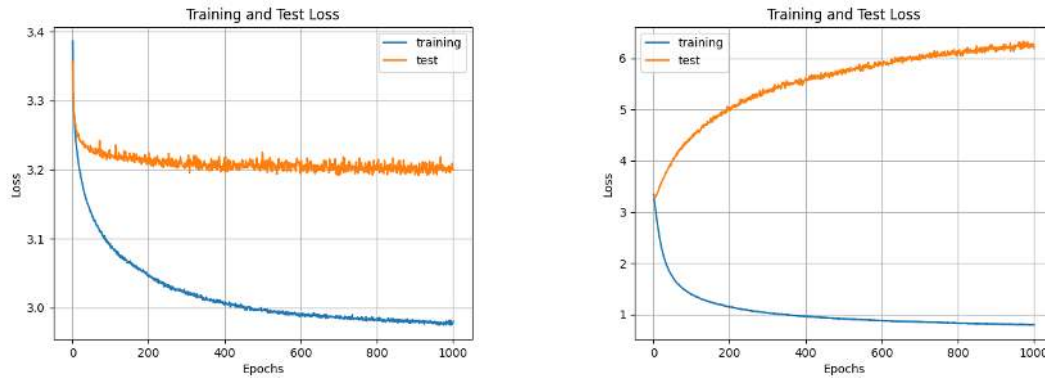
where $n = 52,903$ is the size of the original dataset, and the factor determines the additional data created through rotation. For example, if the factor is set to 1, this would create an additional dataset of the same size as the original by rotating one-sixth of the dataset six times in different ways (using each of the specified angles). Specifically, n_{DA} represents how much of the original data will be rotated to generate augmented data.

The procedure for self-supervised learning of DP policies is achieved by iteratively running the functions `DATA COLLECTION` (Algorithm 11) and `DECISION POSTPONING CLONING` (Algorithm 12). In this approach, expert demonstrations are automatically gathered using the `DATA COLLECTION` function, and the policy is subsequently trained using the `DECISION POSTPONING CLONING` function. By repeating this process iteratively, the policy can be refined in a self-supervised manner without requiring additional manual labeling.

Training and Test Loss

The training and test loss for both MLP32 and MLP256 models is visualized in Fig. 4.10. For the MLP32 model (Fig. 4.10(a)), the training loss decreases smoothly throughout the 1000 epochs, reaching just below 3.0. However, the test loss plateaus at approximately 3.2 after the first 100 epochs and remains high, indicating limited generalization. This behavior suggests that the MLP32 model is underfitting the data. The network's small capacity, with two hidden layers of only 32 neurons each, is insufficient to capture the complexity of the decision postponing prediction task. Both training and test losses remain high, confirming that the model cannot effectively learn the patterns present in the data.

For the MLP256 model (Fig. 4.10(b)), a different behavior is observed. The training loss decreases rapidly, achieving values below 1.0, which indicates that the model can fit the training data effectively. However, the test loss increases consistently after approximately 10 epochs, reaching values over 6.0 by the end of training. This large gap between training and test loss highlights severe overfitting, where the model performs well on the training set but fails to generalize to unseen data. The introduction of L2 regularization, with a value of $l2_{\text{reg}} = 0.0001$, helps mitigate overfitting to some extent, but it is not sufficient to prevent it entirely for the larger-capacity MLP256 network.



(a) Training and test loss for MLP32 with two hidden layers of 32 neurons each. The model exhibits underfitting, as evidenced by the high overall losses for both the training and test sets.

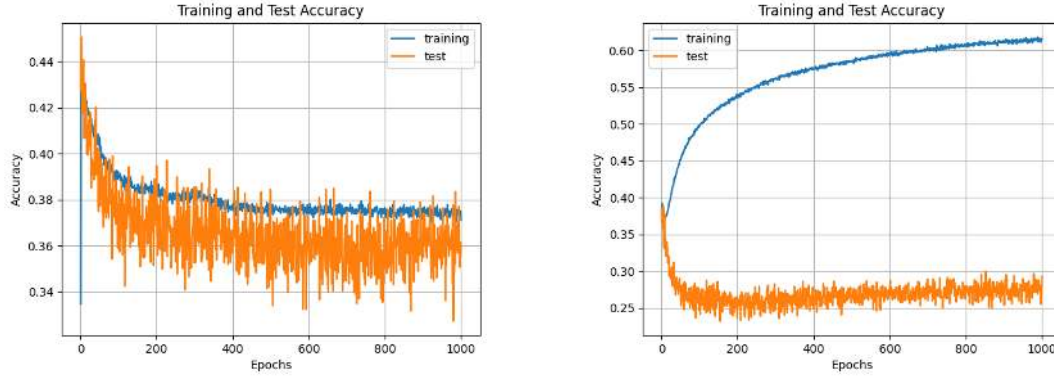
(b) Training and test loss for MLP256 with two hidden layers of 256 neurons each. The model exhibits overfitting, as seen from the low training loss and high generalization loss on the test set.

Figure 4.10: Comparison of training and test losses for the two MLP models with two hidden layers, each using 32 neurons (left) and 256 neurons (right). The MLP32 model demonstrates underfitting due to high overall losses, while the MLP256 model shows overfitting, with a low training loss but a significant gap in test loss, indicating poor generalization. Training both models provides valuable insights into the challenges of balancing model capacity in this domain.

Training and Test Accuracy

The training and test accuracy for the MLP32 and MLP256 models are shown in Fig. 4.11. For the MLP256 model (Fig. 4.11(b)), the training accuracy increases rapidly, reaching approximately 60% by the end of 1000 epochs. However, the test accuracy drops sharply within the first few epochs and stabilizes at a low range around 25–28%, with high fluctuations. This large gap between the training and test accuracy indicates significant overfitting. The MLP256 model, with two hidden layers of 256 neurons each, has a higher capacity that allows it to fit the training data well but fails to generalize to unseen data. Overfitting begins early, around 10 epochs, and is only partially mitigated by the applied L2 regularization with a value of $l2_{reg} = 0.0001$.

For the MLP32 model (Fig. 4.11(a)), the training accuracy decreases gradually, reaching approximately 38% by the end of 1000 epochs. The test accuracy fluctuates heavily around a similar range and does not show significant improvement. This behavior suggests underfitting, as the model struggles to capture the complexity of the data. The limited capacity of MLP32, with two hidden layers of 32 neurons each, restricts its ability to learn meaningful patterns, resulting in low accuracy on both training and test sets.



(a) Training and test accuracy for MLP32 with two hidden layers of 32 neurons each. The model shows underfitting, with low accuracy on both the training and test sets.

(b) Training and test accuracy for MLP256 with two hidden layers of 256 neurons each. The model shows overfitting, with high training accuracy but low generalization accuracy on the test set.

Figure 4.11: Training and test accuracy comparison for two MLP models with two hidden layers, each using 32 neurons (left) and 256 neurons (right). The MLP32 model suffers from underfitting, achieving low accuracy on both training and test data. In contrast, the MLP256 model overfits, having a high accuracy on the training set but poor generalization on the test set.

Training and Test Confusion Matrix

The following figures illustrate the training confusion matrices for the MLP32 and MLP256 models, which display the occurrences with a logarithmic color scale where predicted DP values from the policy (x-axis) coincide with the actual DP values provided by the MCTS expert (y-axis). The diagonal of each confusion matrix represents the frequency with which the policy predicts the exact same DP value as the expert. The closer the values cluster around the diagonal, the better the policy aligns with the expert.

For the MLP32 model (Fig. 4.12), the diagonal is poorly defined, showing that the policy struggles to predict the same DP values as the expert. It achieves some success in predicting DP=1 and DP=30, which are the most common values. Interestingly, the model is also capable of predicting various DP values when the expert DP is 1. This behavior reflects domain knowledge: in certain situations where DP=1 is a desirable decision postponing value, the exact DP value becomes irrelevant for the optimization process because the outcome remains the same due to identical scenarios being selected and similar plans being generated. However, a concerning observation is that the policy frequently predicts DP=1 or DP=30 even when the expert suggests a different DP value, indicating the model's inability to handle complex traffic scenes.

In contrast, the MLP256 model (Fig. 4.13) produces a sharp and well-defined diagonal, demonstrating its ability to predict DP values that closely match those provided by the MCTS expert. The model performs particularly well in understanding the domain

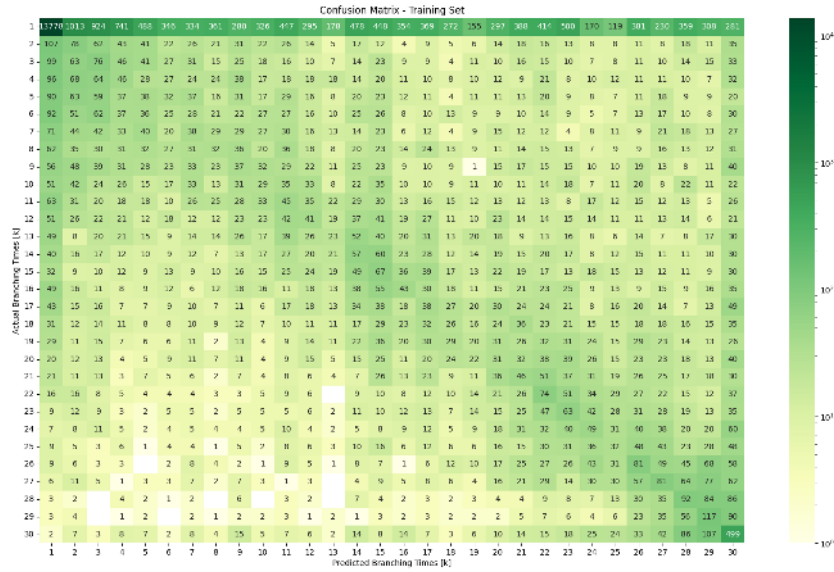


Figure 4.12: Training confusion matrix for MLP32, showing the occurrences with a logarithmic color scale where predicted DP values (x-axis) coincide with the actual DP values from the MCTS expert (y-axis) for a given timestep. The diagonal is poorly defined, indicating that the model struggles to learn and map the feature vectors to the expert DP values effectively.

knowledge: when the expert predicts DP=1, the model more often predicts DP values between 1 and 30, reflecting the irrelevance of precise DP values in certain situations.

Even when the policy does not predict the exact DP value, it often predicts values that are very close to the expert's decision, as seen by the clustering around the diagonal with offsets of 1 or 2. This behavior indicates that the model achieves a nuanced understanding of the task. Additionally, in cases where the expert suggests DP=1, the policy correctly predicts DP=1 most of the time, occurring 25,876 times out of the total 52,903 samples in the dataset.

The following figures show the confusion matrices obtained from the test data. For the MLP32 model (Fig. 4.14), the confusion matrix highlights the model's difficulty in generalizing to unseen data. Most predictions are concentrated at DP=1, with occasional correct predictions for DP=30. This reflects a partial understanding of domain knowledge: in certain traffic situations where DP=1 suffices, the exact DP value may be irrelevant due to identical plans being generated. However, the absence of a clear diagonal shows that the model struggles to align its predictions with the expert's decisions. The scattered nature of predictions suggests underfitting, as the model fails to learn the complex relationships within the decision postponing task.

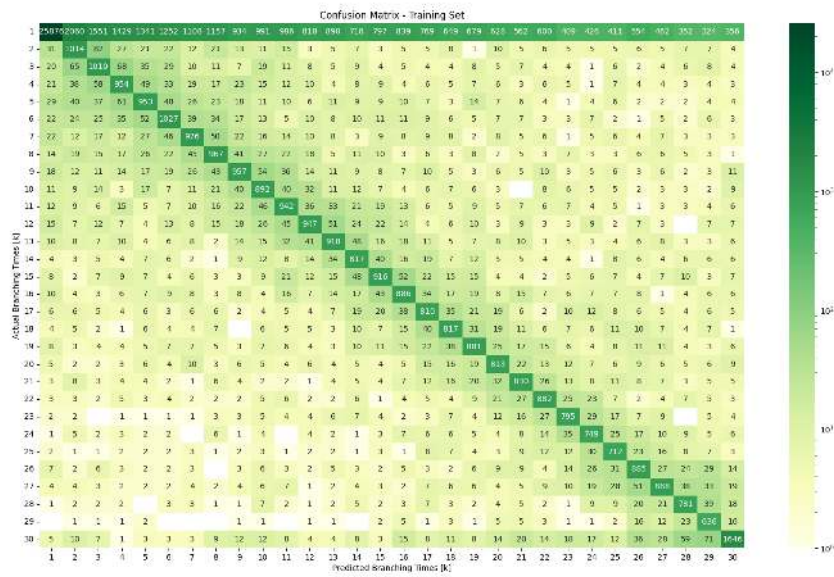


Figure 4.13: Training confusion matrix for MLP256, showing the occurrences with a logarithmic color scale where predicted DP values (x-axis) coincide with the actual DP values from the MCTS expert (y-axis) for a given timestep. The diagonal is sharp and well-defined, demonstrating that the model is able to learn and accurately predict the expert DP values based on the feature vectors.

In contrast, the MLP256 model (Fig. 4.15) demonstrates better performance on the test data. Similar to the training confusion matrix, the model frequently predicts DP=1 and DP=30. Additionally, it predicts various DP values when the expert predicts DP=1, which aligns with domain knowledge. Additionally, there is noticeable clustering of predictions around the diagonal, but less prominent than in the training results. This clustering indicates that while the model does not always predict the exact DP values of the expert, it frequently predicts values that are close, reflecting an improved understanding of the task. However, the diagonal itself remains less defined compared to the training set, highlighting the challenge of generalizing to unseen data. Overfitting likely contributes to this, as the model struggles to replicate the expert’s decisions precisely for new feature vectors. Despite this, the clustering behavior suggests that the MLP256 model captures the underlying task dynamics better than MLP32, making it more effective at approximating the expert’s predictions.

The comparison between the two models underscores the limitations of MLP32 due to underfitting and the stronger performance of MLP256, which exhibits a better understanding of the task but still struggles to fully generalize due to overfitting.

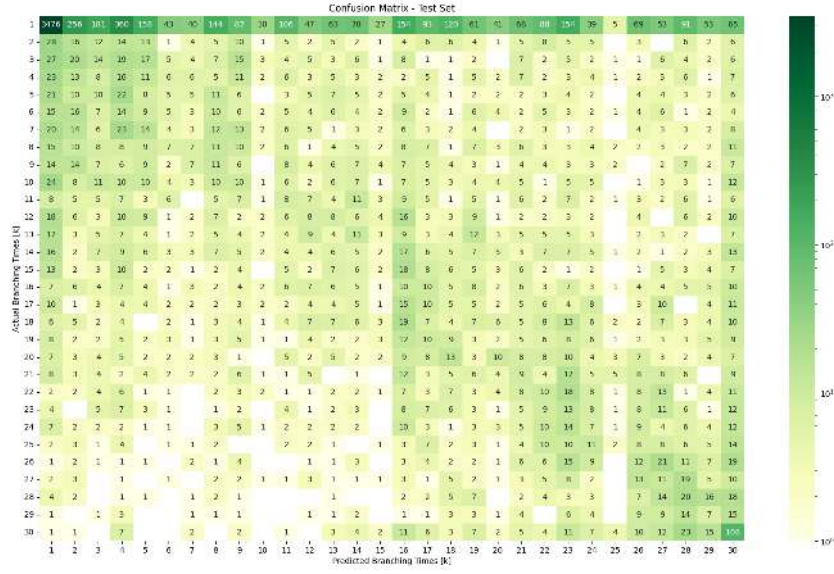


Figure 4.14: Test confusion matrix for MLP32, showing the occurrences with a logarithmic color scale where predicted DP values (x-axis) coincide with the actual DP values from the MCTS expert (y-axis) for unseen feature vectors. The plot lacks a clear diagonal, indicating that the model struggles to generalize and predict DP values effectively for unseen data.

Online Decision Postponing Prediction

Algorithm 13 demonstrates the procedure for predicting DP values online based on feature vectors using a trained policy π .

The process begins with the generation of a rollout via the GENERATEROLLOUT function, which utilizes a pre-defined map dictionary and the number of traffic participants to define the environment and introduce uncertainty in the traffic situation. The SIMULATE function then evaluates the policy π by iteratively constructing feature vectors f for all traffic participants. These vectors are passed to the PREDICT function, where they are clipped, scaled, and fed into the trained neural network model. The model outputs raw scores, which are converted into probabilities using the softmax function, and the DP value corresponding to the highest probability is selected. The maximum predicted DP value across all traffic participants is then used to solve the BMPCC optimization problem, as defined in Eq. 3.20. The resulting output includes the rollout cost C , which is the sum of costs as described in Eq. 3.17, along with an optional log for debugging or further analysis. This algorithm provides a method for predicting DP values, derived from a near-optimal expert, online and in real time, in dynamic traffic scenarios with multimodal uncertainty.

Algorithm 13 Online Prediction of Decision Postponing**Input:** Map Dictionary M , Number of Traffic Participants N_{TP} , Policy π **Output:** Rollout cost C , Log $\log$

```

1: function ONLINEPREDICTION( $M, N_{TP}, \pi$ )
2:    $r, R, O, B \leftarrow \text{GENERATEROLLOUT}(M, N_{TP})$  ▷ Algorithm 8
3:    $C, \log \leftarrow \text{SIMULATE}(\pi, r, R, O, B, \text{logging}=\text{True})$ 
4:   return  $C, \log$ 
5: end function
6: function SIMULATE( $\pi, r, R, O, B, \text{logging}=\text{False}$ )
7:   ... ▷ Scenario selection, etc. (see Sec. 3.2.3)
8:   Initialize empty list  $dp_{\text{list}} \leftarrow \{\}$ 
9:   for  $o \leftarrow 1$  to  $N_{TP}$  do
10:     $f \leftarrow \text{CONSTRUCTFEATUREVECTOR}(o, R, O, B)$  ▷ Sec. 4.2.1
11:     $dp \leftarrow \text{PREDICT}(\pi, f)$ 
12:     $dp_{\text{list}} \leftarrow dp_{\text{list}} \cup \{dp\}$ 
13:   end for
14:    $dp \leftarrow \max(dp_{\text{list}})$ 
15:    $z, u, \text{success} \leftarrow \text{SOLVEBMPCC}(dp, R, O, B)$  ▷ Eq. 3.20
16:   ... ▷ Additional operations, e.g. logging
17:   return  $C, \log$ 
18: end function
19: function PREDICT( $\pi, f$ )
20:    $f \leftarrow \text{CLIP}(f)$  ▷ Clip feature vector to valid ranges
21:    $f \leftarrow \text{SCALE}(f)$  ▷ Scale feature vector for model input
22:    $p_{\text{raw}} \leftarrow \text{MODEL}(f)$  ▷ Eq. 3.65
23:    $p \leftarrow \text{SOFTMAX}(p_{\text{raw}})$  ▷ Convert raw outputs into probabilities
24:    $dp \leftarrow \arg \max_{dp \in \{1, 2, \dots, 29, 30\}} p$  ▷ Select the DP with the highest probability
25:   return  $dp$ 
26: end function

```

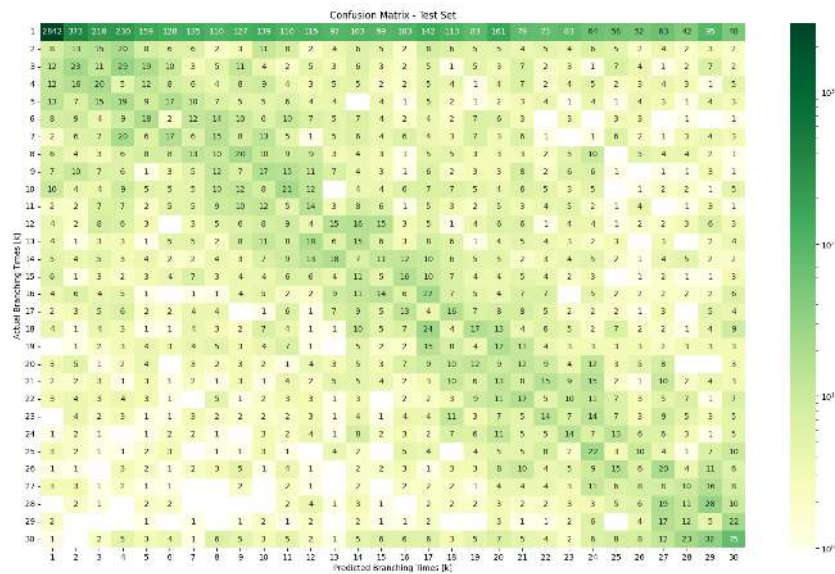


Figure 4.15: Test confusion matrix for MLP256, showing the occurrences with a logarithmic color scale where predicted DP values (x -axis) coincide with the actual DP values from the MCTS expert (y -axis) for unseen feature vectors. The plot exhibits a clear diagonal, indicating that the model generalizes better and outputs DP values similar to those of the expert for the same feature vectors. However, the model does not fully replicate the expert's behavior, as it fails to predict the exact same DP values.

5 Evaluation

This section presents the experimental evaluation conducted to benchmark the baseline methods against various policy variants and the MCTS expert. Specifically, we evaluate the rule-based decision postponing approach detailed in Sec. 3.2.4 and the conventional MPCC, which is equivalent to providing a DP value of 1 to the BMPCC at every timestep. These are compared against the MCTS expert (Sec. 4.1.2) and several policy variants, which include:

- MLP256_1: Derived from MLP256_3, excluding class weighting, L2 regularization, and data augmentation.
- MLP256_2: Similar to MLP256_3, but without data augmentation.
- MLP256_3: As introduced in Sec. 4.2.3, incorporates class weighting, L2 regularization, and data augmentation.
- MLP256_4: Based on MLP256_3, but excludes L2 regularization and data augmentation.
- MLP32_1: A smaller model introduced in Sec. 4.2.3, includes class weighting and L2 regularization.
- MLP32_2: A variation of MLP32_1, without L2 regularization.
- MLP32_3: Derived from MLP32_1, excluding class weighting and L2 regularization.

The evaluation aims to assess the safety, conservativeness, and performance of these approaches. The experiments include scenarios with a single TP and cases with up to three TPs at the intersection, as introduced in Sec. 4.1.1. For each scenario, a quantitative evaluation is performed, consisting of 100 rollouts in total. The primary metrics for evaluation include the mean costs across rollouts, success, abort, and crash rates, as well as the average AV state and control input values, such as mean jerk and velocity. In addition to the quantitative analysis, a qualitative evaluation is conducted for the experiment involving multiple TPs at the intersection. This includes detailed observations of the DP graphs, AV states, and costs to provide further insights into the behavior of the proposed methods.

5.1 Multiple Traffic Participants

In the following, a quantitative evaluation is conducted using 100 sampled rollouts, featuring intersection scenarios with 2 and 3 TPs. Of these, 80 intersection scenarios involve 2 TPs, whereas the remaining 20 scenarios include 3 TPs. The reference lane combinations used by the rollout generator are shown in Table 5.1 and 5.2.

Table 5.1: Combinations of reference lanes for the intersection scenarios in the quantitative evaluation featuring 2 TPs. All reference lanes are shown in Fig. 4.2.

ID	AV	TP1	TP2	ID	AV	TP1	TP2
0	1	4	4	5	4	1	5
1	2	3	5	6	4	1	6
2	2	3	6	7	4	2	5
3	2	4	5	8	4	2	6
4	2	4	6	9	6	4	4

Table 5.2: Combinations of reference lanes for the intersection scenarios in the quantitative evaluation featuring 3 TPs. All reference lanes are shown in Fig. 4.2.

ID	AV	TP1	TP2	TP3	ID	AV	TP1	TP2	TP3
0	2	3	6	6	5	4	1	1	6
1	2	4	4	5	6	4	1	6	6
2	2	4	4	6	7	4	2	2	5
3	2	4	5	5	8	4	2	2	6
4	2	4	6	6	9	4	2	6	6

5.1.1 Quantitative Evaluation

Mean Success, Abort, and Crash Rates

Table 5.3 presents the evaluation results for success rate, abort rate, and crash rate across various policies and baseline methods in intersection scenarios with multiple TPs. Aborted maneuvers indicate that the AV interrupted its planned maneuver and executed an emergency braking, bringing it to a complete stop. A crash describes physical contact with a TP.

The highest success rate 85% is achieved by the expert MCTS, outperforming all other methods. It also demonstrates the lowest abort rate at 8% and shares the lowest crash rate of 7% with several MLP variants. Notably, MLP256_4 achieves a crash rate of only 4%, even surpassing the expert in these complex scenarios with multiple TPs. All learned policies maintain crash rates below 10%, reflecting improved safety compared to the baseline methods, which exhibit a crash rate of 11%.

However, the learned policies display a higher abort rate, averaging around 30%, compared to 16% and 17% for the rule-based baseline and MPCC, respectively. This indicates that the learned policies struggle to fully replicate the expert’s decision postponing in traffic situations involving multiple TPs, as the expert demonstrates roughly half the abort rate of the baselines. This issue arises from the selection of the maximum available DP value for each TP, which serves as an approximation and explains the poorer performance.

In terms of success rates, MLP-based decision postponing policies achieve values between 60% and 70%, with the best-performing variant (MLP32_3) reaching 69%. Nevertheless, these values fall slightly below the baseline methods, which achieve 72% and 73%. This suggests that the decision postponing strategy employed by the MLP variants are not as effective as the near-optimal decision postponing sequence utilized by the expert MCTS.

Table 5.3: Comparison of policies and the MCTS expert against baseline DP approaches in the intersection scenarios with multiple TPs. The table presents the success, abort, and crash rate. Aborted maneuvers indicate that the AV interrupted its maneuver and performed an emergency braking.

Variant	Success [%]	Abort [%]	Crash [%]
MLP256_1	68.0	24.0	8.0
MLP256_2	62.0	31.0	7.0
MLP256_3	62.0	31.0	7.0
MLP256_4	68.0	28.0	4.0
MLP32_1	62.0	30.0	8.0
MLP32_2	57.0	30.0	13.0
MLP32_3	69.0	24.0	7.0
constant-1	72.0	17.0	11.0
expert-mcts	85.0	8.0	7.0
rule-based	73.0	16.0	11.0

Mean Costs and Solving Times

The mean costs (**C1**, **C2**, **C3**), mean times for the BMPCC optimization procedure, as well as the DP calculation (**T1**, **T2**), and the deviation from the expert’s DP sequence (ΔDP) in each timestep are summarized in Table 5.4.

The mean cost **C1** represents the overall cost across all rollouts, including crashes, where the cost after the intersection scenario is the sum of the running costs explained in Eq. 3.17. Since crash costs can vary significantly—ranging from 1,000 to as high as 6,000 or even 11,000—the mean cost **C2** excludes scenarios involving crashes to filter out these extreme values, which can skew the results. Meanwhile, **C3** focuses on costs in scenarios where no aborts or crashes occur, providing insights into the efficiency of successfully

completed maneuvers. For reference, a cost of 20 is considered negligible in this domain, representing a slightly slower AV trajectory, while differences of 100 to 150 can distinguish between an aborted maneuver and a successful one.

The best cost **C1** is achieved by the learned policy variant MLP256_1, with a value of 2364.5. All other policies perform similarly well, except for one instance with a very poor crash scenario, which resulted in a cost exceeding 20,000. The expert MCTS achieves a higher **C1** cost of 8269.4 compared to the rule-based decision postponing method, which achieves 6577.2. The conventional MPCC (without anticipatory driving) has the second-largest mean cost, confirming its reactive driving and highlighting the disadvantage that planning for the most probable outcome is not effective on average, as it does not always align with the real outcome.

When excluding crashes (**C2**), the expert demonstrates the lowest cost of -114.3, significantly outperforming the baseline methods by over 80 in cost difference, which is notable. Among the learned policies, MLP32_3 achieves the best **C2** cost of 19.6, although the policies with the worst success rates also perform poorly in this metric.

For **C3**, which considers only successful rollouts, the best performance is achieved by MLP32.3 (-208.1), followed by the baseline methods and the expert. The learned policies lag behind, unable to replicate the expert's near-optimal DP values accurately. This discrepancy is due to the MAX operator used to select the highest DP value from all TPs, which serves as an approximation and does not fully capture the expert's anticipatory decision-making.

Regarding solving times, all methods exhibit solving times for the BMPCC optimization problem (**T1**) exceeding 1.1 seconds, indicating that the MTR-BMPCC framework struggles in scenarios with up to three TPs. The DP values predicted by the policies require approximately 2.4 ms to be output, demonstrating the real-time capability of the decision postponing approach. The expert's DP computation time is not included as it is calculated offline, whereas the baseline methods achieve DP computation times below 1 ms per DP value.

Furthermore, the mean deviation of the DP sequence (ΔDP) reveals that the baseline approach most closely resembles the expert's near-optimal DP sequence, with a deviation of around 5 per timestep. The policy variant MLP32.3 achieves similar DP sequences to the expert, but all other learned policies display higher deviations. This may result from the MAX operator, which always selects the highest available DP, leading to an approximation that fails to capture the expert's precise behavior.

Table 5.4: Comparison of models and the expert in the intersection scenarios with multiple TPs. The table summarizes mean costs **C1**, **C2**, **C3**, solving times **T1**, **T2**, and the deviation from the expert’s DP in each timestep ΔDP . **C1** represents the mean cost across all rollouts. **C2** indicates the mean cost excluding crashes, while **C3** excludes both abort crashes and regular crashes. **T1** refers to the solving time for the BMPCC optimization problem, and **T2** provides the DP estimation time.

Variant	C1 [-]	C2 [-]	C3 [-]	T1 [ms]	T2 [ms]	ΔDP [-]
MLP256_1	2364.5	80.3	-145.7	1153.7	2.419	7.3
MLP256_2	2911.2	153.2	-134.8	1228.7	2.417	10.7
MLP256_3	20924.2	142.6	-129.6	1141.1	2.417	11.3
MLP256_4	5164.1	197.2	-83.9	1099.0	2.420	9.8
MLP32_1	6363.0	151.3	-153.9	1149.8	2.413	11.0
MLP32_2	4687.3	195.4	-154.6	1284.1	2.410	11.7
MLP32_3	6094.1	19.6	-208.1	1217.8	2.412	6.2
constant-1	14312.1	-27.5	-188.8	1139.3	0.001	5.3
expert-mcts	8269.4	-114.3	-180.7	1105.8	–	–
rule-based	6577.2	-31.1	-193.1	1147.6	0.140	5.5

Mean State and Control Input Values

Table 5.5 presents a comparison of the mean state and control input values in intersection scenarios with multiple TPs. The metrics include the mean velocity μ_v , mean acceleration μ_a , mean jerk μ_j , mean steering angle μ_δ , and mean steering angle velocity $\mu_{\dot{\delta}}$.

The highest success rate, achieved by the expert method, corresponds to the highest mean velocity of 5.862 m/s, indicating the longest distance traveled on average. Among all policies, the lowest mean acceleration is achieved by MLP32_1, with a value of 0.532 m/s², followed closely by other MLP variants. This demonstrates that these learned policies effectively utilize decision postponing to anticipate multiple scenarios, resulting in safer driving behavior. In contrast, the baseline methods exhibit higher acceleration and braking values, reflecting a less anticipatory and more reactive driving style. The lower accelerations of the MLP policies suggest they are more future-oriented and provide a more comfortable driving experience. The lowest mean jerk, indicating the smoothest changes in acceleration, is achieved by the rule-based method and the expert method, with values of 0.683 m/s³ and 0.686 m/s³, respectively.

For the mean steering angle μ_δ , the best performance is observed in MLP256_2, MLP256_4, and MLP32_1, all achieving a value of 0.073 rad. Regarding the mean steering angle velocity $\mu_{\dot{\delta}}$, the expert method achieves the best result with a value of 0.061 rad/s, indicating precise control and minimal effort for steering inputs, further confirming the anticipatory capabilities enabled by the expert’s decision postponing.

Table 5.5: Comparison of mean state and control input values in intersection scenarios featuring multiple TPs. The columns represent the velocity μ_v , acceleration μ_a , jerk μ_j , steering angle μ_δ , and steering angle velocity $\mu_{\dot{\delta}}$.

Variant	μ_v [m/s]	μ_a [m/s ²]	μ_j [m/s ³]	μ_δ [rad]	$\mu_{\dot{\delta}}$ [rad/s]
MLP256_1	5.529	0.573	0.768	0.080	0.076
MLP256_2	5.505	0.561	0.808	0.073	0.078
MLP256_3	5.509	0.586	0.833	0.079	0.083
MLP256_4	5.477	0.560	0.804	0.073	0.079
MLP32_1	5.459	0.532	0.762	0.073	0.081
MLP32_2	5.382	0.538	0.834	0.079	0.090
MLP32_3	5.614	0.566	0.720	0.079	0.086
constant-1	5.625	0.603	0.707	0.083	0.069
expert-mcts	5.862	0.594	0.686	0.078	0.061
rule-based	5.581	0.592	0.683	0.080	0.068

5.1.2 Qualitative Evaluation

In the following, a qualitative evaluation is conducted for the case with 2 TPs in the intersection scenario. For this, the first rollout from the quantitative evaluation (Sec. 5.1.1) is considered.

Figure 5.1 depicts an animation of the selected intersection scenario with 2 TPs, presented as multiple snapshots of the situation. In this particular rollout (ID=0), the AV is following reference lane 3, while the first TP is on reference lane 5 and the second TP is on reference lane 2. The first TP could potentially collide with the AV if it proceeded straight; however, as predicted by the MTR framework, this does not happen. The second TP crosses the AV's reference path, demanding careful coordination and timing.

In this scenario, the AV executes the DP sequence provided by the expert. Initially, the AV slows down slightly in response to the proximity of the first TP, demonstrating caution and adaptability. It then accelerates to the maximum speed of 8 m/s, allowing the second TP to safely pass behind it. This behavior reflects a cautious yet confident driving style that avoids being overly conservative while maintaining safety and efficiency. The intersection scenario is successfully completed, showcasing the effectiveness of the expert's DP sequence and its ability to safely handle complex multi-TP scenarios.

Costs

The costs for the baseline methods, the MCTS expert, and various learned policies in this rollout with multiple TPs are shown in Fig. 5.2. The expert achieves the lowest cost, just over 100, successfully avoiding a collision and exiting the intersection. Among the learned policies, MLP256_1 achieves the second-lowest cost of approximately 600, suc-

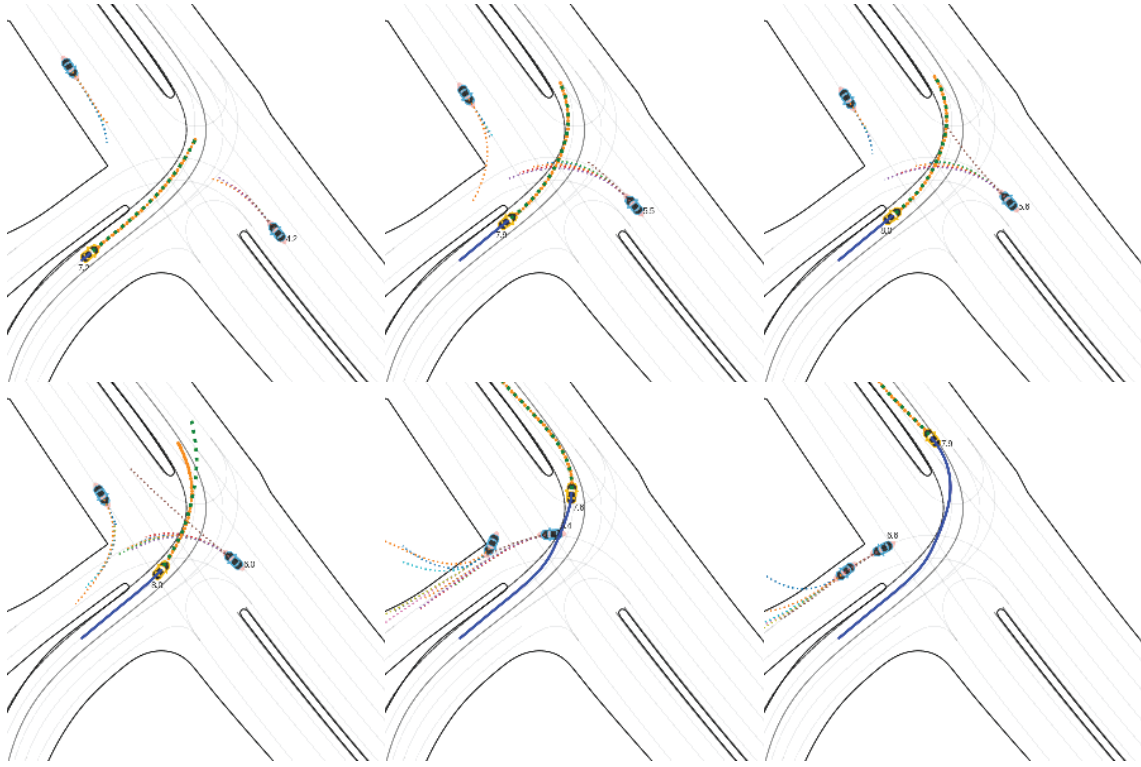


Figure 5.1: Traffic scenario with 2 TPs where the expert’s DP sequence is executed. The scenarios predicted by the MTR and selected by the BMPCC framework are contradictory. Decision postponing is employed wisely to wait for the uncertainty to resolve, allowing a solution to be found. The expert successfully completes this rollout; however, the baselines and most of the MLP models fail.

cessfully executing a slight braking maneuver followed by reacceleration. In contrast, the other learned policies and baseline methods incur significantly higher costs, around 1400 or more, partially failing. This includes the rule-based approach and MPCC, demonstrating that this rollout is particularly challenging. Overall, the results highlight the limitations of the MLP-based decision postponing in the presence of multiple TPs.

Decision Postponing Values

Figure 5.3 depicts the DP values over the timesteps k for rollout 0, with two TPs at the intersection. The x-axis represents the timesteps, while the y-axis indicates the DP values selected by each policy, including the MLP variants, baseline methods, and the expert MCTS.

The rule-based approach exhibits minimal use of decision postponing, with DP values primarily remaining low between timesteps 10 and 30, although peaks are observed at DP values of 11 and 29. This indicates occasional adjustments based on the evolving traffic scene but lacks frequent reassessment. The MCTS expert, on the other hand, demon-

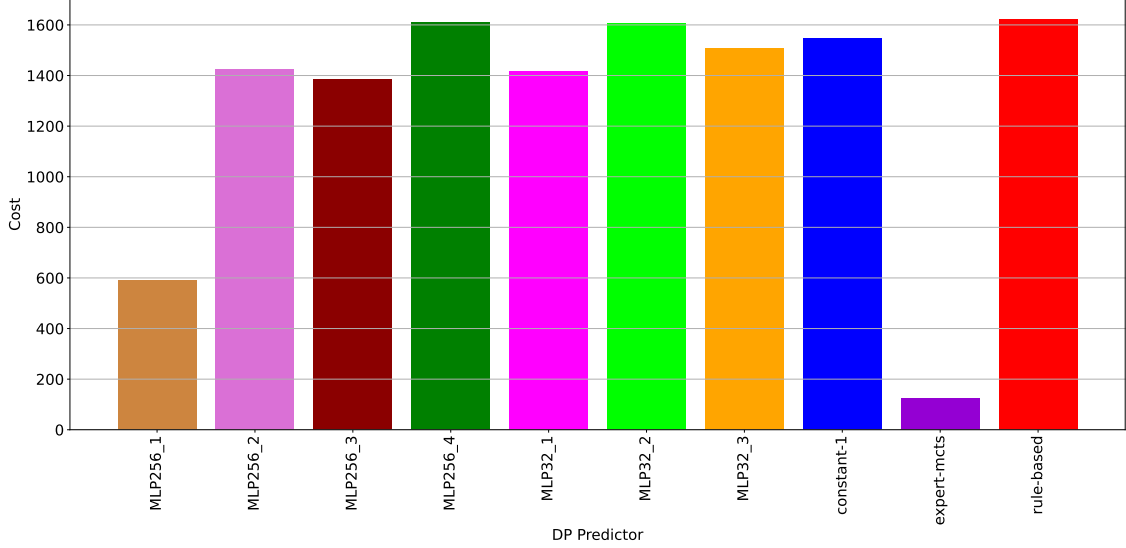


Figure 5.2: Costs for rollout 0 of the qualitative evaluation with multiple TPs. Most variants result in a crash, with costs exceeding 1000. The MCTS expert achieves the lowest cost, just over 100, successfully avoiding a collision and exiting the intersection. One variant of the MLP256 policy manages to execute a braking maneuver and then reaccelerate, achieving a significantly lower cost of approximately 600 compared to the other policies and baseline methods.

states dynamic decision postponing, starting with a medium DP value when it is uncertain whether the first TP will accelerate. The DP value is subsequently reduced but increases again as the AV approaches the second TP, indicating the need for additional caution. After the uncertainty surrounding the second TP resolves around timestep 35, the expert reduces the DP value to near zero, showcasing its near-optimal behavior. Starting from timestep $k = 23$, the additional updates, which occur every 5 timesteps, become visible. This approach allows the MCTS expert, as outlined in Sec. 4.1.2, to use its simulation budget sparingly after a phase of relatively thorough exploration, enabling an increase in its tree depth and, consequently, the DP sequence length.

The MLP256_1 policy also begins with a relatively high DP value and continuously updates its belief as the rollout progresses. At timestep $k = 40$, it reduces the DP value to zero when passing the TPs. Other policies exhibit stronger oscillations in their DP values, indicating less stable decision postponing compared to the expert. Most policies, however, similarly estimate DP values overall. Notably, many policies, such as MLP32_3, overestimate the uncertainty at the beginning of the rollout, maintaining a maximum DP value of 30 from timestep $k = 1$ to $k = 13$.

State and Control Input Values

Figure 5.4 illustrates the evolution of the states and control inputs of the AV over time during rollout 0 of the quantitative evaluation with two TPs at the intersection. The subplots represent the following states and control inputs: x-position, y-position, heading angle ψ , longitudinal velocity v_x , longitudinal acceleration a_x , steering angle δ , longitudinal jerk j_x , and steering angle velocity $\dot{\delta}$.

As discussed previously and shown in Fig. 5.3, many policy variants fail to estimate the DP values correctly, overestimating the uncertainty at the beginning of the traffic scenario and later into the scenario between timesteps $k = 40$ and $k = 60$. For instance, MLP32_3 decides to use maximum decision postponing, resulting in the AV being the slowest after 2 seconds into the scenario. Then, when the second TP's predictions are contradictory, creating high uncertainty, the variant MLP32_3 uses no decision postponing at all (DP = 1). This leads to the latest braking maneuver across all variants, showing overly reactive driving, thus resulting in an emergency braking of this variant when the predicted outcome of the TP does not occur.

The expert, however, estimates the DP values effectively, moderately accelerating to reach the maximum velocity and successfully passing the first TP while managing its uncertainty. It later reduces its speed to around $v_x \approx 7.5$ at approximately timestep 40 due to the contrary predictions of the second TP. At $k = 44$, the expert decides to employ a DP value of 16, making it possible to brake just slightly to wait for the uncertainty of the second TP to resolve, then again accelerating to the maximum speed and smoothly navigating the curve of the intersection, demonstrating excellent performance in the face of 2 TPs. The second-best performing variant, MLP256_1, reduces its speed to just over 6 m/s to account for the second TP. This adjustment compensates for its initial overestimation of uncertainty, allowing the AV to pass the second TP smoothly before accelerating to its maximum velocity and exiting the intersection. MLP32_1, the first variant of the MLP32 family, also overestimates uncertainty at the start (as seen in Fig. 5.3), though less significantly than the failing variants. Consequently, it decelerates to under 4 m/s, slower than MLP256_1, to wait until the uncertainty created by the second TP resolves. The AV then completes the intersection scenario at approximately $t = 11$ seconds.

The expert demonstrates the most comfortable and smoothest driving as seen in the acceleration graph. The longitudinal acceleration ranges between approximately $a_x = -1 \text{ m/s}^2$ and $a_x = +0.5 \text{ m/s}^2$. The other succeeding variants, MLP32_1 and MLP256_1, have significantly higher acceleration curves due to their stronger braking and less efficient decision postponing. The acceleration of MLP32_1 ranges between -2.5 m/s^2 and around $+1 \text{ m/s}^2$, while the acceleration of MLP256_1 ranges between around -3 m/s^2 and $+2 \text{ m/s}^2$. The high acceleration of MLP256_1 can be explained by the AV trying to catch the opportunity to drive in front of the second TP, which is not entirely safe. This sit-

uation occurs due to its oscillating DP curve, which is also often too close to $DP = 1$. The other failing variants similarly brake with an acceleration peaking at around -3 m/s^2 , and then, when it becomes clear that the second TP cannot be passed, they perform an uncomfortable braking maneuver with an acceleration of $a_x \approx -4 \text{ m/s}^2$.

The plot showing the longitudinal jerk is similar to the one showing longitudinal acceleration. Notably, the expert achieves the smallest jerk deviations in this uncertain traffic scenario, demonstrating good driving comfort and performance compared to other policy variants and baseline methods. This underscores the expert's ability to effectively balance safety, efficiency, and comfort in the face of uncertainty. The other succeeding variants, MLP32_1 and MLP256_1, reach the saturation of $a_x = \pm 3 \text{ m/s}^2$ due to their less performant decision postponing. Similarly, all failing variants land in the saturation of jerk too due to their emergency braking.

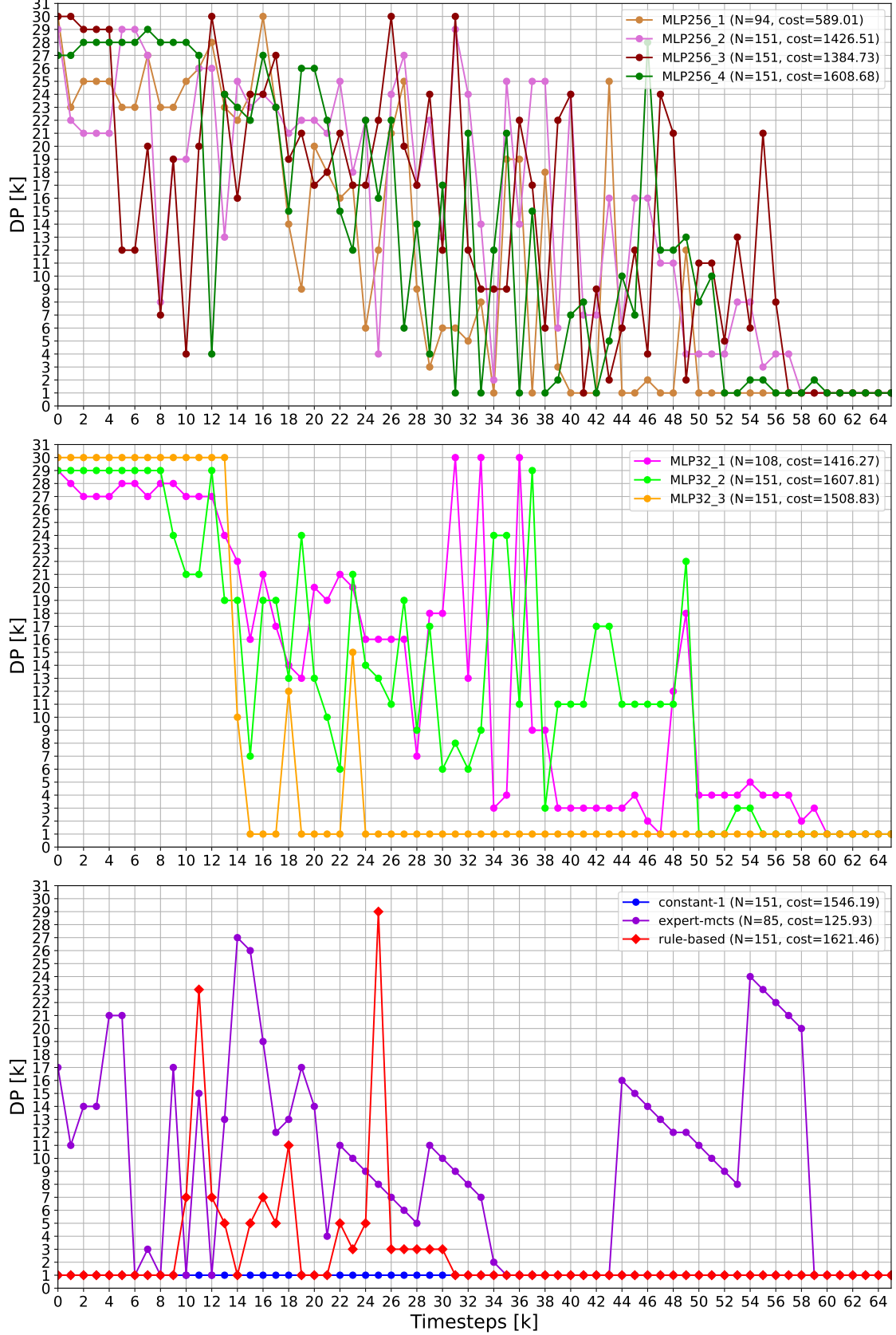


Figure 5.3: History of DP values for rollout 0 with two TPs at the intersection. The x-axis represents the timesteps, while the y-axis shows the DP values selected by each MLP variant, baseline, or expert. The expert MCTS adapts its DP values more frequently, dynamically reassessing the situation to make informed decisions. In contrast, the rule-based approach primarily uses a static DP value of 1, with minimal adjustments early on. The DP sequence plays a critical role within the BMPCC framework, influencing the ability to wait until a traffic scene’s uncertainty resolves.

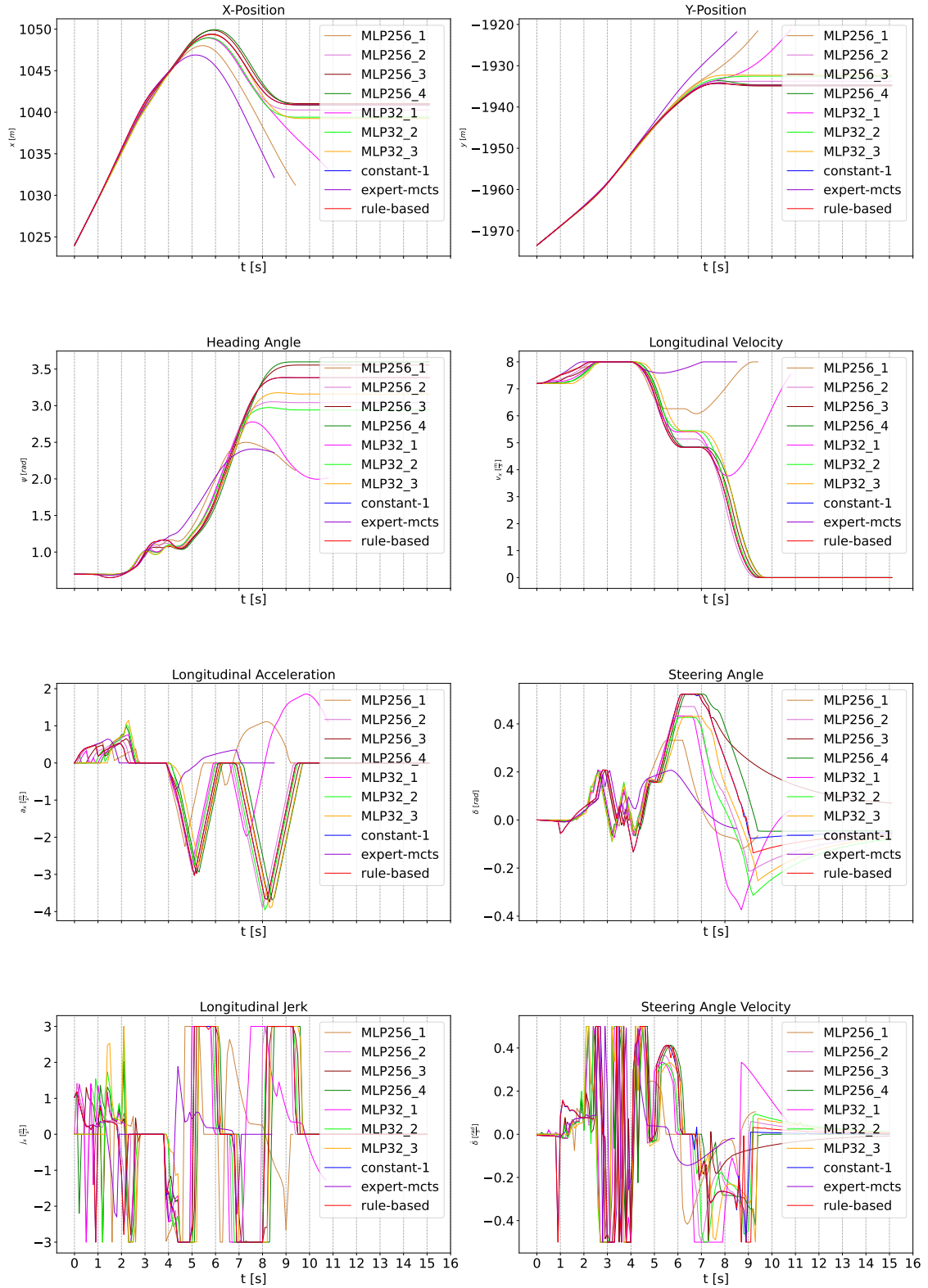


Figure 5.4: State and control input evolution over time for rollout 0 with two TPs at the intersection. The subplots depict the following states and control inputs: x -position, y -position, heading angle ψ , longitudinal velocity v_x , longitudinal acceleration a_x , steering angle δ , longitudinal jerk j_x , and steering angle velocity $\dot{\delta}$. The expert reduces its speed to approximately to 7.5 m/s before accelerating again, successfully navigating the scenario. In contrast, most variants fail to resolve the traffic situation, ending the simulation with a standstill.

5.2 Single Traffic Participant

In the following, a quantitative evaluation is conducted featuring 100 sampled rollouts with a single TP at the intersection. The reference lane combinations employed by the rollout generator in this evaluation are given by Table 4.1.

Mean Success, Abort, and Crash Rates

Table 5.6 presents the success rates, abort rates, and crash rates of various policies, the MCTS expert, and baseline methods in intersection scenarios with a single TP.

The decision postponing strategy from the expert achieves a success rate of 90%, which is one of the best among all approaches. Even better are the policy variants MLP256_2 and MLP32_1, both achieving success rates of 92%. These results demonstrate that the expert and these policy variants outperform or match the baseline methods, indicating superior overall performance. This can be attributed to the fact that, in the single-TP scenario, policy variants utilize their full potential, avoiding the need for DP value approximation through the MAX operator required in multi-TP cases. The variants with the highest success rates, MLP256_2 and MLP32_1, also exhibit the lowest abort rates, at 7% and 4%, respectively. The expert achieves an abort rate of 9%, which is comparable but slightly higher. Most policy variants outperform the baseline methods, which have abort rates of 10% and 12%. An exception is MLP256_4, which has an abort rate of 11%, possibly due to its focus on DP values near 1 and the omission of class weighting. In terms of safety, the expert demonstrates the lowest crash rate of 1%, matched by the policy variants MLP256_2 and MLP256_4. The rule-based approach has a higher crash rate of 3%, which may be caused by oscillations in the DP values.

A comparison with Table 5.3 highlights the reliability of the expert's performance across scenarios. The expert maintains a success rate of 90% in the single-TP case and only drops to 85% in the multi-TP case. In contrast, baseline methods exhibit a more significant performance degradation: the rule-based approach decreases from 85% to 73%, and the conventional MPCC drops from 88% to 72%. These reductions of 16% and 13%, respectively, underscore the limitations of the baseline methods. The expert, with only a 5% performance drop, provides near-optimal DP values in both scenarios, including the particularly challenging multi-TP case.

Mean Costs and Solving Times

Table 5.7 provides a comparison of the mean costs across rollouts (C1), the mean costs excluding crashes (C2), the mean costs excluding both crashes and aborts (C3), the solving

Table 5.6: Performance comparison of policies and the MCTS expert against baseline DP approaches in the intersection scenarios with a single TP. The table presents the success, abort, and crash rate. Aborted maneuvers indicate that the AV executed an emergency braking.

Variant	Success [%]	Abort [%]	Crash [%]
MLP256_1	90.0	8.0	2.0
MLP256_2	92.0	7.0	1.0
MLP256_3	90.0	8.0	2.0
MLP256_4	88.0	11.0	1.0
MLP32_1	92.0	4.0	4.0
MLP32_2	89.0	9.0	2.0
MLP32_3	89.0	9.0	2.0
constant-1	88.0	10.0	2.0
expert-mcts	90.0	9.0	1.0
rule-based	85.0	12.0	3.0

times for BMPCC optimization (**T1**), the time required to predict a DP value (**T2**), and the deviation from the expert's DP values (**Δ DP**) across intersection scenarios featuring one TP.

The top three mean costs (**C1**) in the single-TP case are achieved by MLP256_2 with **-4.9**, MLP256_4 with 20.1, and the expert with 24.5. These results indicate better performance, as well as anticipatory and efficient driving, compared to the baseline methods, where the rule-based approach scores 163.1 and the conventional MPCC scores 180.4 on average. These high costs for the baseline methods confirm their reactive nature, which relies on responding to the most probable outcome. As a result, these methods frequently require emergency braking or crash when the actual outcome deviates from predictions made by the MTR. Certain policy variants, such as MLP256_1 and MLP256_3, have significantly higher mean costs exceeding 3000. This is due to the inclusion of crash costs, which can heavily skew the results. Although these variants have a crash rate of only 2%, similar to other approaches, the variability in crash costs leads to higher average costs, making comparisons difficult.

Excluding scenarios with crashes (**C2**), MLP32_1 achieves the best performance with a mean cost of -197.6, followed by the expert with -166.1. This indicates that the DP sequences provided by both the policy and the expert outperform those of the baseline methods, whose mean costs are higher than those of all other policy variants. When considering only successfully completed driving scenarios (**C3**), the expert again demonstrates the best performance with a mean cost of -245.8. Other policy variants and baseline methods perform similarly in this category, with mean costs between approximately -220 and -240.

The solving times for the BMPCC optimization problem (**T1**) range between approxi-

mately 120 ms and 150 ms, with MLP256_1 achieving the fastest computation time on average at 119.7 ms. It should be noted, however, that *IPOPT* was not configured with JIT compilation due to issues with the workstation, preventing real-time performance for the MTR-BMPCC framework. DP inference times (**T2**) are reduced to approximately 1.3 ms for the policy variants. The rule-based approach requires only 0.1 ms in the single-TP case.

The smallest deviation from the expert's DP values (ΔDP) is achieved by MLP256_1 with an average absolute difference of **5.016**. This demonstrates that, in the single-TP case, the policy variants reflect the expert's DP estimation more accurately than in the multi-TP case due to the missing requirement of the MAX operator.

Table 5.7: Comparison of policies and the expert in the intersection scenarios with a single TP. The table summarizes mean costs **C1**, **C2**, **C3**, solving times **T1**, **T2**, and the deviation from the expert's DP in each timestep ΔDP . **C1** represents the mean cost across all rollouts. **C2** indicates the mean cost excluding crashes, while **C3** excludes both abort crashes and regular crashes. **T1** refers to the solving time for the BMPCC optimization problem, and **T2** provides the DP estimation time.

Variant	C1 [-]	C2 [-]	C3 [-]	T1 [ms]	T2 [ms]	ΔDP [-]
MLP256_1	3469.8	-165.9	-238.3	119.7	1.358	5.016
MLP256_2	-4.9	-156.8	-231.0	132.9	1.360	6.472
MLP256_3	3042.4	-147.9	-219.8	140.3	1.357	7.198
MLP256_4	20.1	-138.1	-234.2	147.6	1.357	6.463
MLP32_1	219.7	-197.6	-239.7	134.8	1.358	6.931
MLP32_2	87.1	-143.9	-220.3	136.9	1.375	7.919
MLP32_3	59.9	-142.7	-224.6	121.3	1.354	6.364
constant-1	180.4	-122.1	-215.2	134.5	0.001	6.992
expert-mcts	24.5	-166.1	-245.8	130.3	–	–
rule-based	163.1	-114.2	-240.2	136.5	0.109	7.112

Mean State and Control Input Values

Table 5.8 lists the mean states and control inputs of the AV across rollouts in intersection scenarios with a single TP. The columns include the mean velocity μ_v , mean acceleration μ_a , mean jerk μ_j , mean steering angle μ_δ , and mean steering angle velocity $\mu_{\dot{\delta}}$.

The expert achieves the highest mean velocity of 6.088 m/s, representing the longest distance traveled on average. Following the expert, the second-best performance is achieved by the policy variants, while the baseline methods take the last positions. The slowest performance is exhibited by the conventional MPCC with a mean velocity of 5.820 m/s, primarily due to its lack of anticipatory driving capabilities.

Among the policy variants, MLP32_1, which has the highest success rate, also achieves

the lowest mean acceleration of 0.508 m/s^2 . The smallest mean jerk (0.597 m/s^3) is driven by the conventional MPCC, which does not account for multiple scenarios. The mean steering angle (μ_δ) and mean steering angle velocity ($\mu_{\dot{\delta}}$) are approximately equal across all approaches, with minimal variations.

Table 5.8: Comparison of mean state and control input values in intersection scenarios featuring a single TP. The columns represent the velocity μ_v , acceleration μ_a , jerk μ_j , steering angle μ_δ , and steering angle velocity $\mu_{\dot{\delta}}$.

Variant	μ_v [m/s]	μ_a [m/s ²]	μ_j [m/s ³]	μ_δ [rad]	$\mu_{\dot{\delta}}$ [rad/s]
MLP256_1	6.046	0.561	0.656	0.079	0.058
MLP256_2	6.010	0.540	0.652	0.077	0.058
MLP256_3	5.939	0.519	0.650	0.077	0.059
MLP256_4	5.917	0.517	0.641	0.075	0.057
MLP32_1	5.932	0.508	0.623	0.076	0.056
MLP32_2	6.015	0.527	0.655	0.076	0.065
MLP32_3	5.960	0.528	0.613	0.081	0.059
constant-1	5.820	0.567	0.597	0.080	0.055
expert-mcts	6.088	0.543	0.615	0.079	0.059
rule-based	5.864	0.556	0.621	0.080	0.059

6 Discussion

6.1 Results

The proposed approach demonstrates competitive performance, particularly in scenarios with a single traffic participant (TP), where policy variants match or surpass baseline methods, such as the rule-based decision postponing method from the MTR-BMPCC framework [8] and the conventional MPCC. Policies such as MLP256.2 and MLP32.1 achieve higher success rates, improved crash metrics, and reduced abort rates, effectively replicating expert-like behavior. Moreover, incorporating techniques like class weighting, L2 regularization, and data augmentation results in slightly better performance compared to other policies.

In multi-TP scenarios, however, using a single neural network with the MAX operator to approximate decision postponing (DP) behavior limits performance in handling multiple participants. This creates a performance ceiling where policies fail to fully replicate the expert’s anticipatory decision postponing abilities. Mixed trends in comfort and jerk metrics, along with contradictory crash cost outcomes, further complicate direct comparisons.

6.2 Limitations

The approach faces several notable limitations spanning data quality, model architecture, and computational constraints. A key issue lies in the limited generalization of policies to novel scenarios, likely due to both the neural network architecture and the quality as well as quantity of the training data. The dataset was derived from a single intersection with a fixed set of reference path combinations (Sec. 4.1.1), resulting in a lack of diversity in traffic situations. Additionally, the IDM model used for generating rollouts does not account for road curvature, rendering scenarios less realistic. These factors contributed to an overrepresentation of highly certain scenarios, leading to a disproportionate number of $DP = 1$ instances in the dataset. This lack of variety significantly hinders the model’s ability to generalize effectively.

Another limitation is the computational constraints imposed by the simulation budget for

the MCTS expert. While the MCTS expert employs a general expansion limit of 5 nodes to enable exploration throughout, this restriction occasionally forces the DP sequence into local minima. A higher simulation budget would allow for better exploration, and using UCT could produce more optimal results due to its theoretical guarantees, but it would require significantly greater computational resources. However, even with increased budgets, oscillations in DP sequences may still occur due to uncertainty in the traffic scene.

The data collection process also faced technical challenges. The program frequently crashed after several days of operation due to excessive RAM usage, and the root cause of this issue could not be determined within the scope of the thesis. Addressing this problem could significantly improve the speed and reliability of data collection, enabling larger and higher-quality datasets to further enhance policy performance.

Finally, while the MCTS approach generally requires minimal hand-tuning, some adjustments were necessary to adapt to the constrained simulation budget, such as configuring the standard deviation for Gaussian sampling of new actions. These adjustments ensured reasonable computation times for data collection but highlighted the trade-offs between efficiency and optimality. Despite these challenges, once properly configured, the approach no longer requires hand-tuning and can enable self-supervised learning of decision postponing policies.

6.3 Outlook

Future work could explore several alternatives to address these limitations, which were not addressed in this work due to time constraints for the thesis:

- Using a neural network to perform regression instead of classification for DP prediction could potentially simplify the learning task, making it easier to predict the expert’s DP in unseen scenarios and improving generalization.
- Efforts to counter overfitting for the MLP256 model, such as L2 regularization and data augmentation, have been implemented but remain insufficient. Scaling curves in Appendix B indicate that at least two to three times more data would be required to further reduce the test loss and achieve expert-like performance in unseen scenarios. Adding additional hidden layers to the neural network did not resolve the overfitting issue, suggesting that higher-quality data is a more critical factor for improvement.
- Implementing online learning methods like DAgger (Sec. 3.4.2) could iteratively query the expert in regions of poor performance, thereby addressing distribution

mismatch inherent to Behavior Cloning. This approach would also improve data quality and enhance generalization.

- Enhancing the feature representation with graph neural networks [43] to include spatial context in the form of map features could improve generalization by replacing the current neural network architecture in the approach.
- A larger feature vector could be collected and fed into a neural network that considers the order of the partial feature vectors for each TP. DeepSets [44] would be a promising approach for this task, particularly improving generalization and performance in multi-TP scenarios, as the MAX operator would no longer be required.
- Incorporating reinforcement learning, as demonstrated in [13], could iteratively refine policies by leveraging the expert's DP sequence as a bootstrap. This method could enhance policy performance, compensating for the expert's near-optimality limitations due to computational budget constraints.
- A major computational burden lies in reward calculation for actions. Currently, a complete rollout must be simulated with BMPCC calculating the optimal input at every timestep. The rollout needs to run until the end to capture the BMPCC's reaction to the actual TP trajectories, allowing reward calculation only at the end. Developing a method to approximate this reward or simplify the simulation procedure using ground truth data for TPs in advance could significantly reduce computational demands and facilitate the collection of more high-quality data.
- Exploring hybrid approaches where neural networks guide MCTS, as seen in AlphaGo-like systems [10], could allow for continuous policy improvement during data collection, further reducing the time to collect valuable data.

7 Conclusion

In this thesis, a hybrid approach incorporating learning-based adaptive decision postponing into Branch Model Predictive Contouring Control (BMPCC) was developed to address the challenges of safe and anticipatory motion planning in traffic scenarios featuring multimodal uncertainty. By leveraging a Monte Carlo Tree Search (MCTS) expert for generating near-optimal decision postponing sequences, the method effectively balances conservativeness and increases performance.

Evaluation results demonstrate that the learned policies exhibit improved safety, comfort, and decision postponing capabilities compared to rule-based and conventional MPCC approaches. This improvement is evident in scenarios involving both a single and up to three traffic participants (TPs), with particularly notable performance in single-TP scenarios. The MCTS expert consistently outperformed baseline decision postponing methods in terms of performance and safety.

Despite these advancements, limitations in data quality and the generalization of policies highlight areas for future improvement. Enhancing and enlarging the training dataset, along with exploring more advanced model architectures, could further strengthen this approach.

A MCTS Analysis

This appendix presents the description of various Monte Carlo Tree Search (MCTS) variants, including the successor variant introduced in Sec. 4.1.2 and evaluated in Chapter 5. These MCTS variants were tested to identify the best-performing approach for near-optimal decision postponing (DP) in complex traffic scenarios. The analysis was conducted under a fixed simulation budget of 165 per traffic scenario, ensuring consistent conditions across all variants. This limit arises from the need for feasible runtimes during data collection, where each MCTS variant generates one near-optimal DP label in approximately 10 seconds of runtime.

The selected MCTS variant, `MAX_VALUE_GAUSSIAN_RES5_WITH_EXPANSION_LIMIT` (V8), was chosen based on its superior overall performance. This variant demonstrated:

- The best cost performance among tested variants,
- Fast MPC solving times, and
- Smooth decision postponing sequences and ego vehicle states, minimizing unnecessary oscillations.

Each MCTS variant was qualitatively evaluated with 10 scenarios featuring 1 TP, requiring 1–2 hours per rollout each. The qualitative evaluation is similar to the evaluation in Sec. 5.1.2, but the plots are not included in this thesis to limit its scope. Table A.1 summarizes the selection strategies, expansion methods, expansion limits, update frequencies, and simulation budgets for each MCTS variant. In the following, all MCTS variants are listed and described:

- **V0, `UCT_VANILLA`:** Vanilla MCTS as introduced in Sec. 3.3.6, using the UCT formula from Eq. 3.58. The exploration constant was chosen as $c = 2$. No expansion limits were applied.
- **V1, `UCT_VANILLA_WITH_EXPANSION_LIMIT`:** Similar to V0, but with an expansion limit to facilitate longer trees with longer DP sequences. The root was allowed to expand 15 nodes, while afterward a general expansion limit of 5 was enforced.
- **V2, `UCT_VANILLA_GREEDY`:** Identical to V0, but with greedy selection (i.e., the exploration constant was set to $c = 0$).

- **V3, UCT_VANILLA_GREEDY_WITH_EXPANSION_LIMIT:** Identical to V1, but also using greedy selection, with the exploration constant set to $c = 0$.
- **V4, MAX_VALUE_WITH_EXPANSION_LIMIT:** The selection strategy chooses the child with the highest maximum reward to build deeper trees. Uniform sampling of new actions during expansion was used. After 23 expansions, simulations were limited, and updates were allowed every 1 second (e.g., at timesteps 23, 30, 40, 50, 60, 70, 80, 90, and 100). Between updates, the decision postponing values were counted down, and when extra updates occurred, 5 more actions could be explored.
- **V5, UCT_GAUSSIAN_WITH_EXPANSION_LIMIT:** Similar to V1, but with Gaussian sampling in the expansion step instead of uniform sampling. At the root, 15 nodes were sampled uniformly, followed by 5 nodes sampled using a Gaussian distribution with a mean equal to the parent's DP reduced by one and a standard deviation of 14. This standard deviation ensures that the entire range $DP = \{1, 2, \dots, 30\}$ is broadly covered.
- **V6, MAX_VALUE_ARCADE_WITH_EXPANSION_LIMIT:** This variant selects the child with the maximum reward only. The action space is reduced to $DP = \{1, 15, 30\}$, corresponding to low, medium, and high DP values. The idea is to mimic human-like decision postponing with a high update frequency, but with a smaller action space resolution.
- **V7, MAX_VALUE_GAUSSIAN_WITH_EXPANSION_LIMIT:** Similar to V4, but with Gaussian sampling of new DP values in the expansion step. A standard deviation of 14 was used, as in V5. Extra updates were performed at timesteps 23, 30, 40, 50, 60, 70, 80, 90, and 100.
- **V8, MAX_VALUE_GAUSSIAN_RES5_WITH_EXPANSION_LIMIT:** Similar to V7, but with a higher rate of extra updates to facilitate exploration during the later stages of the tree-building process. Updates were scheduled at timesteps 23, 30, 35, 40, 45, 50, 55, 60, and 65, providing opportunities for exploration every 0.5 seconds.

The qualitative analysis revealed that:

- UCT-based variants (e.g., V0, V1, V2, V3, and V5) excel in scenarios with high uncertainty at the beginning (approximately the first 3 seconds) but struggle to build deep trees for scenarios with uncertainty persisting beyond 3 seconds into the intersection scenario. This limitation is due to the constrained simulation budget, which impedes robustness in complex scenarios.
- MAX-based variants (e.g., V4, V6, V7, and V8) perform better at generating long DP sequences, achieving higher success rates and greater robustness overall.

- Gaussian expansions (e.g., V5, V7, and V8) introduce smoothing to DP sequences, reducing oscillations in ego vehicle states. This is desirable in many scenarios and also beneficial for the optimization problem of the BMPCC, as it leads to similar initial guesses.

Despite the potential for MAX-based variants to fall into local minima, they outperform UCT-based variants within the constrained simulation budget. The chosen variant, `MAX_VALUE_GAUSSIAN_RES5_WITH_EXPANSION_LIMIT`, combines Gaussian expansions and higher update frequencies, achieving the best trade-off between performance and exploration.

Table A.1 provides an overview of all tested MCTS variants. Examples of decision-postponing trees for all variants, generated during traffic scenario 10 of the MCTS analysis, are shown in the figures on the following pages (Fig. A.1 – A.8). These figures illustrate how the tree structure differs between the evaluated variants. Each figure is displayed in PDF format, allowing the DP value of each node to be seen when zooming into the figure using a suitable PDF viewer.

Table A.1: Overview of MCTS variants compared in the qualitative MCTS analysis, detailing the selection strategies, expansion methods, root and general expansion limits, extra update frequencies, and simulation budgets.

ID	Variant	Selection	Expansion	Exp. Limit 1	Exp. Limit 2	Updates	Budget
0	uct_vanilla	UCT	Uniform	30	30	No	165
1	uct_vanilla_with_expansion_limit	UCT	Uniform	15	5	No	165
2	uct_vanilla_greedy	UCT	Uniform	30	30	No	165
3	uct_vanilla_greedy_with_expansion_limit	UCT	Uniform	15	5	No	165
4	max_value_with_expansion_limit	MAX	Uniform	15	5	Yes	165
5	uct_gaussian_with_expansion_limit	UCT	Uniform + Gaussian	15	5	No	165
6	max_value_arcade_with_expansion_limit	MAX	Custom	15	5	No	165
7	max_value_gaussian_with_expansion_limit	MAX	Uniform + Gaussian	15	5	Yes	165
8	max_value_gaussian_res5_with_expansion_limit	MAX	Uniform + Gaussian	15	5	Yes	165

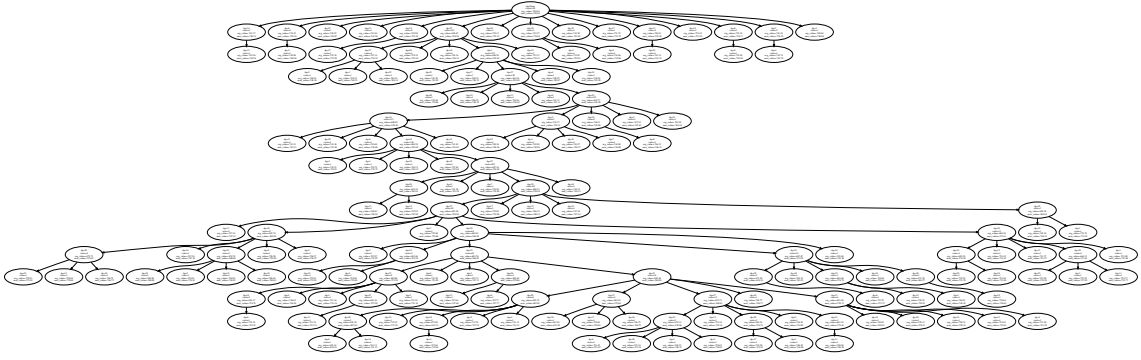


Figure A.1: Tree constructed by the variant UCT_VANILLA_WITH_EXPANSION_LIMIT (V1). The tree is relatively wide, showing its inability to build long decision-postponing sequences under the given simulation budget of 165 with a DP sequence length of only 15.

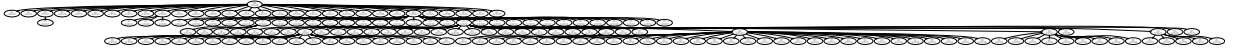


Figure A.2: Tree constructed by the variant UCT_VANILLA_GREEDY (V2). The tree is extremely wide, having a depth of only 4. Although greedy selection is already employed in this UCT variant, the method fails because the rewards for different DP values in one generation are nearly the same due to a certain traffic situation and therefore resulting in similar or even identical BMPCC plans when using a high expansion limit of 30.

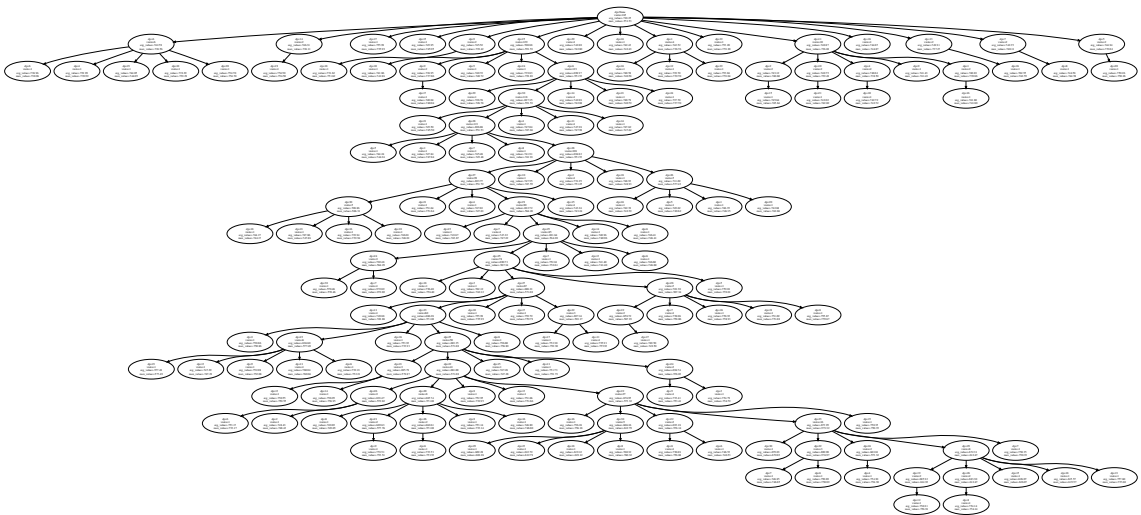


Figure A.3: Tree constructed by the variant UCT_VANILLA_GREEDY_WITH_EXPANSION_LIMIT (V3). Similar to the tree built by V1 but having a slightly longer DP sequence due to the greedy selection of nodes.

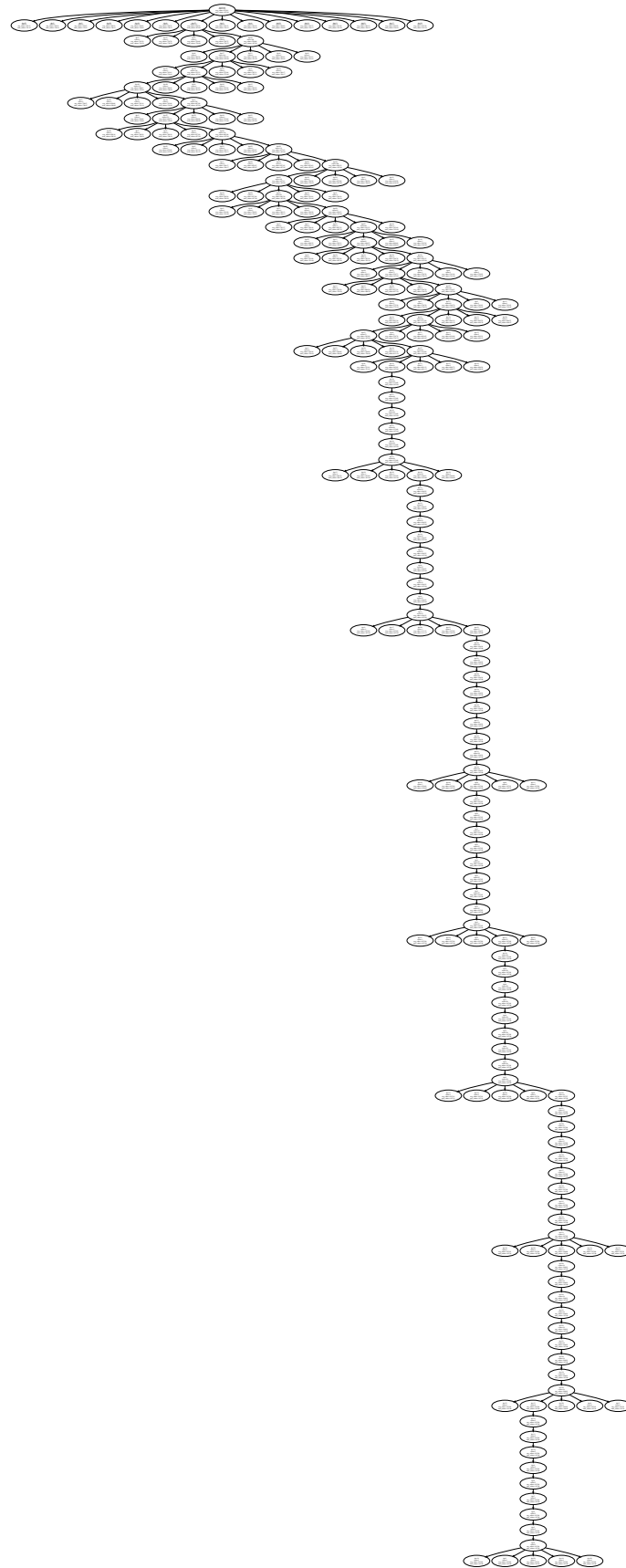


Figure A.4: Tree constructed by the variant `MAX_VALUE_WITH_EXPANSION_LIMIT` (V4). MAX-Selection leads to significantly longer decision-postponing trees. The tree updates every 10 timesteps (1 second) starting at a length of 23, enabling the construction of deeper decision-postponing sequences with a length of 100 timesteps.

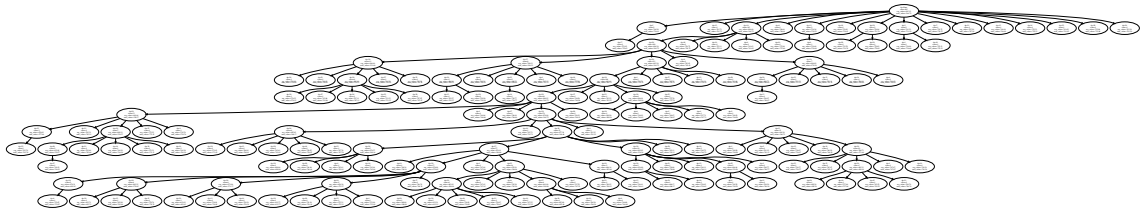


Figure A.5: Tree constructed by the variant UCT_GAUSSIAN_WITH_EXPANSION_LIMIT (V5). Tree structure is similar to variant V1 with exploration constant $c = 2$. Gaussian sampling introduces a bias in the construction of the decision-postponing sequence.

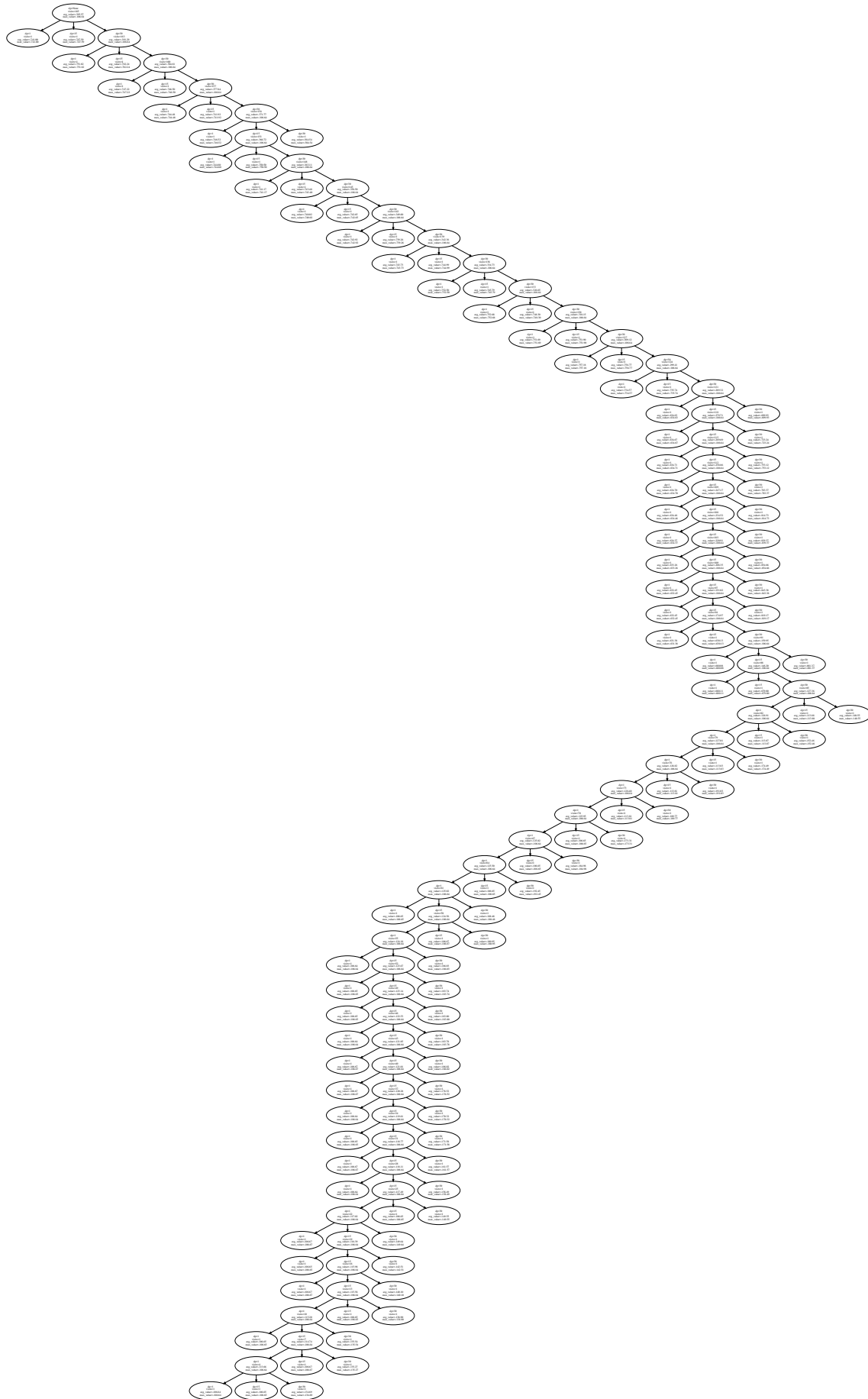


Figure A.6: Tree constructed by the variant MAX_VALUE_ARCADE_WITH_EXPANSION_LIMIT (V6). The action space consists of $DP = \{1, 15, 30\}$, hence making it possible to build longer trees while representing human-like decision postponing with a high update frequency due to a smaller action space resolution.

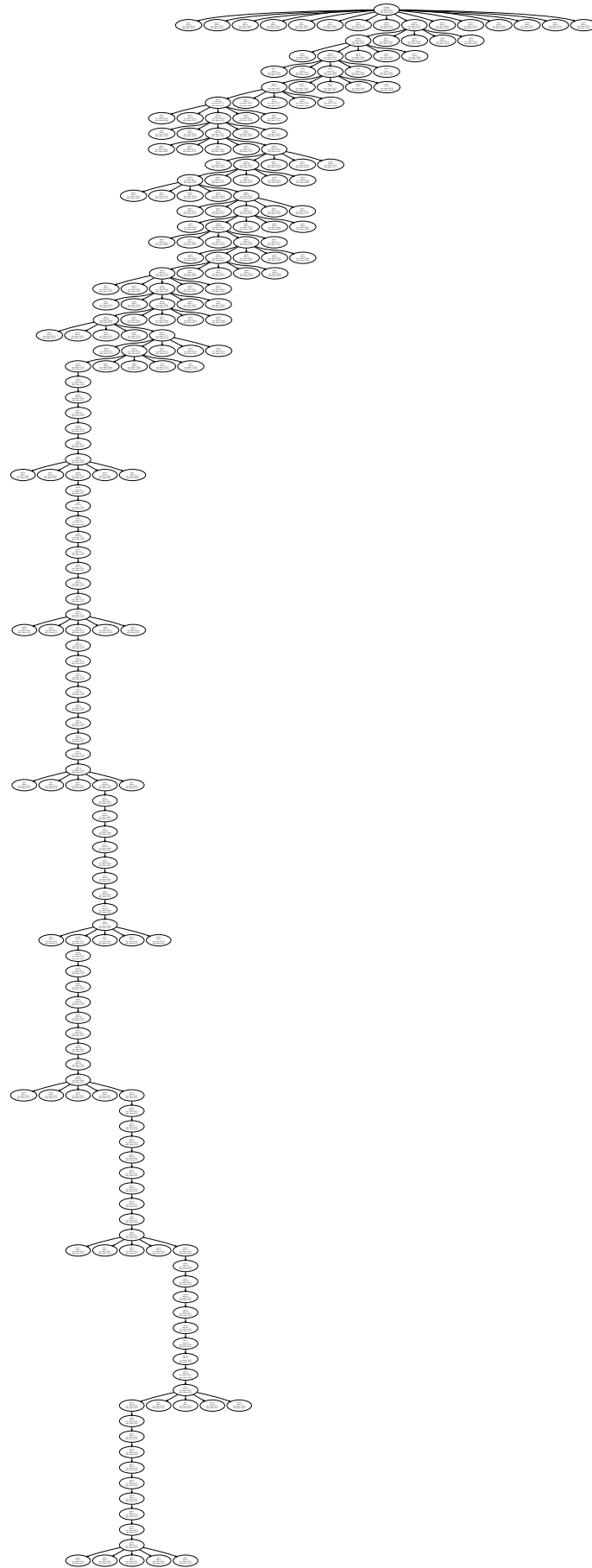


Figure A.7: Tree constructed by the variant MAX-VALUE-GAUSSIAN-WITH-EXPANSION-LIMIT (V7). It is identical to V5 but introduces Gaussian sampling in the expansion step.

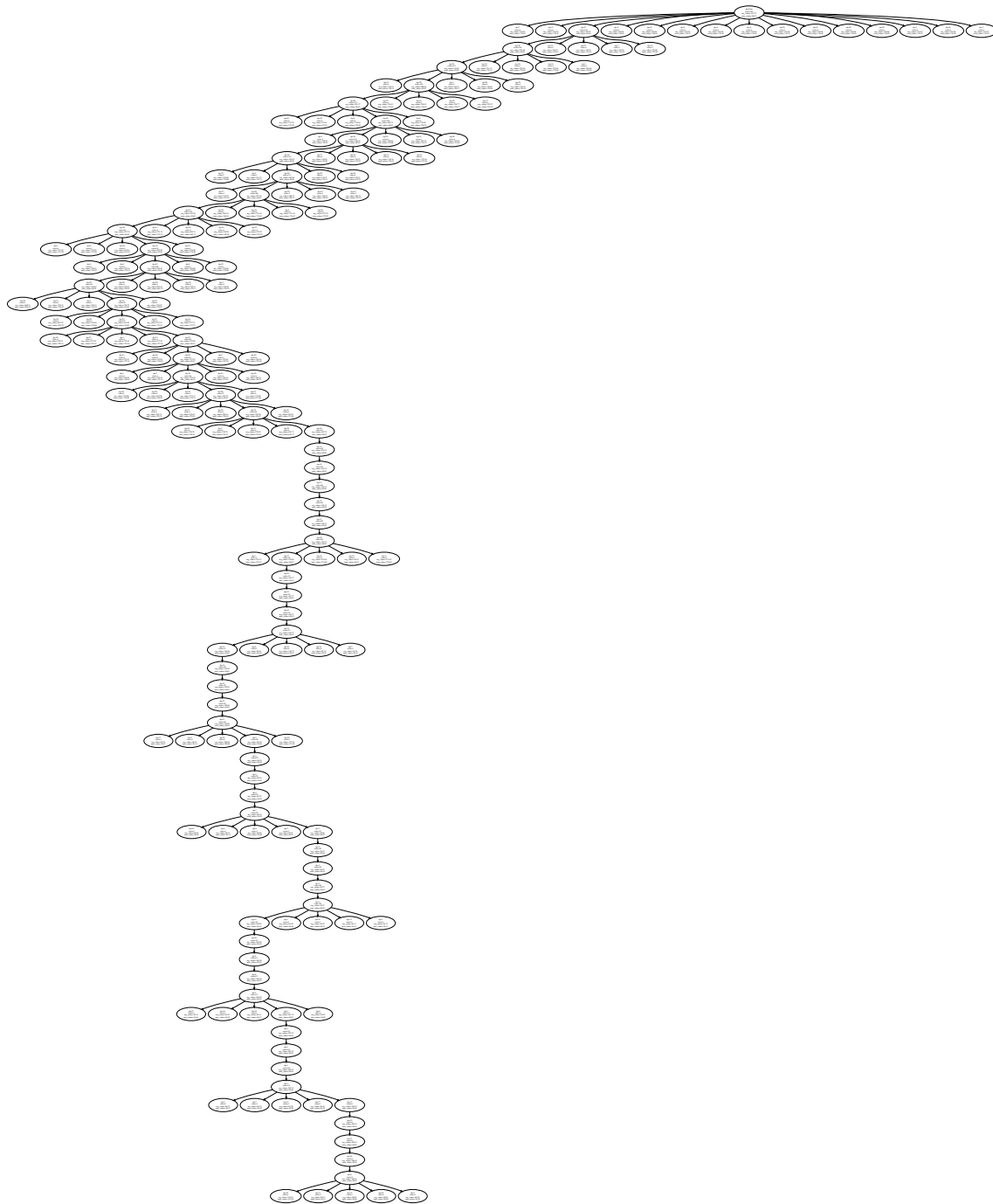


Figure A.8: Tree constructed by the variant MAX_VALUE_GAUSSIAN_RES5_WITH_EXPANSION_LIMIT (V8). It is identical to V7 but features more extra updates at timesteps 23, 30, 35, 40, 45, 50, 55, 60, and 65, providing opportunities for exploration every 0.5 seconds, leading to an overall length of 65 which often is enough to cover the uncertainty over the whole period of the intersection scenario.

B Policy Training

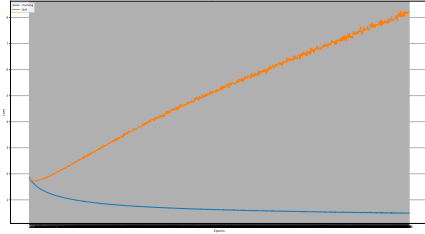
This appendix presents the results of the policy training for the different policy variants introduced in the evaluation in Chapter 5. Additionally, it includes scaling curves for the policy variant MLP256_2, highlighting the influence of dataset sizes on training and test performance. 4.1.2

B.1 Policy Variants

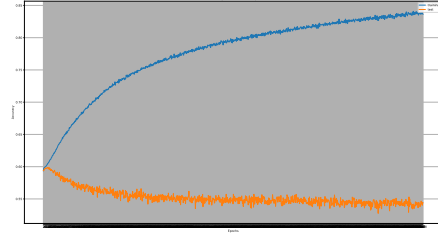
For each policy variant, the training and test loss, training and test accuracy, and the training and test confusion matrix are presented. These results are detailed in the following subsections:

- MLP256_1: Derived from MLP256_3, excluding class weighting, L2 regularization, and data augmentation. Results are shown in Section B.1.1.
- MLP256_2: Similar to MLP256_3, but without data augmentation. Results are shown in Section B.1.2.
- MLP256_3: As introduced in Section 4.2.3, incorporates class weighting, L2 regularization, and data augmentation.
- MLP256_4: Based on MLP256_3, but excludes L2 regularization and data augmentation. Results are shown in Section B.1.3.
- MLP32_1: A smaller model introduced in Section 4.2.3, includes class weighting and L2 regularization.
- MLP32_2: A variation of MLP32_1, without L2 regularization. Results are shown in Section B.1.4.
- MLP32_3: Derived from MLP32_1, excluding class weighting and L2 regularization. Results are shown in Section B.1.5.

B.1.1 MLP256_1



(a) Training and test loss for MLP256_1.



(b) Training and test accuracy for MLP256_1.

Figure B.1: Training and test results for MLP256_1, a variant of MLP256_3 introduced in Sec. 4.2.3. This model excludes class weighting, L2 regularization, and data augmentation. The model was trained for 1000 epochs.

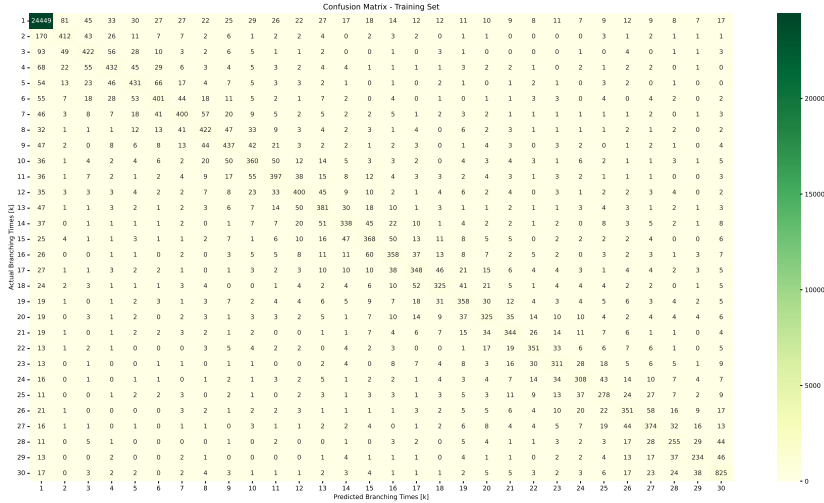


Figure B.2: Training confusion matrix for MLP256_1, a variant of MLP256_3 introduced in Sec. 4.2.3. This model excludes class weighting, L2 regularization, and data augmentation. The model was trained for 1000 epochs. The matrix displays occurrences on a logarithmic color scale, where the predicted DP values (x-axis) align with the actual DP values from the MCTS expert (y-axis) in the dataset.

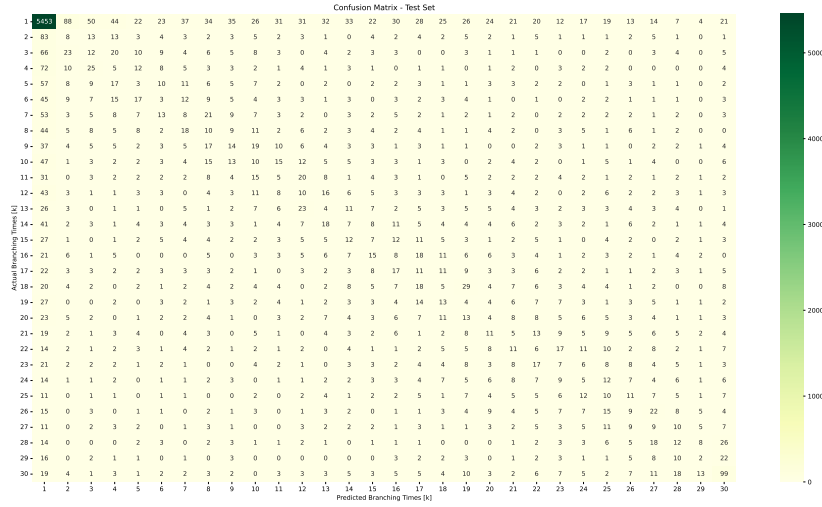
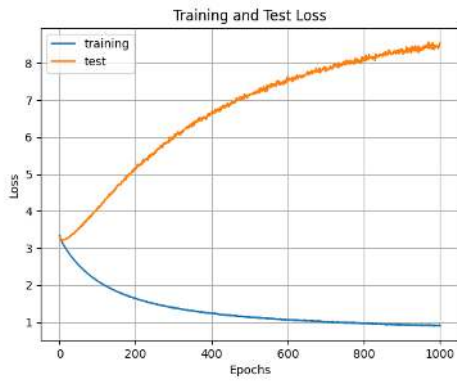


Figure B.3: Test confusion matrix for MLP256_1, a variant of MLP256_3 introduced in Sec. 4.2.3. This model excludes class weighting, L2 regularization, and data augmentation. The model was trained for 1000 epochs. The matrix displays occurrences on a logarithmic color scale, where the predicted DP values (x-axis) align with the actual DP values from the MCTS expert (y-axis) in the dataset.

B.1.2 MLP256_2



(a) Training and test loss for MLP256_2.



(b) Training and test accuracy for MLP256_2.

Figure B.4: Training and test results for MLP256_2, a variant of MLP256_3 introduced in Sec. 4.2.3. This model excludes data augmentation but retains class weighting and L2 regularization.

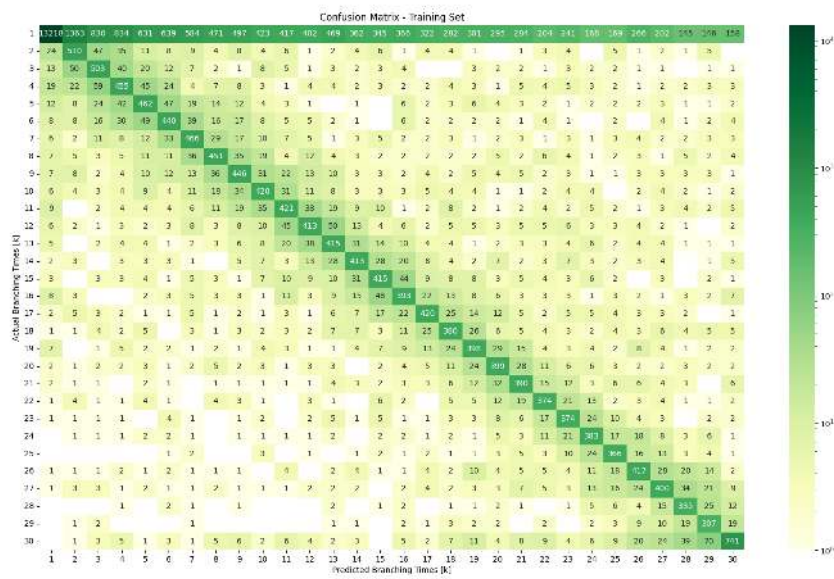


Figure B.5: Training confusion matrix for MLP256_2, a variant of MLP256_3 introduced in Sec. 4.2.3. This model excludes data augmentation but retains class weighting and L2 regularization. The matrix displays occurrences on a logarithmic color scale, where the predicted DP values (x-axis) align with the actual DP values from the MCTS expert (y-axis) in the dataset.

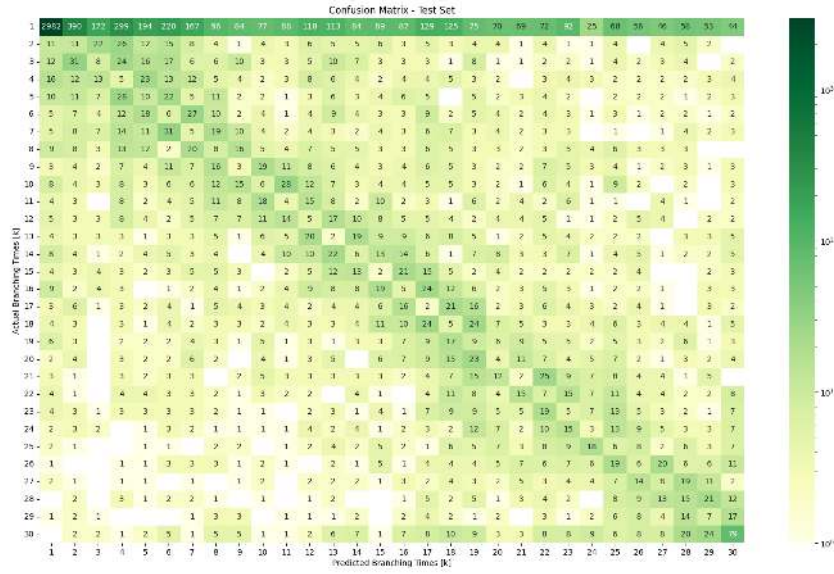


Figure B.6: Test confusion matrix for MLP256_2, a variant of MLP256_3 introduced in Sec. 4.2.3. This model excludes data augmentation but retains class weighting and L2 regularization. The matrix displays occurrences on a logarithmic color scale, where the predicted DP values (x-axis) align with the actual DP values from the MCTS expert (y-axis) in the dataset.

B.1.3 MLP256_4



(a) Training and test loss for MLP256_4.



(b) Training and test accuracy for MLP256_4.

Figure B.7: Training and test results for MLP256_4, a variant of MLP256_3 introduced in Sec. 4.2.3. This model excludes L2 regularization and data augmentation.

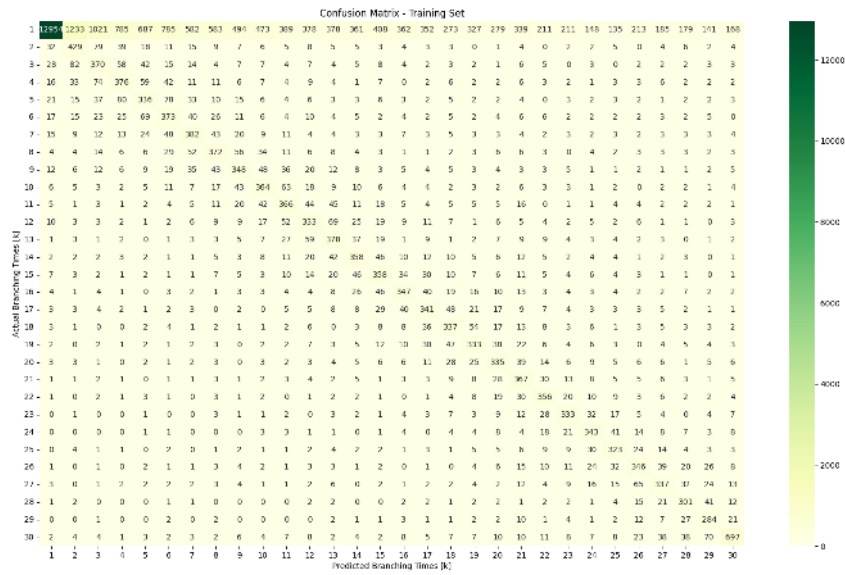


Figure B.8: Training confusion matrix for MLP256_4, a variant of MLP256_3 introduced in Sec. 4.2.3. This model excludes L2 regularization and data augmentation. The matrix displays occurrences on a logarithmic color scale, where the predicted DP values (x-axis) align with the actual DP values from the MCTS expert (y-axis) in the dataset.

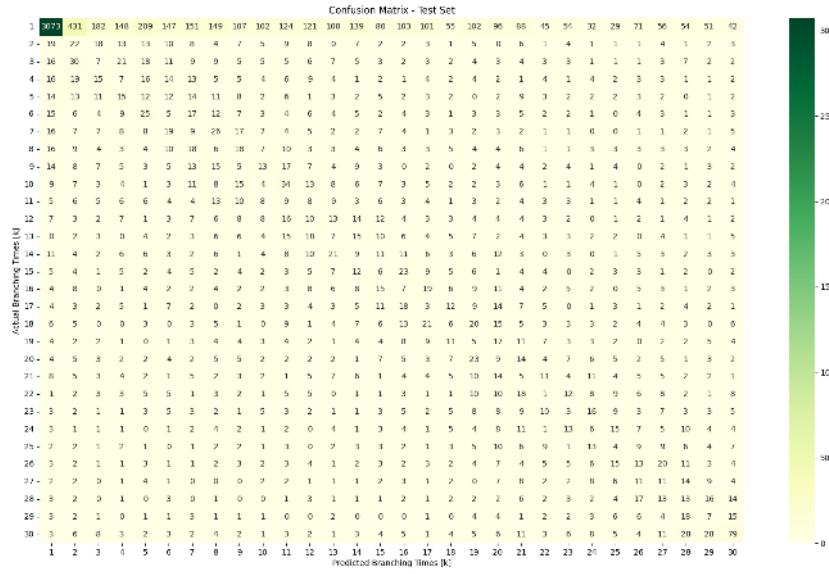
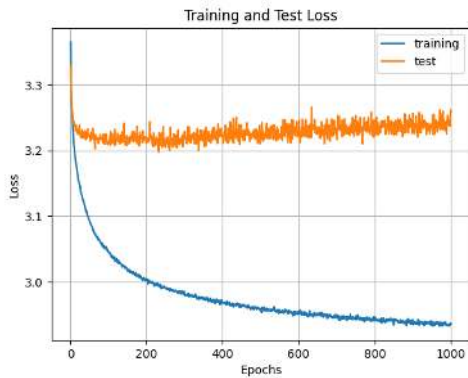
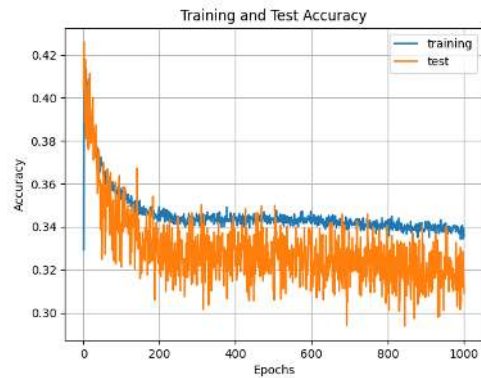


Figure B.9: Test confusion matrix for MLP256_4, a variant of MLP256_3 introduced in Sec. 4.2.3. This model excludes L2 regularization and data augmentation. The matrix displays occurrences on a logarithmic color scale, where the predicted DP values (x-axis) align with the actual DP values from the MCTS expert (y-axis) in the dataset.

B.1.4 MLP32_2



(a) Training and test loss for MLP32_2.



(b) Training and test accuracy for MLP32_2.

Figure B.10: Training and test results for MLP32_2, a variant of MLP32_1 introduced in Sec. 4.2.3. This model excludes L2 regularization but retains class weighting.

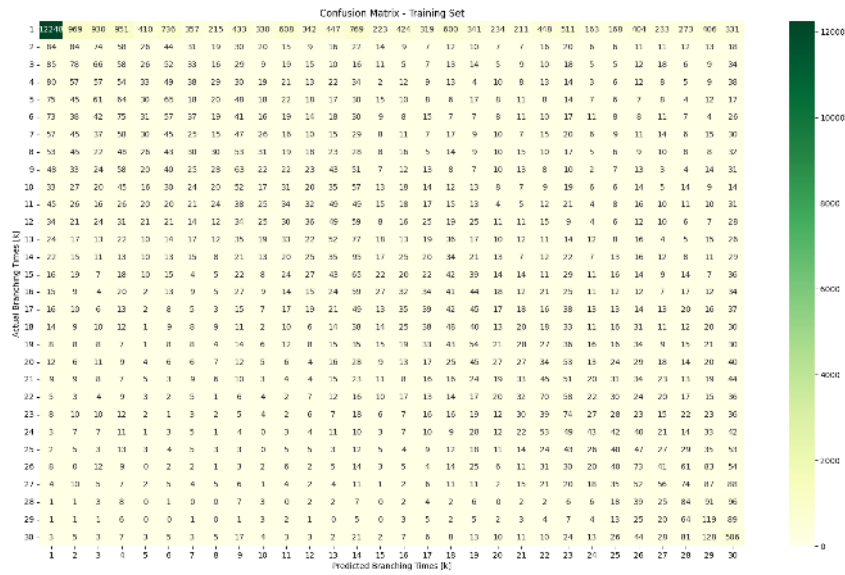


Figure B.11: Training confusion matrix for MLP32.2, a variant of MLP32.1 introduced in Sec. 4.2.3. This model excludes L2 regularization but retains class weighting. The matrix displays occurrences on a logarithmic color scale, where the predicted DP values (x-axis) align with the actual DP values from the MCTS expert (y-axis) in the dataset.

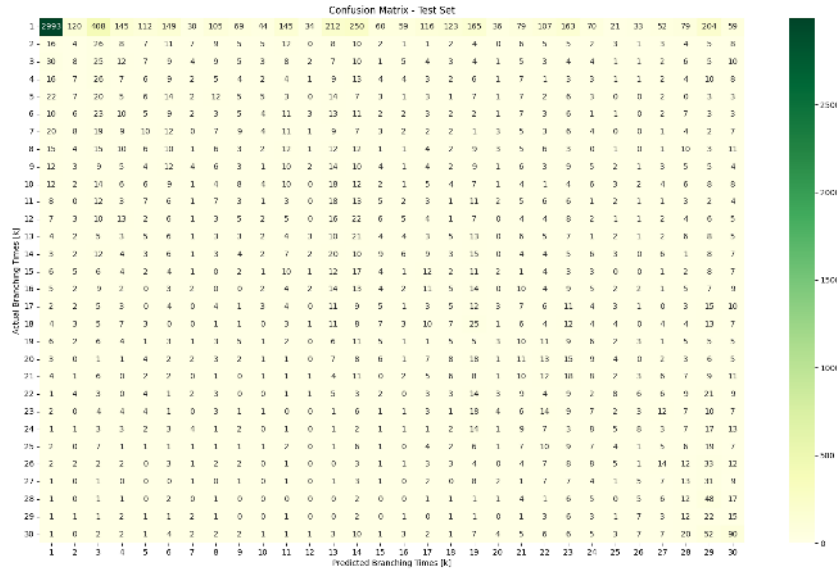
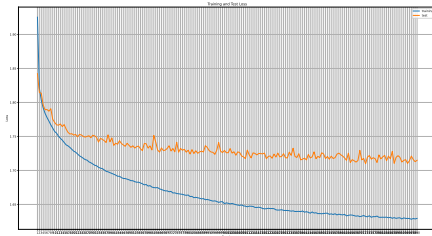
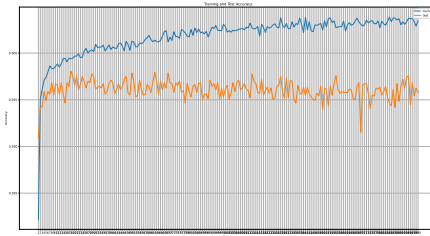


Figure B.12: Test confusion matrix for MLP32.2, a variant of MLP32.1 introduced in Sec. 4.2.3. This model excludes L2 regularization but retains class weighting. The matrix displays occurrences on a logarithmic color scale, where the predicted DP values (x-axis) align with the actual DP values from the MCTS expert (y-axis) in the dataset.

B.1.5 MLP32.3



(a) Training and test loss for MLP32.3.



(b) Training and test accuracy for MLP32.3.

Figure B.13: Training and test results for MLP32.3, a variant of MLP32.1 introduced in Sec. 4.2.3. This model excludes class weighting and L2 regularization. The model was trained for 200 epochs.

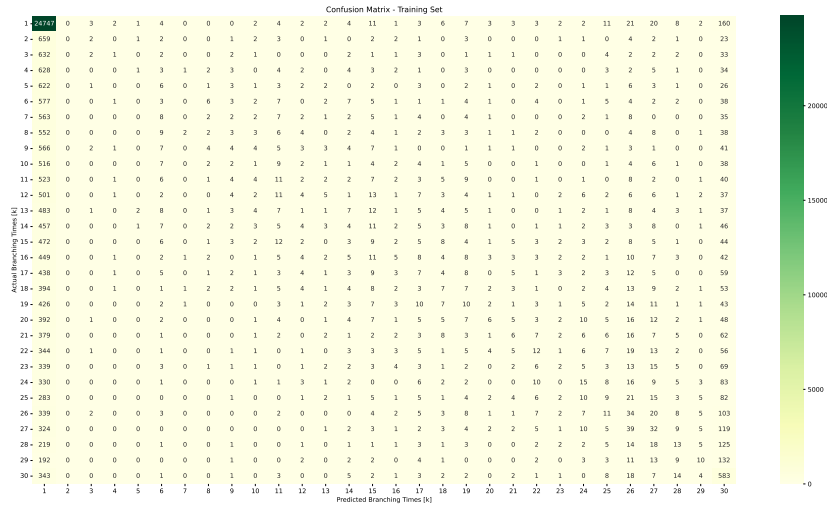


Figure B.14: Training confusion matrix for MLP32.3, a variant of MLP32.1 introduced in Sec. 4.2.3. This model excludes class weighting and L2 regularization. The model was trained for 200 epochs. The matrix displays occurrences on a logarithmic color scale, where the predicted DP values (x-axis) align with the actual DP values from the MCTS expert (y-axis) in the dataset.

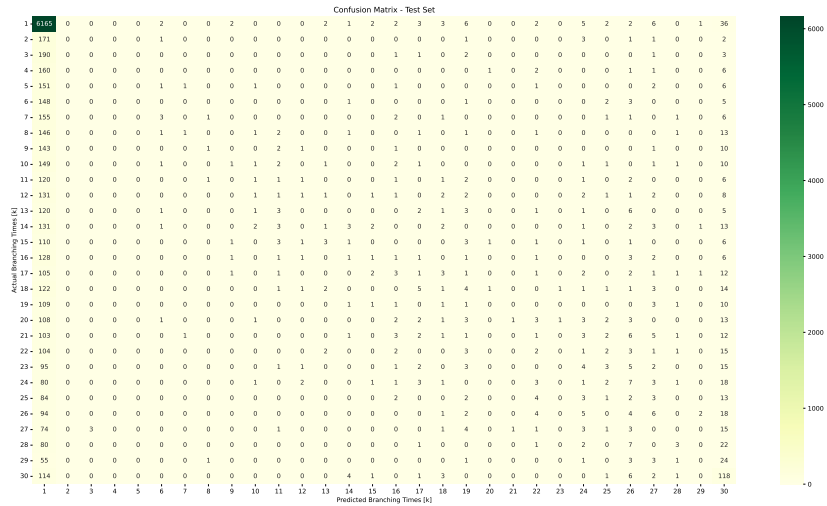


Figure B.15: Test confusion matrix for MLP32.3, a variant of MLP32.1 introduced in Sec. 4.2.3. This model excludes class weighting and L2 regularization. The model was trained for 200 epochs. The matrix displays occurrences on a logarithmic color scale, where the predicted DP values (x-axis) align with the actual DP values from the MCTS expert (y-axis) in the dataset.

B.2 Scaling Curves

The following section provides scaling curves comparing training and test losses, as well as accuracies, for the policy variant MLP256_2. Additionally, confusion matrices for training and testing across different dataset sizes are included.

The policy variant MLP256_2, originally named MLP256_1000_LR00005_L2REG00001, incorporates class weighting and L2 regularization. For instance, the variant MLP256_1000_LR00005_L2REG00001_SC02 was trained using 20% of the feature vectors and labels from the original dataset introduced in Section 4.2.2, amounting to a total of 52,903 data points. In total, the policy variant MLP256_1000_LR00005_L2REG00001 was trained using four different dataset sizes: 20%, 40%, 60%, and 80%.

B.2.1 Training and Test Loss

Figure B.16 illustrates the training and test losses for the policy variant MLP256_2 when trained with different dataset sizes (20%, 40%, 60%, and 80%). The model trained with 20% of the data exhibits a test loss of approximately 15. As the dataset size increases, the test loss decreases significantly, reaching slightly above 8 when using the full dataset, effectively halving the generalization error. This improvement highlights the benefit of enlarging the training dataset, which also reduces overfitting. The training loss, on the other hand, shows an increase from nearly 0 to approximately 1 as the dataset size grows.

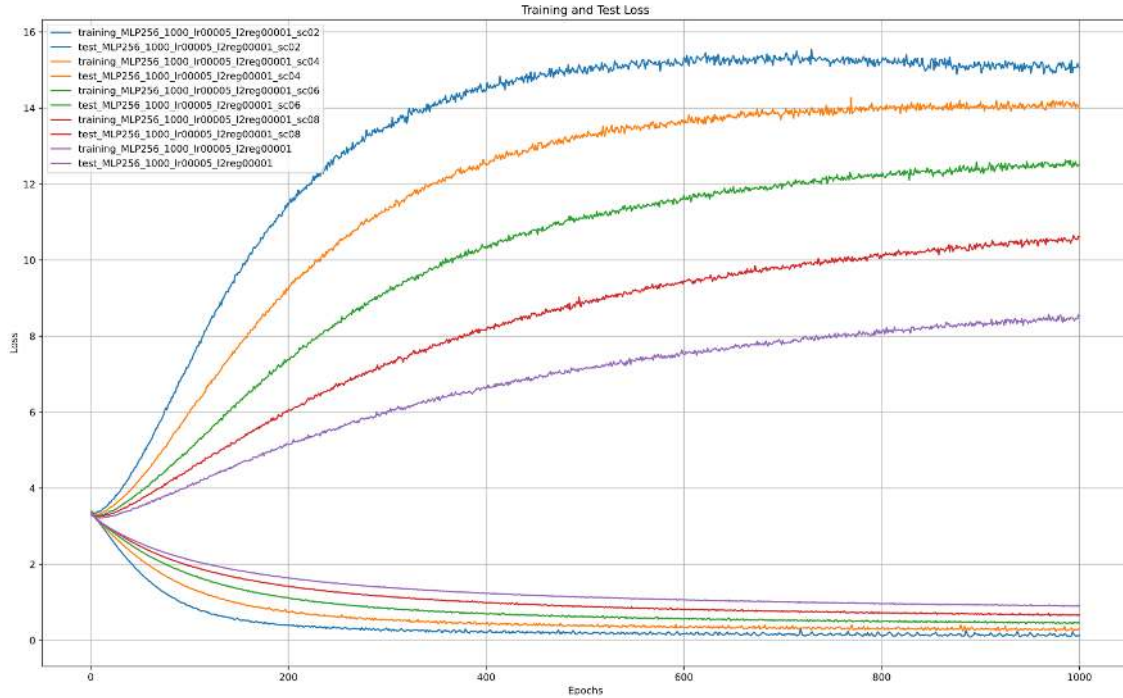


Figure B.16: Comparison of training and test losses for the policy variant MLP256.2. The original name of the policy variant MLP256.2 is MLP256_1000_LR00005_L2REG00001, which includes class weighting and L2 regularization. For example, policy MLP256_1000_LR00005_L2REG00001_sc02 has been trained with 20% of the feature vectors and labels from the original dataset introduced in Section 4.2.2.

B.2.2 Training and Test Accuracy

Figure B.17 illustrates the training and test accuracies for the policy variant MLP256.2 trained with different dataset sizes (20%, 40%, 60%, and 80%). The variant trained with 20% of the data achieves a maximum training accuracy of approximately 0.85. However, as the dataset size increases, the training accuracy decreases significantly, dropping to about 0.6 with the full dataset. The test accuracy exhibits a similar trend, showing a reduction as the dataset size grows. For the smallest dataset size (20%), the test accuracy exceeds 40%. In contrast, with the full dataset, the test accuracy drops to slightly above 30%, indicating poor generalization and overall accuracy degradation as more data is used.

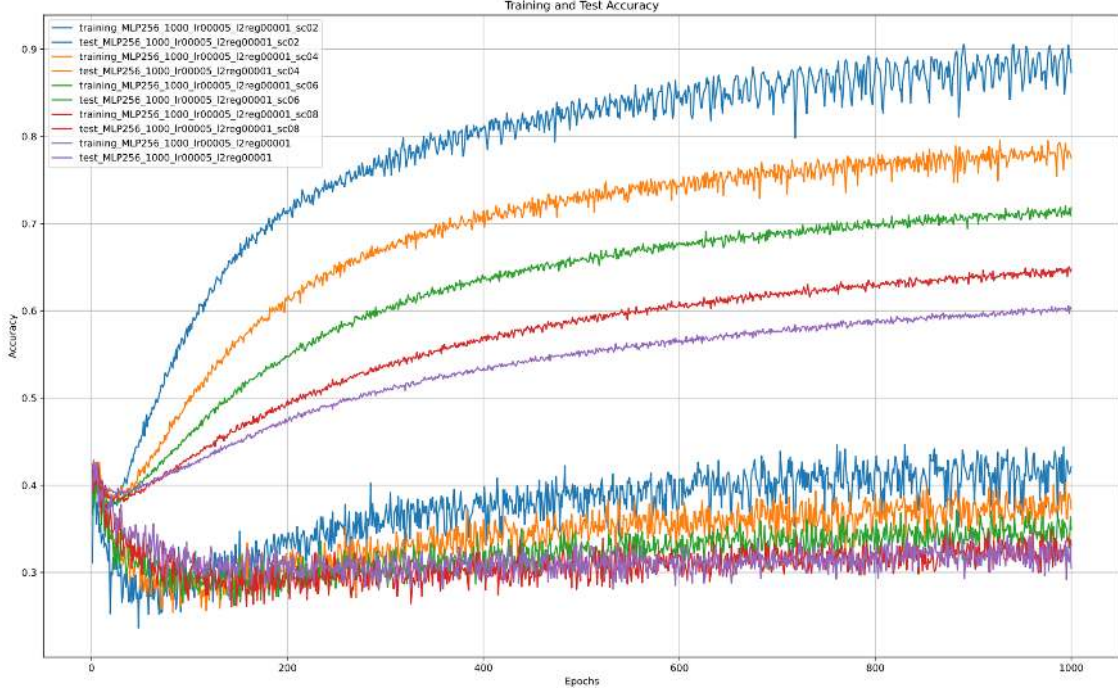


Figure B.17: Comparison of training and test accuracy for the policy variant MLP256_2. The original name of the policy variant MLP256_2 is MLP256_1000_LR00005_L2REG00001, which includes class weighting and L2 regularization. For example, policy MLP256_1000_LR00005_L2REG00001_sc02 has been trained with 20% of the feature vectors and labels from the original dataset introduced in Section 4.2.2.

B.2.3 Training Confusion Matrix

The following figures present the training confusion matrices for the policy variant MLP256_2 trained with different dataset sizes. Each matrix visualizes the occurrences, displayed on a logarithmic color scale, where the predicted DP values (x-axis) align with the actual DP values from the MCTS expert (y-axis). The figures are as follows:

- Figure B.18: Trained with 20% of the full dataset.
- Figure B.19: Trained with 40% of the full dataset.
- Figure B.20: Trained with 60% of the full dataset.
- Figure B.21: Trained with 80% of the full dataset.

As the dataset size increases, the occurrences in the training confusion matrix also grow, resulting in a progressively clearer diagonal. This indicates that the policy is increasingly able to align its predictions with those of the expert. Additionally, clustering around the diagonal becomes more apparent as the dataset size grows, suggesting that the predicted

DP values approximate the ground truth DP values over time rather than perfectly match them. From the beginning, the policy demonstrates an understanding that in scenarios with no multimodal uncertainty, such as when the expert predicts a DP value of 1, the DP value provided to the BMPCC is irrelevant. In such cases, all DP values are valid because the plans generated by the BMPCC for these scenarios are equivalent. A similar relationship exists for DP=30, which in certain scenarios is equivalent to DP=1. This understanding becomes more robust as the dataset size increases.

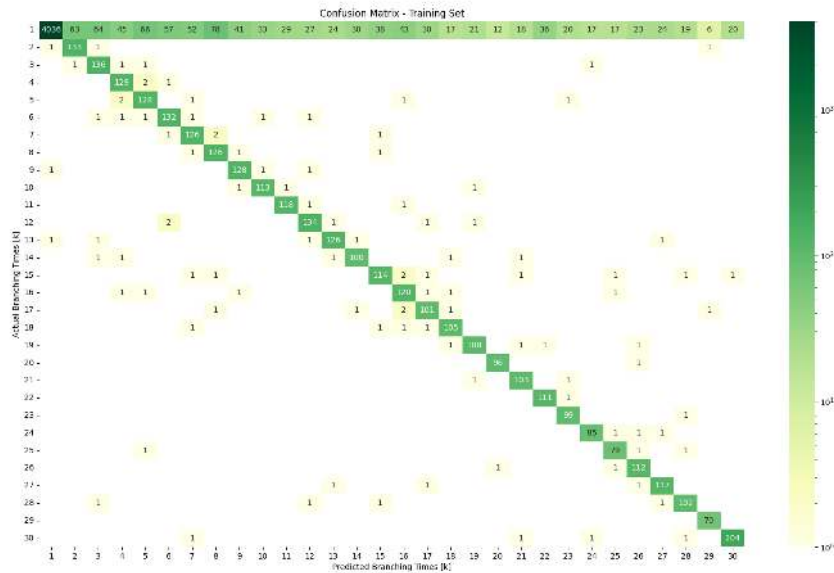


Figure B.18: Training confusion matrix for the policy variant MLP256_2, trained with a dataset size corresponding to 20% of the full data. The matrix shows the occurrences, displayed on a logarithmic color scale, where the predicted DP values (x-axis) coincide with the actual DP values from the MCTS expert (y-axis) in the dataset.

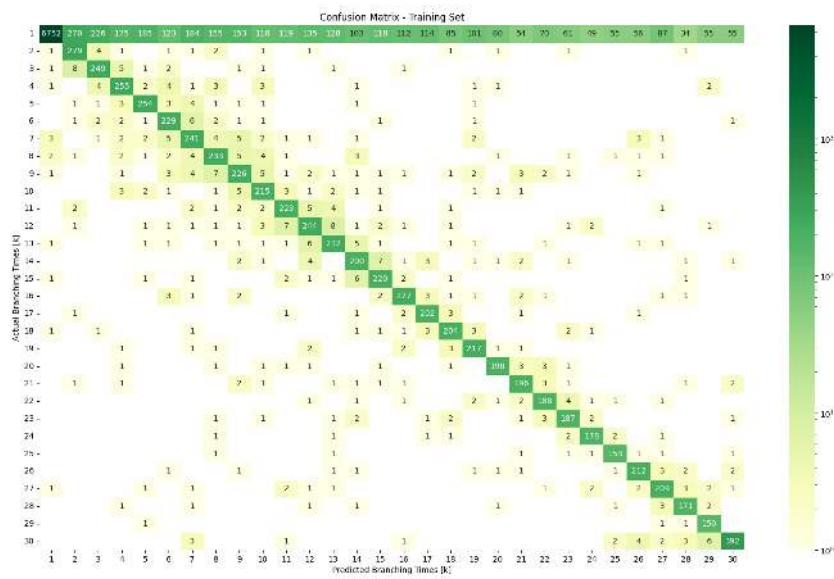


Figure B.19: Training confusion matrix for the policy variant MLP256_2, trained with a dataset size corresponding to 40% of the full data. The matrix shows the occurrences, displayed on a logarithmic color scale, where the predicted DP values (x-axis) coincide with the actual DP values from the MCTS expert (y-axis) in the dataset.

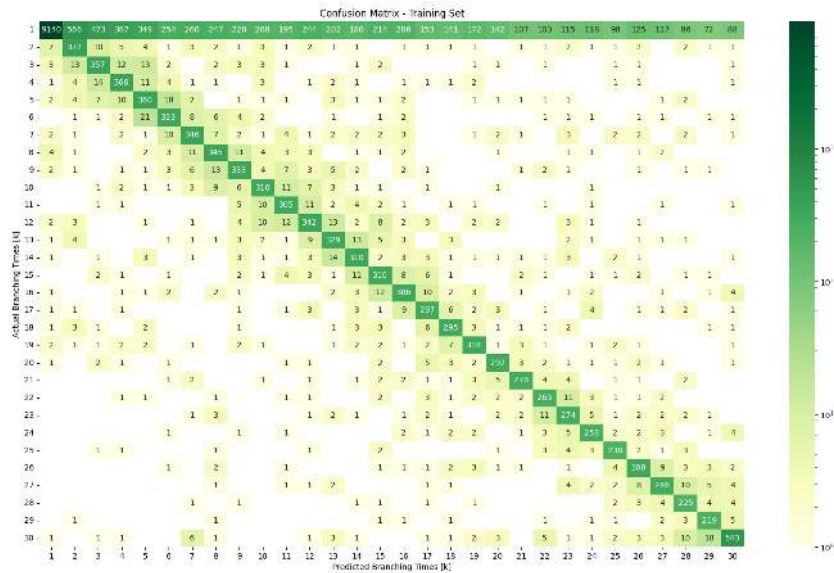


Figure B.20: Training confusion matrix for the policy variant MLP256_2, trained with a dataset size corresponding to 60% of the full data. The matrix shows the occurrences, displayed on a logarithmic color scale, where the predicted DP values (x-axis) coincide with the actual DP values from the MCTS expert (y-axis) in the dataset.

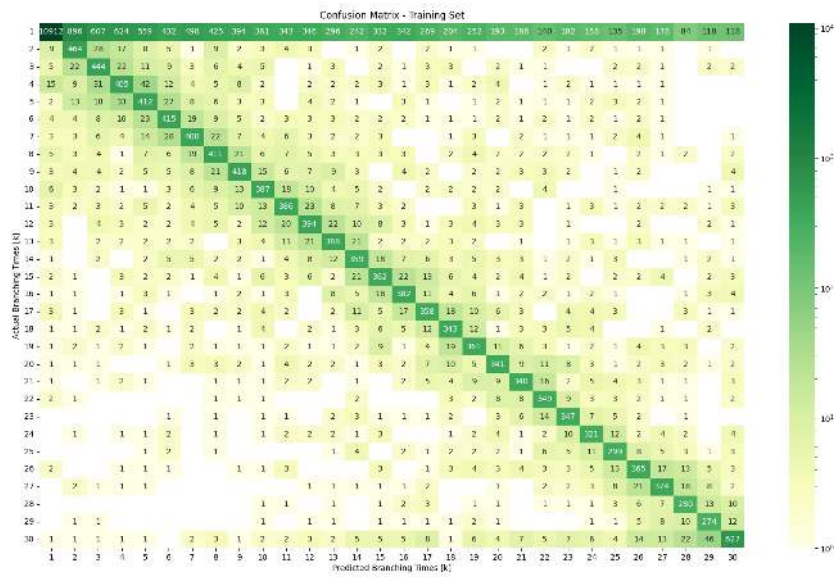


Figure B.21: Training confusion matrix for the policy variant MLP256_2, trained with a dataset size corresponding to 80% of the full data. The matrix shows the occurrences, displayed on a logarithmic color scale, where the predicted DP values (x-axis) coincide with the actual DP values from the MCTS expert (y-axis) in the dataset.

B.2.4 Test Confusion Matrix

The following figures present the test confusion matrices for the policy variant MLP256.2 trained with different dataset sizes. Each matrix visualizes the occurrences, displayed on a logarithmic color scale, where the predicted DP values (x-axis) align with the actual DP values from the MCTS expert (y-axis) in unseen scenarios. The figures are as follows:

- Figure B.22: Trained with 20% of the full dataset.
- Figure B.23: Trained with 40% of the full dataset.
- Figure B.24: Trained with 60% of the full dataset.
- Figure B.25: Trained with 80% of the full dataset.

For the policy trained with 20% of the dataset, there is no clearly visible diagonal in the test confusion matrix. The policy primarily learns the relationship with DP=1 for traffic situations without multimodal uncertainty, but it demonstrates poor generalization. This is evident as the policy predominantly predicts DP=1, even in scenarios where the expert considers a range of decision postponing values, from 1 to 30, necessary for a

given scenario at a specific timestep. As the dataset size increases, the diagonal in the test confusion matrix becomes progressively clearer. However, this clarity arises through clustering around the diagonal rather than the policies predicting the exact DP values as the expert in unseen scenarios. Additionally, the tendency of the policy to mostly predict DP values equal to 1 diminishes with larger datasets, leading to more diverse and generalized predictions.

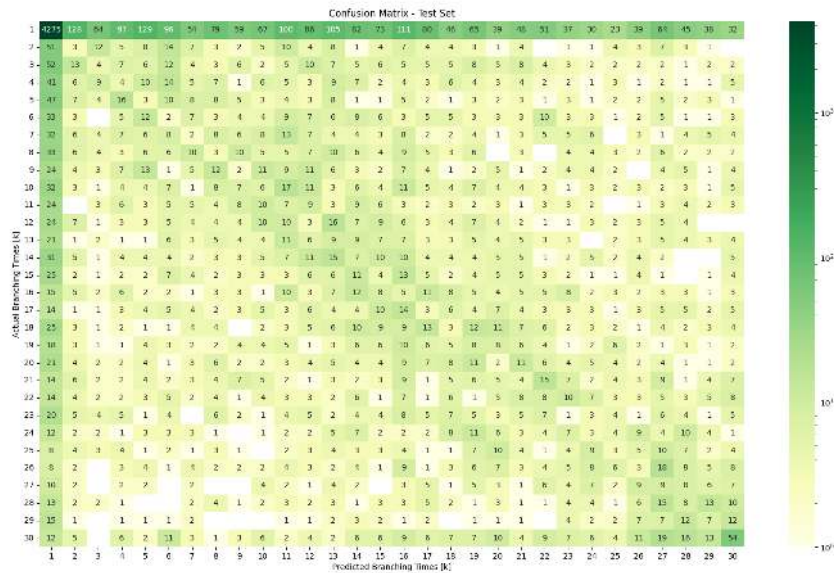


Figure B.22: Test confusion matrix for the policy variant MLP256_2, trained with a dataset size corresponding to 20% of the full data. The matrix shows the occurrences, displayed on a logarithmic color scale, where the predicted DP values (x-axis) coincide with the actual DP values from the MCTS expert (y-axis) in the dataset.

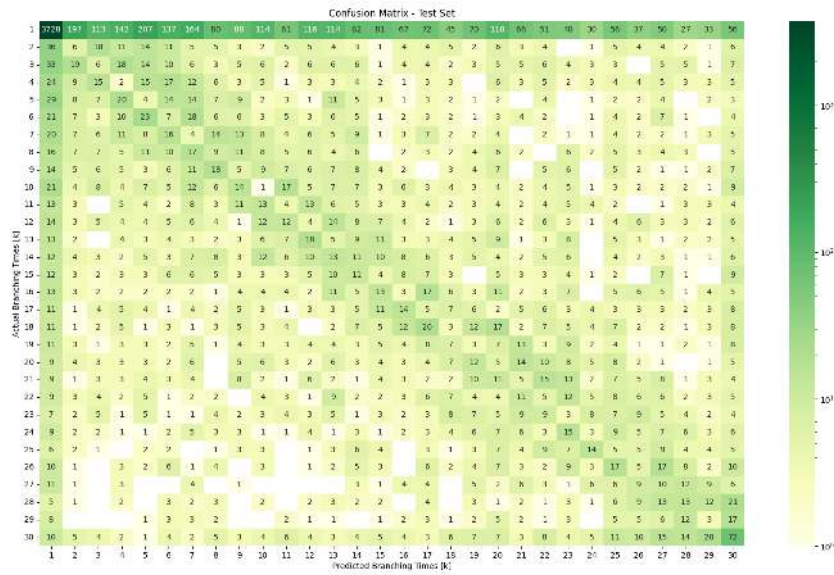


Figure B.23: Test confusion matrix for the policy variant MLP256.2, trained with a dataset size corresponding to 40% of the full data. The matrix shows the occurrences, displayed on a logarithmic color scale, where the predicted DP values (x-axis) coincide with the actual DP values from the MCTS expert (y-axis) in the dataset.

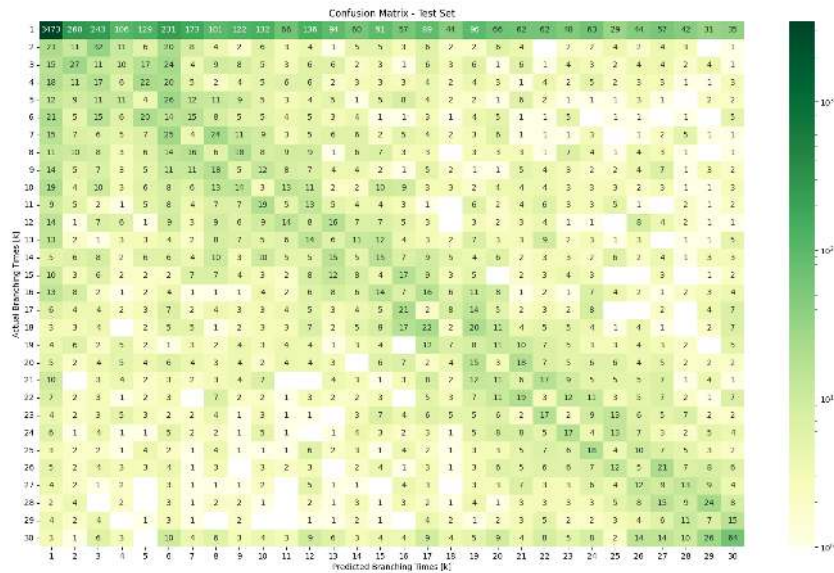


Figure B.24: Test confusion matrix for the policy variant MLP256.2, trained with a dataset size corresponding to 60% of the full data. The matrix shows the occurrences, displayed on a logarithmic color scale, where the predicted DP values (x-axis) coincide with the actual DP values from the MCTS expert (y-axis) in the dataset.

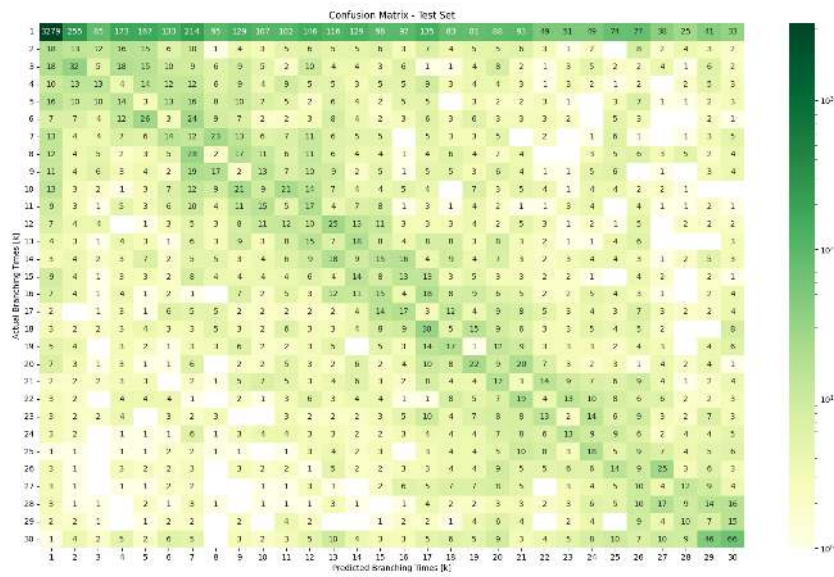


Figure B.25: Test confusion matrix for the policy variant MLP256_2, trained with a dataset size corresponding to 80% of the full data. The matrix shows the occurrences, displayed on a logarithmic color scale, where the predicted DP values (x-axis) coincide with the actual DP values from the MCTS expert (y-axis) in the dataset.

Bibliography

- [1] Santokh Singh. Critical Reasons for Crashes Investigated in the National Motor Vehicle Crash Causation Survey. *Traffic Safety Facts - Crash Stats*, February 2015. Number: DOT HS 812 115.
- [2] Ekim Yurtsever, Jacob Lambert, Alexander Carballo, and Kazuya Takeda. A Survey of Autonomous Driving: Common Practices and Emerging Technologies. *IEEE Access*, 8:58443–58469, 2020. Conference Name: IEEE Access.
- [3] Max Schwenzer, Muzaffer Ay, Thomas Bergs, and Dirk Abel. Review on model predictive control: an engineering perspective. *The International Journal of Advanced Manufacturing Technology*, 117(5):1327–1349, November 2021.
- [4] Szilárd Aradi. Survey of Deep Reinforcement Learning for Motion Planning of Autonomous Vehicles, January 2020. arXiv:2001.11231 [cs, eess, stat].
- [5] Luc Le Mero, Dewei Yi, Mehrdad Dianati, and Alexandros Mouzakitis. A Survey on Imitation Learning Techniques for End-to-End Autonomous Vehicles. *IEEE Transactions on Intelligent Transportation Systems*, 23(9):14128–14147, September 2022.
- [6] Dean A. Pomerleau. ALVINN: An Autonomous Land Vehicle in a Neural Network. In *Advances in Neural Information Processing Systems*, volume 1. Morgan-Kaufmann, 1988.
- [7] Chang Liu, Seungho Lee, Scott Varnhagen, and H. Eric Tseng. Path planning for autonomous vehicles using model predictive control. In *2017 IEEE Intelligent Vehicles Symposium (IV)*, pages 174–179, June 2017.
- [8] Mohamed-Khalil Bouzidi, Bojan Derajic, Daniel Goehring, and Joerg Reichardt. Motion Planning under Uncertainty: Integrating Learning-Based Multi-Modal Predictors into Branch Model Predictive Control. Berlin, May 2024.
- [9] Yunlong Song and Davide Scaramuzza. Policy Search for Model Predictive Control With Application to Agile Drone Flight. *IEEE Transactions on Robotics*, 38(4):2114–2130, August 2022.

- [10] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, January 2016. Publisher: Nature Publishing Group.
- [11] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A Survey of Monte Carlo Tree Search Methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, March 2012. Conference Name: IEEE Transactions on Computational Intelligence and AI in Games.
- [12] Google DeepMind. AlphaGo - The Movie | Full award-winning documentary, March 2020. Accessed: Dec. 1, 2024. [Online]. Available: <https://www.youtube.com/watch?v=WXuK6gekU1Y>.
- [13] Bruno Brito, Achin Agarwal, and Javier Alonso-Mora. Learning Interaction-Aware Guidance for Trajectory Optimization in Dense Traffic Scenarios. *IEEE Transactions on Intelligent Transportation Systems*, 23(10):18808–18821, October 2022.
- [14] Peter Trautman and Andreas Krause. Unfreezing the robot: Navigation in dense, interacting crowds. In *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 797–803, October 2010. ISSN: 2153-0866.
- [15] Georg Schildbach, Matthias Soppert, and Francesco Borrelli. A collision avoidance system at intersections using Robust Model Predictive Control. In *2016 IEEE Intelligent Vehicles Symposium (IV)*, pages 233–238, June 2016.
- [16] Ali Mesbah. Stochastic Model Predictive Control: An Overview and Perspectives for Future Research. *IEEE Control Systems Magazine*, 36(6):30–44, December 2016. Conference Name: IEEE Control Systems Magazine.
- [17] O. de Groot, B. Brito, L. Ferranti, D. Gavrila, and J. Alonso-Mora. Scenario-Based Trajectory Optimization in Uncertain Dynamic Environments, March 2021. arXiv:2103.12517 [cs].
- [18] Martin Treiber, Ansgar Hennecke, and Dirk Helbing. Congested traffic states in empirical observations and microscopic simulations. *Physical Review E*, 62(2):1805–1824, August 2000. Publisher: American Physical Society.
- [19] Treiber, Martin. Longitudinal Traffic model: The IDM. Accessed: Nov. 20, 2024. [Online]. Available: <https://mtreiber.de/MicroApplet/IDM>.

- [20] Scott Ettinger, Shuyang Cheng, Benjamin Caine, Chenxi Liu, Hang Zhao, Sabeek Pradhan, Yuning Chai, Ben Sapp, Charles R. Qi, Yin Zhou, Zoey Yang, Aurélien Chouard, Pei Sun, Jiquan Ngiam, Vijay Vasudevan, Alexander McCauley, Jonathon Shlens, and Dragomir Anguelov. Large Scale Interactive Motion Forecasting for Autonomous Driving: The Waymo Open Motion Dataset. pages 9710–9719, 2021.
- [21] Shaoshuai Shi, Li Jiang, Dengxin Dai, and Bernt Schiele. Motion Transformer with Global Intention Localization and Local Movement Refinement, March 2023. arXiv:2209.13508 [cs].
- [22] Steven M. LaValle. *Planning Algorithms*. Cambridge University Press, 1 edition, May 2006.
- [23] Mohamed-Khalil Bouzidi, Yue Yao, Daniel Goehring, and Joerg Reichardt. Learning-Aided Warmstart of Model Predictive Control in Uncertain Fast-Changing Traffic, October 2023. arXiv:2310.02918 [cs, eess].
- [24] Bruno Brito, Boaz Floor, Laura Ferranti, and Javier Alonso-Mora. Model Predictive Contouring Control for Collision Avoidance in Unstructured Dynamic Environments, October 2020. arXiv:2010.10190 [cs].
- [25] Florian Siebenrock, Manuel Günther, and Sören Hohmann. LTV-MPC Based Trajectory Planning Considering Uncertain Object Prediction Through Adaptive Potential Fields. In *2020 IEEE Conference on Control Technology and Applications (CCTA)*, pages 666–672, August 2020.
- [26] Florent Altché, Philip Polack, and Arnaud de La Fortelle. High-speed trajectory planning for autonomous vehicles using a simple dynamic model. In *2017 IEEE 20th International Conference on Intelligent Transportation Systems (ITSC)*, pages 1–7, October 2017. ISSN: 2153-0017.
- [27] Oscar de Groot, Laura Ferranti, Darius Gavrila, and Javier Alonso-Mora. Globally Guided Trajectory Planning in Dynamic Environments. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 10118–10124, May 2023.
- [28] Shilp Dixit, Umberto Montanaro, Mehrdad Dianati, David Oxtoby, Tom Mizutani, Alexandros Mouzakitis, and Saber Fallah. Trajectory Planning for Autonomous High-Speed Overtaking in Structured Environments Using Robust MPC. *IEEE Transactions on Intelligent Transportation Systems*, 21(6):2310–2323, June 2020. Conference Name: IEEE Transactions on Intelligent Transportation Systems.
- [29] Andreas Philipp and Daniel Goehring. Analytic Collision Risk Calculation for Autonomous Vehicle Navigation. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 1744–1750, May 2019. ISSN: 2577-087X.

- [30] Richard S. Sutton and Andrew Barto. *Reinforcement learning: an introduction*. Adaptive computation and machine learning. The MIT Press, Cambridge, Massachusetts London, England, second edition edition, 2020.
- [31] David Silver. Lectures on Reinforcement Learning, 2015. Accessed: Nov. 30, 2024. [Online]. Available: <https://www.davidsilver.uk/teaching/>.
- [32] Alexander Weingart. Enhancing Sampling-Based Kinodynamic Motion Planning with Reinforcement Learning Policies. Master’s thesis, Technische Universität Berlin, August 2023.
- [33] Levente Kocsis and Csaba Szepesvári. Bandit Based Monte-Carlo Planning. In David Hutchison, Takeo Kanade, Josef Kittler, Jon M. Kleinberg, Friedemann Mattern, John C. Mitchell, Moni Naor, Oscar Nierstrasz, C. Pandu Rangan, Bernhard Steffen, Madhu Sudan, Demetri Terzopoulos, Dough Tygar, Moshe Y. Vardi, Gerhard Weikum, Johannes Fürnkranz, Tobias Scheffer, and Myra Spiliopoulou, editors, *Machine Learning: ECML 2006*, volume 4212, pages 282–293. Springer Berlin Heidelberg, Berlin, Heidelberg, 2006. Series Title: Lecture Notes in Computer Science.
- [34] Marc Toussaint and Wolfgang Hönig. Robot Learning Lecture, 2024. Technische Universität Berlin.
- [35] J. Andrew Bagnell. An Invitation to Imitation. 2015. Carnegie Mellon University.
- [36] Yisong Yue and Hoang M. Le. Imitation Learning Tutorial (ICML), 2018. Accessed: Dec. 5, 2024. [Online]. Available: <https://sites.google.com/view/icml2018-imitation-learning/>.
- [37] Stephane Ross, Geoffrey Gordon, and Drew Bagnell. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, pages 627–635. JMLR Workshop and Conference Proceedings, June 2011. ISSN: 1938-7228.
- [38] Guanya Shi. Introduction to Robot Learning, 2024. Accessed: Dec. 5, 2024. [Online]. Available: <https://16-831-s24.github.io/lectures>.
- [39] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [40] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization, January 2017. arXiv:1412.6980 [cs].
- [41] Andreas Wächter and Lorenz T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Pro-*

- gramming*, 106(1):25–57, March 2006.
- [42] Computational Mathematics Group. Harwell Subroutines Library (HSL). Accessed: Dec. 14, 2024. [Online]. Available: <https://www.hsl.rl.ac.uk/>.
- [43] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. Graph neural networks: A review of methods and applications. *AI Open*, 1:57–81, 2020.
- [44] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Russ R. Salakhutdinov, and Alexander J. Smola. Deep Sets. *Advances in Neural Information Processing Systems*, 30, 2017.